New Era University
College of Informatics and Computing Studies
Department of Information Technology

BrewManager: An Integrated Offline-First Mobile POS and Reactive Inventory Management System for Local Coffee Shops

An Undergraduate Capstone Project presented to the Faculty of New Era University, College of Informatics and Computing Studies, Department of Information Technology, New Era, Quezon City

In Partial Fulfilment of the Requirements for the Degree Bachelor of Science in Information Technology

Celajes, Ron Aedriel R.
Ramos, John Ashley R.
Costin, Angelic Pauleen

May 2026

TABLE OF CONTENTS

CHAPTER 1: THE PROBLEM AND ITS BACKGROUND 3
Introduction 3
Objectives of the Study 5
General Objective 5
Specific Objectives 5
Scope and Limitations 6
Scope 6
Limitations 9
Significance of the Study 11
Definition of Terms 12
CHAPTER 2: REVIEW OF RELATED LITERATURE 16
Evolution of POS and Offline-First Architectures in Small Retail 16
Automated Inventory Tracking, Recipe-Level Deduction, and Data Drift 18
Shelf-Life Monitoring and FEFO Algorithms in the Food Industry 20
Rule-Based Artificial Intelligence and Decision Support Systems 22
End-of-Shift Reconciliation, Cognitive Load, and Usability Evaluation 24
Synthesis of the Reviewed Literature 26
Conceptual Framework 28
Input 29
Process 30
Output 31
CHAPTER 3: TECHNICAL BACKGROUND 32
Project Description 32
System Development Life Cycle (SDLC) 35
Increment 1: System Foundation and Security Layer 35
Increment 2: Back-Office Configuration and FEFO Engine 36
Increment 3: POS, Expiry Monitoring, Reconciliation, and AI Layer 37
Increment 4: Analytics, Export, and System Evaluation 39
Environment 40
Locale 40
Population of the Study 41
Organizational Chart 42
Requirements Modeling 42
Input 43
Process 44
Output 45
Performance 46
Control 46
Requirements Specification 48

Operational Feasibility....48
System Flowchart....49
Site Map....50
User Journey Map....50
Schedule Feasibility....51
Economic Feasibility....52
Cost and Benefit Analysis....53
Cost Recovery Scheme....54
Data and Process Modelling....55
Entity Relationship Diagram....55
Data Dictionary....56
Context Diagram....61
Data Flow Diagram....62
Object Modelling....63
Use Case Diagram....64
Class Diagram....65
Sequence Diagram....66
Risk Assessment / Analysis....67
Risk 1: Data Loss Due to Hardware Failure....68
Risk 2: Inaccurate FEFO Routing Caused by Incorrect Device Clock....68
Risk 4: Incomplete Recipe Blueprints Leading to Incorrect Deductions....69
Risk 5: Staff Resistance to Adopting the New System....69
Risk 6: Volumetric Estimation Errors for Thick Liquids....70
System Design....70
User-Interface....71
Wireframes....71
Reports....72
User Inquiry / View....72
Print-out Report....73
System Development....74
Technical Feasibility....74
Minimum Requirements....75
Compatibility Checking....75
Relevance of the Technologies....76
References....78
Appendices....84

Chapter 1

THE PROBLEM AND ITS BACKGROUND

Introduction

The local coffee shop industry is growing quickly, making the food and beverage market highly competitive. Unlike large cafe chains that rely on expensive and complex cloud software, independent coffee shops usually operate with smaller budgets and simpler daily routines. A major part of their business involves managing items that spoil easily, such as milk, syrups, and coffee beans. Because of this, combining Point-of-Sale (POS) software with automated inventory tracking has become an important area of study in system development. To manage a cafe properly, owners need software that can accurately track raw materials, monitor expiration dates, and link daily sales directly to their backroom storage.

Even though modern software exists, many local coffee shops still track their supplies manually using paper logbooks, clipboards, or scattered Excel files, which are slow to update and prone to human error. Staff must manually count containers, estimate remaining liquid in open bottles, and check expiration dates one by one, adding extra work to daily operations. During busy rush hours, baristas prioritize speed and intuition, which leads to unrecorded spills, over-pouring, and unnoticed spoilage. Over time, the records in these manual systems drift away from the actual stock on shelves. This gap in data leads to

sudden shortages of ingredients, unnoticed waste, and potential food safety risks from expired products.

This progressive gap between recorded and physical stock, referred to in this study as inventory drift, is the central problem BrewManager addresses. Automating deduction alone does not solve it. Even a system that perfectly subtracts recipe quantities at every sale cannot observe what happens outside the transaction: the spilled shot, the over-poured cup, the bottle knocked off the counter, the batch quietly left to spoil. Software can track what was sold, but it cannot see what was lost. A system that only deducts will therefore appear accurate while drifting steadily further from reality, and the longer it runs unchecked, the more confidently it will report figures that are wrong.

Existing research addresses this gap primarily through hardware. Sensor-based and Internet-of-Things inventory systems achieve continuous physical verification, but their cost places them beyond the reach of the independent operators who need accuracy most. This leaves a practical gap between what the literature recommends and what a small cafe can actually deploy, and it is this gap that motivates the present study.

To address the problem of inventory drift without relying on specialized hardware or network infrastructure, this study proposes BrewManager, a mobile Point-of-Sale and reactive inventory management system for local coffee shops. The system operates on a single Android tablet using an embedded SQLite database, an offline-first architecture chosen deliberately because independent shops cannot guarantee stable connectivity and cannot afford for sales to stop

when a connection fails. Within this architecture, BrewManager implements a closed correction loop. Every completed sale automatically deducts the recipe quantity of each ingredient, routing the deduction to the earliest-expiring batch first under a First-Expired, First-Out rule. Because deduction alone cannot account for unrecorded physical loss, a structured end-of-shift audit allows staff to record actual remaining stock through a simple container count and fractional estimate, requiring no scale, no calculation, and no external device. The system computes the resulting variance, writes the correction back to the affected inventory batches, and thereby begins each new shift from a physically counted baseline rather than an accumulated estimate. A deterministic rule-based decision support module completes the loop by identifying ingredients approaching expiration and recommending the menu items that will consume them soonest, converting measured expiry risk into an actionable intervention at the counter.

By making inventory drift visible, measurable, and correctable within the daily workflow of a small cafe, BrewManager offers a low-cost alternative to hardware-dependent inventory accuracy, with the aim of reducing preventable waste, shortening end-of-shift administrative work, and supporting more reliable stock decisions for independent coffee shop owners.

Objectives of the Study

General Objective

To create a mobile Point-of-Sale and reactive inventory management system for independent coffee shops that automates ingredient-level deduction, detects and corrects the variance between recorded and physical stock through a simplified end-of-shift audit, and converts measured expiry risk into actionable recommendations, thereby reducing preventable material waste and manual inventory labor without requiring specialized hardware or network connectivity.

Specific Objectives

1. To develop an offline-first mobile Point-of-Sale module that stores recipe blueprints and automatically deducts correct ingredient quantities upon checkout. The system routes each deduction to the earliest-expiring available batch using First-Expired, First-Out (FEFO) logic with multi-batch splitting, while preserving inventory integrity through a Manager-authorized transaction reversal function.
2. To create a reactive inventory monitoring module that evaluates the shelf-life of every inventory batch against set expiry timeframes, presents the resulting status through color-coded visual alerts, and automatically restricts the availability of menu items whose required ingredients have been depleted.
3. To develop a simplified end-of-shift reconciliation process that measures inventory drift by comparing theoretical stock against a physical count recorded through container tallies and fractional estimation, and that

writes the resulting variance back to the affected inventory batches so that each subsequent shift begins from a verified stock baseline.

4. To implement a deterministic rule-based decision support module that ranks available menu items according to ingredient expiry risk and trailing sales popularity in order to prompt the consumption of near-expiry stock before spoilage occurs, and to record the acceptance of these recommendations for evaluation.

5. To evaluate the system in terms of deduction and reconciliation accuracy, end-of-shift administrative time, and usability, through testing and structured feedback from independent coffee shop staff and owners.

Scope and Limitations

Scope

This study focuses on the design, development, and evaluation of BrewManager, an offline-first mobile Point-of-Sale (POS) and reactive inventory management application. The application is developed exclusively for local, independent coffee shops and is designed specifically as a standalone application for Android tablet devices. To ensure uninterrupted daily operations without reliance on an internet connection, the system utilizes a localized SQLite relational database.

This scope is explicitly limited to a single-branch, single-device deployment and excludes multi-branch corporate dashboards, predictive

analytics, and cloud synchronization. The core functionalities covered in the scope of this study include:

- **Menu and Blueprint Configuration:** A back-office module that allows the manager to perform standard CRUD (Create, Read, Update, Delete) operations for menu items, raw ingredient profiles, and recipe blueprints. A recipe blueprint maps exactly how much of each ingredient is required for a specific beverage, such as the milliliters of milk and espresso in a latte. Records that are referenced by completed transactions are archived instead of permanently deleted, which protects the accuracy of historical sales and deduction data.
- **Inventory Receiving and Batch Registration:** A receiving module in which the manager records incoming stock as inventory batches. Independent coffee shops acquire ingredients through varied channels, including supplier deliveries and retail purchases. As a result, containers of the same ingredient entering the shop at the same time often carry different expiration dates. The system therefore defines an inventory batch by expiration date rather than by receiving event, so a single receiving session can create several batch records. For each combination of expiry date and container size found in a purchase, the manager records the number of containers with that date and the capacity of a single container. Container capacity is stored on the batch rather than on the ingredient, which allows the same ingredient to be held in different

container sizes at the same time. Expiration dates are never grouped or rounded.

- **Sales and Recipe Mapping Engine:** A POS module that processes customer transactions and generates on-screen digital order summaries (e-receipts) instead of printed paper receipts. At checkout, the module automatically deducts the recipe amount of each ingredient using a strict First-Expired, First-Out (FEFO) and multi-batch splitting algorithm. When a transaction requires more than the earliest-expiring batch holds, the engine empties that batch and carries the remainder to the next batch in expiry order until the full amount is satisfied. The module also includes a real-time “declare container lost” override on the cashier screen for mid-shift accidents, such as dropping a bottle. Staff select the affected batch and indicate whether the loss was of sealed containers or of an opened one, together with how many. The system deducts the corresponding quantity, so that a partly used container is not recorded as a full one. The system deducts that amount from the batch and logs it as a recorded loss event, separate from the unexplained variance measured during reconciliation. If the deduction empties the batch, FEFO routes the next order to the next available batch. If no stock remains for an ingredient, all dependent menu items are grayed out. To protect data accuracy when an order is recorded by mistake, the module includes a Manager-authorized void function that reverses a completed transaction and returns each deducted amount to the exact batches it came from.

- **Inventory and Expiry Monitoring:** A tracking matrix that checks the shelf-life of every active inventory batch against the device date and shows color-coded alerts to warn staff of nearing expirations. Each batch is also given a normalized expiry risk score, which is passed to the decision support module.
- **End-of-Shift Reconciliation:** An audit module that measures inventory drift by comparing the system's theoretical stock against a physical count submitted by staff, then applies the correction back to inventory records. Reconciliation is done per batch. Each active batch is audited through two inputs: a count of sealed, unopened containers, and the amount left across all opened containers of that batch, entered in quarter-fraction steps of the batch's registered container capacity. Because the fraction control does not stop at one full container, staff report one combined value no matter how many containers are open. Each control starts at the value entered during the previous shift. Before the audit can be submitted, every batch must be marked as checked, either by adjusting a control or by confirming that the quantity is unchanged, so that a value carried over from the previous shift cannot be recorded as a counted quantity without having been verified. The system multiplies the submitted values by the batch container capacity to get the audited quantity, computes the variance against the theoretical quantity, and writes the corrected value directly to the batch. Each shift therefore begins from a verified stock level rather than an estimate that has drifted. Theoretical quantities stay hidden

from staff until the audit is submitted. If a submitted count is higher than the total containers recorded for a batch, the system asks for confirmation and flags it as possible unrecorded stock receiving. If two batches of the same ingredient return variances that cancel each other out, the system flags this as a likely break in expiry-order stock rotation.

- **Decision Support and Waste Prevention:** A deterministic, rule-based algorithm that identifies ingredients close to expiring and determines which available menu items will use them up fastest. These appear as ranked suggestions on the cashier screen, shown together with the expiry condition that triggered them. Menu items are ranked using a fixed weighted scoring matrix that combines the highest expiry risk score among an item's ingredients with its recent sales popularity. Popularity serves as a measure of whether the item is likely to actually sell within the remaining shelf-life. Every suggestion and whether it was acted on is recorded, so the module's effect on spoilage can be evaluated.
- **Descriptive Analytics:** A back-office dashboard that shows material loss over time, drawn from reconciliation logs and filterable by date range and ingredient, so the manager can identify ongoing sources of waste.
- **Manual Data Export and Database Backup:** Because the system runs offline with no cloud synchronization, it provides two ways to preserve data. For review, the manager can export sales, inventory, and reconciliation reports as CSV or PDF files to the tablet's internal storage. For data recovery, a one-tap function copies the complete SQLite

database file to external or user-selected storage, and a matching restore function loads a saved database back onto a device. Unlike the report exports, the database copy holds the full state of the system and can be used to rebuild operations after device loss or failure.

- **User Access Control (Permission Matrix):** A single-device PIN-based authentication system that separates user roles. Baristas are given access only to POS operations and blind end-of-shift audit inputs. CRUD configuration, inventory receiving, transaction voiding, and the analytics dashboard are reserved for the Manager account.

Limitations

While the system is designed to improve operational efficiency, the study and the software are subject to the following limitations:

- **Architectural and Hardware Constraints:** The offline-first, single-device design excludes cloud hosting, multi-branch syncing, and remote access. Because backup is started by the user rather than run automatically, data protection depends on the manager backing up regularly. Any activity recorded after the last backup cannot be recovered if the tablet is damaged, lost, or reset. The system does not connect to Bluetooth or IoT devices, including receipt printers. Expiry tracking and FEFO routing also depend entirely on the device's internal clock. The system warns the user when the clock appears to have moved backwards compared to earlier activity, but it cannot check the correct time without a network connection, so a wrongly set clock will reduce routing accuracy.

- **Reconciliation Resolution:** The audit records remaining stock in quarter-container steps rather than exact measured amounts. This is an intentional design choice. It removes any need for weighing equipment, tare weight setup, or density conversion, and applies one consistent method across liquids, thick syrups, and granular solids. The resulting resolution is one quarter of a container per batch per shift. Because the module is meant to detect drift building up across shifts rather than to measure single servings, this resolution is enough to identify ongoing material loss at the level where purchasing and portioning decisions are actually made. It is not enough to measure the variance of any single transaction, and estimation error is expected to rise slightly when several containers of the same batch are open at once.
- **Dependence on Accurate Receiving Entry:** All expiry monitoring, FEFO routing, and reconciliation calculations come from the batch records the manager creates at receiving. If an expiration date is misread or mistyped, a container capacity is entered wrongly, or a purchase is not recorded at all, that batch will be tracked inaccurately for as long as it exists. The system flags counts that exceed recorded holdings as possible unrecorded receiving, but it cannot detect a batch that was never entered, and it cannot check an entered expiry date against the physical container.
- **Batch Identification in Physical Storage:** Because batches are defined by expiration date, containers of the same ingredient belonging to different

batches must be told apart on the shelf. The system depends on staff distinguishing containers belonging to different batches. This is supported at two levels: containers are labeled with their batch identifier at receiving, and stock is arranged in storage so that earlier-expiring batches are positioned for first access. Where labeling is left out or storage order is not maintained, staff may audit containers under the wrong batch, producing inaccurate batch-level variance even when the total ingredient quantity is correct.

- **Functional and Analytical Boundaries:** The system tracks materials only. It does not handle financial accounting, tax tracking, or employee payroll. It is built for Android tablets and will not run on iOS or web browsers. The system calculates how much material was lost, but it cannot determine why. It cannot tell the difference between spillage, over-pouring, and theft. It detects breaks in expiry-order rotation only when two batches of the same ingredient show variances that cancel each other out. A rotation break that happens alongside material loss of a similar size may not be distinguishable from ordinary waste.
- **Beverage-Centric Architecture and Food Menu Constraints:** The ingredient model, FEFO logic, and reconciliation method are designed for beverage materials such as milk, syrups, and powders held in countable containers of known capacity. The system is not built to fully track prepared food items such as assembled sandwiches or hot meals. These items can still be sold through the POS as whole units, but their

preparation, portioning, and waste are not tracked with the same ingredient-level detail as beverages.

- **Beverage Customization Constraints:** The system does not handle per-order modifiers such as ingredient substitutions, add-ons, or sweetness and strength adjustments. Each menu item deducts one fixed recipe blueprint. Where staff accommodate a verbal customization at the counter without a corresponding menu item, the deduction will not match what was actually poured, and the difference will appear as batch-level variance during reconciliation. Customizations that reduce ingredient usage will register as unexplained surplus, while those that increase it will register as additional loss. Any customization a shop wishes to track accurately must be configured as a separate menu item.
- **Static Decision Support Adaptability:** The decision support module uses a fixed, rule-based weighted scoring matrix. It does not use machine learning or predictive forecasting, so it cannot learn from or adjust to changing seasonal sales patterns on its own.
- **Evaluation Generalizability:** The evaluation of this system is limited to independent coffee shops handling perishable beverage materials. Findings on usability and effectiveness may not apply directly to large franchise operations or other retail types, and testing is limited to the timeframe of the capstone development cycle.

Significance of the Study

The development of the BrewManager system aims to modernize the operational workflows of independent local coffee shop establishments. Specifically, this study will be highly beneficial to the following groups:

Coffee Shop Owners and Management. The system replaces error-prone paper and spreadsheet tracking with automated ingredient-level monitoring. Automated deduction alone cannot account for losses that occur outside a sale, so the system pairs it with a structured end-of-shift audit that measures the gap between recorded and physical stock and corrects the records to match. This gives owners a stock figure that is re-verified every shift rather than one that drifts further from reality over time. Material loss is recorded per ingredient and charted over time, allowing owners to identify which ingredients are consistently lost and to act on patterns that manual tracking leaves invisible. The offline-first local database keeps the POS running during internet outages, while the manual database backup allows records to be preserved and restored if the device fails. The PIN-based permission matrix protects recipes and inventory settings from unauthorized edits and reserves transaction voiding for managerial approval, so sales and inventory records cannot be quietly altered by front-of-house staff.

Baristas and Cafe Staff. By deducting recipe amounts automatically during POS transactions, the system removes the need for manual stock tracking during busy shifts. At closing, the audit asks for only two inputs per batch: a count of sealed containers and the amount left in the opened ones. There is no weighing, no unit conversion, and no arithmetic. Because the fraction control accepts a single

combined value for all opened containers, the number of inputs stays the same regardless of how many containers are in use, and each control starts at the previous shift's value so most entries require only a small adjustment. This directly reduces the cognitive load and administrative stress placed on staff at the end of a shift, allowing baristas to focus on service speed and beverage quality.

Customers. Customers benefit from more consistent product safety and taste. FEFO routing, proactive expiry alerts, and the decision support module work together to ensure that ingredients closest to expiring are used or sold before they spoil, reducing the risk of expired materials reaching a drink. The reactive interface also prevents staff from accepting orders that cannot be fulfilled with available stock, ensuring reliable and transparent service.

Future IT Researchers and Systems Developers. This study provides a reference architecture for closing the gap between recorded and physical inventory in small retail environments without specialized hardware. Existing research addresses this gap largely through sensor-based and Internet-of-Things systems, which remain financially out of reach for independent operators. BrewManager demonstrates that a software-only approach, combining automated deduction with a structured physical audit that writes corrections back to batch records, can maintain inventory accuracy on a single offline device. It further demonstrates that deterministic, rule-based decision support can convert measured expiry risk into a practical intervention without the data requirements of predictive machine learning, offering a model for similar low-resource operational settings.

Definition of Terms

Automated Recipe Mapping Engine. The background process triggered at checkout that reads the recipe blueprint of each ordered item, determines the required quantity of every component ingredient, and issues the corresponding deductions against inventory batches according to expiry order.

Reactive Inventory Management. An inventory approach in which recorded stock levels respond automatically to both recorded sales and verified physical counts, so that the system adjusts its own records in response to measured conditions rather than relying solely on assumed deductions.

Blind Audit. A control applied during end-of-shift reconciliation in which the system withholds its theoretical inventory quantity from staff until their physical count has been submitted and locked, preventing the submitted figure from being adjusted to match the system's expectation.

Container Capacity. The full volume or weight held by a single unopened container of an ingredient, recorded at the batch level so that the same ingredient may be stocked in differing container sizes simultaneously, and used to convert audit inputs into measured quantities.

Inventory Drift. The progressive variance between a system's recorded theoretical inventory and the actual physical stock available, typically resulting from unrecorded events such as micro-spills, over-pouring, or missed manual entries accumulated over time. Also referred to as data drift.

Declare Container Lost. A point-of-sale override enabling staff to record a mid-shift physical loss, such as a dropped bottle, by specifying the affected batch

and the number of containers lost in quarter-container steps. The amount is deducted from the batch and logged as a recorded loss event, separate from the unexplained variance measured during reconciliation.

Decision Support Module. A deterministic, rule-based component employing a fixed weighted scoring matrix to identify ingredients approaching expiration and rank the available menu items that would consume them soonest, presenting these as prioritized suggestions on the cashier screen together with the expiry condition that triggered them.

Descriptive Analytics Dashboard. A back-office reporting interface reserved for the Manager that charts material loss per ingredient over time, derived from reconciliation logs and filterable by date range, enabling review of measurable ingredient waste and identification of recurring loss patterns.

End-of-Shift Reconciliation. The structured audit conducted at the close of each operational shift, in which staff submit a physical count for every active inventory batch, and the system computes the variance against its theoretical quantity and writes the corrected value back to the batch record.

First-Expired, First-Out (FEFO). An inventory management rule requiring that ingredient batches be depleted in order of expiration date, with multi-batch splitting applied automatically when a transaction's requirement exceeds the quantity remaining in the earliest-expiring batch. Where two batches share an expiration date, the batch holding the smaller remaining quantity is depleted first.

Fractional Input. An interface control through which staff record the amount remaining across all opened containers of a batch, expressed in

quarter-container increments. The control is not limited to a single container, so one combined value is submitted regardless of how many containers are open, and each control is initialized at the value recorded in the previous shift.

Inventory Batch. A database record representing a quantity of an ingredient sharing a single expiration date and a single container capacity, holding the number of containers received, the capacity of one container, and the quantity remaining. A single receiving session may produce several batches where expiry dates or container sizes differ.

Offline-First Architecture. A software design in which the application operates entirely on a locally embedded SQLite database within a single Android tablet, making internet connectivity and cloud synchronization non-essential to daily transactional and inventory operations.

Point-of-Sale (POS). The front-of-house module through which baristas process customer orders, record sales transactions, and initiate automated background deduction of ingredient quantities at checkout.

Proactive Expiry Monitoring Matrix. A color-coded interface that evaluates the shelf-life of every active batch against the device date, flags batches according to defined expiry timeframes, and supplies a normalized expiry risk score to the decision support module.

Recipe Blueprint. A configuration record mapping a menu item to the exact quantity of each ingredient required to prepare one unit of that item, forming the basis for automated deduction at checkout.

Suggestion Acceptance. The recorded outcome of a decision support recommendation, indicating whether the suggested menu item was subsequently sold, used to evaluate the module's effectiveness in prompting the consumption of near-expiry stock.

Theoretical Quantity. The quantity of an ingredient the system calculates should remain in a batch, based on its received amount and all recorded deductions, prior to verification against a physical count.

Transaction Void. A Manager-authorized reversal of a completed sale that removes the transaction from sales records and returns each deducted quantity to the specific inventory batches from which it was originally taken.

User Access Control (Permission Matrix). A PIN-based, single-device authentication system enforcing role separation, restricting baristas to point-of-sale operations and blind audit submission while reserving configuration, receiving, transaction voiding, and analytics for managerial accounts.

Volumetric Variance. The measured difference between a batch's theoretical quantity and the physical quantity recorded during end-of-shift reconciliation, representing material loss not accounted for by recorded sales.

Chapter 2

REVIEW OF RELATED LITERATURE

This chapter presents an evaluated, organized, and synthesized collection of recent academic literature. The selected journal articles and studies serve as the theoretical foundation for the development of BrewManager. Rather than reviewing technologies one by one, the review is arranged as a problem chain. It first establishes that recorded inventory drifts away from physical stock in small food service operations. It then examines each technical response in turn, showing what each one solves and what it leaves unsolved. This structure locates the specific gap this study addresses: correcting accumulated inventory drift in operations that cannot afford the hardware or staff time that existing solutions assume. The review is organized thematically and aligned to the specific objectives of this study.

Inventory Drift in Small Food Service Operations

Every inventory system assumes its records match what is physically on the shelf. In practice that assumption fails, and it fails gradually. The gap between recorded and physical stock is well documented in the operations literature, where it is called inventory record inaccuracy. It does not come from one large failure. It builds up from many small unrecorded events during normal daily work.

Destro, Staudt, Somensi, and Taboada (2023) modeled this build-up using system dynamics simulation. They found that even a small undetected source of inaccuracy can accumulate over time and eventually destabilize the whole inventory system. Their work also shows why the timing of correction matters. Where stock is verified only through occasional full counts, error keeps accumulating in the records until the next count happens. The gap between counts therefore decides how far records are allowed to drift before being brought back to reality.

The problem is worse for perishable goods. Rekik, Oliva, Glock, and Syntetos (2026) analyzed around twenty-four thousand stock keeping units across eleven grocery stores. They found that inventory record inaccuracy is higher for items that are restocked often, held in larger quantities, and are perishable. Perishable items drift more than durable ones. This applies directly to a coffee shop, where the ingredients that matter most, such as milk, syrups, and dairy alternatives, are exactly the perishable, frequently restocked items the literature identifies as most at risk.

In smaller operations the problem is made worse by how records are kept. Neve and Schmidt (2021) modeled point-of-use inventory systems and showed that recorded and actual stock separate over time because usage is never captured perfectly. They also found that the labor needed to correct this gap becomes a barrier to correcting it at all. Gong and Hui (2024) observed something similar, noting that manual recording introduces calculation errors that

real-time data processing removes, which reduces how clearly a business can see its own operations.

Together these studies establish the problem this study addresses. Records drift from physical reality. The drift builds up rather than settling. Perishable stock drifts fastest. And the work of correcting it is itself an obstacle to getting it corrected. This is the situation independent coffee shops operate in, and it is what BrewManager is designed to address.

Offline-First POS Architectures in Small Retail

Small food businesses operate differently from large chains. They do not have dedicated IT teams or stable network infrastructure, which means the type of point-of-sale system they use is not just a technical choice but an operational one. If the system fails because the internet drops, the entire business stops. This makes the architecture of a POS system a critical concern for small independent shops.

Several studies have examined how POS systems perform in small food business environments. Divina, Olan, Perez, Sarmiento, Acepcion, and Cabututan (2025) developed a web-based POS with integrated forecasting. They found that while the system improved operational speed, they strongly recommended offline capabilities so the system would stay usable during connectivity problems. Natividad, Musngi, Molinyawe, Rivera, Torres, and Moreno (2024) supported this, observing that while small food businesses are open to cloud-based POS systems, their success depends heavily on a stable

internet connection that many independent shops cannot guarantee. On the device level, Sari and Purba (2021) showed that an Android cashier application using a local SQLite database can handle sales and inventory tracking without an external server, making it a realistic and affordable option for small business owners.

These findings point to offline-first architecture, which is the idea that an application should be built to work without the internet rather than simply cope when it is missing. Pothineni (2024) defines this approach as treating network access as optional and relying on local storage to keep the experience consistent regardless of connectivity. Doodala (2023) applied this in an Android application for field operations, using modular caching to prevent data loss when the device went offline. Ramidi (2022) demonstrated the same approach in a mobile health context, using local storage and timestamp versioning to keep data collection running in low-bandwidth environments.

What this literature establishes is that offline operation is already a solved problem. Local storage on a single Android device is enough to run sales and inventory work without a server, and the approach is well understood. BrewManager therefore treats offline-first architecture as a settled design decision rather than a contribution, using a standalone Android application with an embedded SQLite database so that all transactions and ingredient deductions are processed on the device itself. What this architecture does not address is whether the records it keeps stay accurate. An offline system calculates stock just as reliably as an online one, and drifts from physical reality just as steadily.

Automated Recipe-Level Deduction and the Limits of Sales-Based Tracking

If manual record-keeping causes drift, automation is the obvious answer, and the literature consistently identifies it as such. Ugbebor, Adeteye, and Ugbebor (2024) found that automated inventory systems significantly improve stock accuracy and reduce shortages by replacing error-prone manual records with continuous digital updates. Gong and Hui (2024) made a similar argument, stating that moving from manual recording to real-time data processing reduces calculation errors and improves how clearly a business can see its operations. In the food and beverage industry specifically, Geminarqi and Purnomo (2023) identified process automation and intelligent data analysis as key factors in improving efficiency, linking technology adoption directly to better service accuracy. Salih, Ghazi, and Aljanabi (2023) added that automated inventory tracking in medium-sized operations reduces holding costs, though they also noted that structured staff training is needed to move workers away from paper-based habits.

Applied at the ingredient level, this principle produces recipe-based deduction. Each completed sale automatically subtracts the standard amount of every ingredient from recorded stock. This removes manual entry from the transaction entirely and keeps a continuously updated theoretical stock level without staff doing anything.

There is a limit to what this achieves, and it is a limit of principle rather than of how well the code is written. Automated deduction works from recorded sales. It can only account for material that a sale accounted for. Losses that

happen outside a sale are invisible to it, no matter how accurate the deduction logic is. Neve and Schmidt (2021) describe exactly this condition as imperfect usage capture, and show that the resulting gap persists and grows. A deduction engine that subtracts recipe amounts perfectly at every sale will still report figures that become steadily more wrong, because spillage, over-pouring, dropped containers, and spoilage produce no sale to deduct from.

This matters for how automation should be understood. Automated deduction removes one source of drift, which is manual entry error, and leaves another untouched. Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) point to this limit indirectly. For perishable inventory in small and medium enterprises, they recommend continuous physical sensing rather than relying on sales records alone, though they note that the upfront cost of such systems often discourages small operators from adopting them.

BrewManager uses automated recipe-level deduction as the foundation of its inventory tracking. The Automated Recipe Mapping Engine runs at every checkout and calculates theoretical stock with no manual input. The system treats this figure as a theoretical quantity rather than an accurate one, which is a direct response to the limit this literature identifies. It is paired with a physical verification process covered later in this review.

Expiry-Driven Stock Routing in Perishable Inventory

Managing perishable ingredients is one of the highest-stakes parts of running a food service business. When stock rotation is handled poorly, the

consequences go beyond financial loss. Expired ingredients can cause food safety violations, damage a shop's reputation, and in serious cases put customers at risk. The literature consistently shows that manual shelf-life tracking is not reliable enough to prevent these outcomes, and that an algorithmic approach to expiry routing is needed to meaningfully reduce spoilage.

Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) found that digital inventory monitoring in small and medium enterprises significantly improves shelf-life performance and reduces inventory costs. They also noted that the upfront cost of such systems often discourages small operators from adopting them. Herron, Garner, Siddique, Huang, Campbell, Rao, and Morey (2022) argued specifically for First-Expired, First-Out routing over the more traditional First-In, First-Out method. They demonstrated that prioritizing ingredients by expiry date produces much greater reductions in food waste than simply using the oldest delivery first.

This distinction matters in practice. First-In, First-Out assumes that the order stock arrives matches the order it expires. That assumption holds for a single wholesale supplier delivering uniform lots. It does not hold where a shop buys through several channels, since a newer purchase often expires sooner than an older one. Routing by expiration date rather than by arrival date is therefore not a refinement of stock rotation but a correction to it.

The effectiveness of FEFO at the system design level has been demonstrated across multiple studies. Carbajal-Vásquez, Piscoya-Alvites, Quiroz-Flores, García-López, and Nallusamy (2023) integrated FEFO protocols

into a food production facility and reported a 65.57 percent reduction in deteriorated products by explicitly prioritizing items closest to expiration. Hietasari, Subianto, and Adi (2024) proposed warehouse management frameworks that automatically enforce FEFO by dynamically sorting storage locations based on expiry timestamps. Suárez, Guaño, Pérez, and Roa-López (2024) formalized this further through a multi-objective FEFO dispatch model that ensures only ingredients meeting strict quality and time thresholds are used in production. At the restaurant level, Taufiqqurrachman, Nurrohmah, Warno, and Kusumawati (2025) built an Android-based expiration alert system using FEFO methods, showing that food service staff could detect near-expiry items and maintain food safety standards without centralized warehouse infrastructure.

These studies confirm that FEFO routing and proactive shelf-life alerts are proven tools for reducing spoilage. A common limitation across most of these frameworks, however, is that they assume centralized warehouses, IoT hardware, or complex optimization software. None of these reflect the reality of a small independent coffee shop. A second limitation is more fundamental. FEFO governs which batch the system deducts from, but it cannot ensure that staff physically take stock from that batch. The routing rule is enforced in software while the actual withdrawal happens by hand, and the literature reviewed here does not address what happens when the two diverge.

BrewManager adapts these validated FEFO principles into a lightweight offline environment. A Proactive Expiry Monitoring Matrix visually alerts staff when stock is approaching expiration, addressing the food safety goals identified

by Taufiqqurrachman, Nurrohmah, Warno, and Kusumawati (2025). FEFO is enforced at every checkout through a multi-batch splitting algorithm that routes ingredient deductions to the earliest-expiring active batch, with no external hardware or centralized warehouse required. Because the system cannot enforce which container staff physically open, it treats deviation from expiry-order withdrawal as a condition to be detected rather than assumed away, which is addressed in the following section.

Physical Verification and the Correction of Accumulated Drift

The previous sections establish a boundary. Automated deduction cannot see losses that happen outside a sale. Expiry routing cannot check what staff physically take off a shelf. Both work from records. Neither observes the shelf. Closing the gap between recorded and physical stock therefore needs a step that no amount of software logic can provide. Someone has to look.

The operations literature treats this as standard practice. Physical verification takes two broad forms. A full inventory count checks all items at once, usually rarely. Cycle counting checks small groups of items continuously on a rotating schedule. Destro, Staudt, Somensi, and Taboada (2023) compared these through simulation. They found that relying on infrequent full counts lets error build up in the records until the next count corrects it. Cycle counting reduced inaccuracy in every warehouse category they modeled. It also required a dedicated team of counters, and the number needed changed depending on how badly the operation was already performing. Their results discourage using

an annual full count as the only correction measure, while showing that continuous counting carries a real and ongoing labor cost.

The benefit of correction has been measured directly. Rekik, Oliva, Glock, and Syntetos (2026) ran a field quasi-experiment across grocery stores. They found that performing an inventory audit produced an eleven percent store-wide sales lift in the months after the count. The effect was not spread evenly. The entire lift came from items where system inventory was higher than actual inventory, meaning items the system thought were in stock when they were not. The benefit was also stronger for perishable items, which the same study found carry higher inaccuracy to begin with. Physical verification therefore gives the most value in exactly the conditions this study deals with.

Research has also looked at how counting should be organized. DeHoratius, Holzapfel, Kuhn, Mersereau, and Sternbeck (2023) compared several procedures for deciding which items in a retail store should be counted and in what order. Their aim was to detect both inventory record inaccuracy and unknown out-of-stocks. Their work shows that deciding what to count is a design decision with measurable consequences, not simply a matter of counting whatever is convenient.

Discrepancies do not all point the same way, and the difference matters when reading a count. Best, Glock, Grosse, Rekik, and Syntetos (2022) studied positive inventory discrepancies, meaning cases where more stock is physically present than the system records. Using a simulation of a retail store with several error-prone processes, they found that misplacement of items was one of the

main causes. This matters for batch-level tracking. Where stock of the same item is held under separate records, taking material from the wrong record creates a surplus in one and a shortfall in the other, even though nothing was actually lost.

Two limits run through this body of work when it is read against the conditions of a small cafe. The first is labor. Every correction method reviewed assumes staff who can be assigned to counting. Cycle counting needs dedicated counters, and Destro and colleagues quantify how many. A cafe running with one or two staff on shift has no such capacity, and Neve and Schmidt (2021) identify this labor requirement as a barrier to accuracy on its own. The second is hardware. The alternative most of this literature points to is sensing equipment. Maheshwari and colleagues recommend continuous physical sensing for perishable inventory in small and medium enterprises, while noting that its cost discourages the operators who need it most. Ugbebor, Adeteye, and Ugbebor (2024) likewise treat IoT integration as the route to real-time accuracy.

This is the gap this study addresses. The drift problem is documented and measured. Perishable stock is known to drift fastest. Physical verification is known to correct it and to produce measurable benefit. What the literature does not offer is a correction method sized for an operation that can afford neither counting staff nor sensing hardware.

BrewManager answers this with a structured end-of-shift reconciliation built around that constraint. Instead of counting selected items on a rotating

schedule, it checks every active batch at the close of each shift. The interval that Destro and colleagues identify as governing accumulated error is therefore reduced to a single shift. Instead of requiring measurement equipment, it records remaining stock as a count of sealed containers plus a fractional estimate of what is open. This needs no scale, no unit conversion, and no arithmetic from staff. The measured variance is written back to the affected batch record, so each shift starts from a physically counted quantity rather than an estimate that has been drifting since the last correction. Because reconciliation is done per batch rather than per ingredient, offsetting variances of the kind Best and colleagues link to misplacement stay visible, which allows deviation from expiry-order withdrawal to be detected instead of cancelling out.

Deterministic Decision Support in Low-Data Environments

Detecting and correcting drift tells an operator what has already been lost. Preventing loss requires acting on stock before it expires. This is a decision problem, and not all decision support approaches suit the environment of a small cafe. Some rely on large datasets and probabilistic models that learn over time. Others use clearly defined rules and scoring criteria. Where data is limited, decisions must be made quickly, and staff need to understand why a suggestion appeared, the second approach is more appropriate. The literature supports this distinction and validates rule-based scoring in localized recommendation settings.

Gupta, Modgil, Bhattacharyya, and Bose (2021) distinguished AI systems by their underlying logic. They argued that deterministic rule-based methods remain effective and appropriate alongside modern machine learning, particularly where limited data or the need for transparency makes probabilistic models impractical. Van der Waa, Nieuwburg, Cremers, and Neerincx (2021) examined the transparency advantage directly. They found that rule-based explanations in a decision support system let users see clearly which factors drove a recommendation. This increases user trust, which matters in fast-paced retail where staff must act on suggestions quickly without second-guessing the system.

In practical recommendation scenarios, multi-criteria decision-making frameworks have proven reliable for ranking options without opaque models. Çalı and Yilmaz Balaman (2019) used criteria weighting to rank alternatives in the hospitality sector and showed that explicit scoring can handle complex, multi-variable business data. Paradowski, Shekhovtsov, Bączkiewicz, Kizielewicz, and Sałabun (2021) analyzed objective weighting methods in multi-criteria decision support and demonstrated that the choice of weighting directly affects which criteria get prioritized. Their work underlines the importance of selecting a transparent weighting approach suited to the specific business context, and of being able to justify it. Shu, Carrasco, Sánchez-Montañés, and Portela García-Miguel (2024) extended these principles into the food and beverage industry by generating restaurant rankings through explicit demand variables rather than hidden representations.

What these studies show collectively is that rule-based scoring can produce useful, trustworthy decision support without large datasets or heavy computation. What they do not address is the specific decision this study faces, which is not what a customer is most likely to want but what the shop most urgently needs to sell before it spoils. Ranking for spoilage prevention inverts the usual objective of a recommendation system.

BrewManager applies these principles through a localized decision support module that runs a deterministic weighted scoring formula entirely on the tablet. The formula ranks available menu items using two explicit variables: the highest expiry risk among an item's required ingredients, and how often the item has sold over the trailing seven days. The purpose of the module is waste prevention rather than sales maximization. Expiry risk therefore carries the greater weight, while recent popularity serves as a feasibility measure of whether an item is likely to sell within the remaining shelf-life. An urgent ingredient in a rarely ordered drink will not be consumed in time regardless of how urgent it is. Because the logic is fully deterministic, the system behaves predictably and its suggestions can be explained to staff, and each suggestion is presented together with the expiry condition that triggered it. Following Paradowski and colleagues on the significance of weighting choice, the weighting applied is stated explicitly and justified by the purpose of the module rather than left implicit, and is implemented as a configurable parameter rather than a fixed constant. Each suggestion and its outcome is recorded so the module’s effect on spoilage can be measured.

Cognitive Load and System Evaluation in Operational Settings

The accuracy of any inventory system depends on whether the people using it can complete its procedures correctly. This applies with particular force to a physical audit, which is performed at the end of a shift by staff who are already tired. If the reconciliation workflow is too complicated, too slow, or too mentally demanding, staff will make mistakes, skip steps, or avoid it entirely. When that happens, the accuracy gained through automation is cancelled out by human error at the audit stage. Usability is therefore not a presentation concern in this system but a condition of the correction loop working at all.

Dewi, Dantes, and Divayana (2023) found that poor usability, reflected in low System Usability Scale scores, directly causes more user errors during inventory operations. These errors increase the reconciliation workload and undermine the accuracy that automation is meant to provide. The cognitive workload research adds a further layer. Patel and Naini (2025) conducted a field study in a small restaurant comparing fragmented manual tools against an integrated management platform, and observed significant reductions in both cognitive workload and error rates after staff switched to the integrated system. Kosch, Karolus, Zagermann, Reiterer, Schmidt, and Woźniak (2023) surveyed cognitive workload measurement in human-computer interaction and argued for workload-aware designs that actively support users performing time-sensitive, multi-step tasks. A closing-shift inventory audit is exactly that kind of task.

When systems are designed with usability in mind, outcomes improve measurably. Wulandari, Rahman, Fitriyanto, Dewanto, and Munir (2026)

developed a web-based inventory system evaluated against ISO 25010 quality attributes including functional suitability, reliability, performance efficiency, and usability, and achieved strong SUS scores alongside improved workflow efficiency. Kurniawan, Arrozi, Endika, and Afdhal (2026) applied the SUS framework to a web-based POS system and demonstrated that standardized usability evaluation produces quantifiable evidence of how well a system works, while also revealing differences in how staff in different roles learn and adapt to it.

These evaluation frameworks measure whether a system is usable and whether it meets its functional specification. They do not, on their own, establish whether the figures a system produces are correct. For a system whose central claim concerns inventory accuracy, usability results and functional conformance are necessary but not sufficient evidence. Establishing that claim requires measuring the system's output against independently known quantities.

BrewManager responds to the cognitive load findings of Patel and Naini (2025) and Kosch and colleagues (2023) by reducing the closing audit to two inputs per batch, a count of sealed containers and a fractional estimate of what remains open, with no weighing, unit conversion, or arithmetic required of staff. Each control is initialized at the previous shift's value so that routine closings require only small adjustments. Following the evaluation approaches of Dewi, Dantes, and Divayana (2023) and Wulandari and colleagues (2026), the system is assessed using ISO 25010 and the System Usability Scale. These are supplemented by controlled accuracy testing against known starting quantities

and by a comparison of closing procedure duration before and after deployment, so that the system's accuracy and efficiency claims are supported by measurement rather than by user perception alone.

Synthesis of the Reviewed Literature

Taken in order, the reviewed literature forms a single line of reasoning that leads to the design of BrewManager.

The starting condition comes from Destro, Staudt, Somensi, and Taboada (2023), Rekik, Oliva, Glock, and Syntetos (2026), and Neve and Schmidt (2021). Recorded inventory drifts away from physical stock. The drift builds up rather than settling. Perishable and frequently restocked items drift fastest. And the labor of correcting it is itself a barrier to correction. These are the conditions independent coffee shops work under.

The first response is architectural. Divina, Olan, Perez, Sarmiento, Acepcion, and Cabututan (2025), Natividad, Musngi, Molinyawe, Rivera, Torres, and Moreno (2024), and Sari and Purba (2021) show that internet-dependent systems create unacceptable operational risk for small food businesses, and that an offline Android application with local storage is a practical alternative. Pothineni (2024) sets out the principle. This settles how the system should be built, but not whether its records stay accurate. An offline database drifts just as much as a networked one.

The second response is automation. Ugbebor, Adeteye, and Ugbebor (2024), Gong and Hui (2024), Geminarqi and Purnomo (2023), and Salih, Ghazi,

and Aljanabi (2023) show that automated tracking improves accuracy and removes manual entry error. Applied at ingredient level this produces recipe-based deduction, which supports the first objective of this study. It also reaches a limit. Deduction works from recorded sales and cannot account for material lost outside one, a condition Neve and Schmidt (2021) call imperfect usage capture.

The third response is expiry-driven routing. Herron, Garner, Siddique, Huang, Campbell, Rao, and Morey (2022), Carbajal-Vásquez and colleagues (2023), Suárez and colleagues (2024), and Taufiqqurrachman and colleagues (2025) show that FEFO routing measurably reduces spoilage and works without warehouse infrastructure, which supports the second objective. FEFO decides which batch the system deducts from. It cannot control which container staff actually open.

Both limits lead to the same requirement, and it is what the third objective addresses. Correcting drift means observing physical stock, not calculating it. Destro and colleagues (2023) show that the gap between verifications decides how far records drift, and that infrequent full counts let error build up. Rekik and colleagues (2026) measured the benefit of correcting it, finding an eleven percent sales lift concentrated on items where recorded stock was higher than actual stock, with the strongest effect on perishables. DeHoratius, Holzapfel, Kuhn, Mersereau, and Sternbeck (2023) show that deciding what to count is a consequential design choice. Best, Glock, Grosse, Rekik, and Syntetos (2022) show that discrepancies run in both directions and that misplacement causes

positive discrepancy. This is why reconciling per batch rather than per ingredient keeps the signature of misrouted stock visible.

Every correction method in this literature, however, assumes resources a small cafe does not have. Cycle counting needs dedicated counters, and Destro and colleagues quantify how many. The alternative most often recommended is sensing hardware, which Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) and Ugbebor and colleagues (2024) endorse while admitting its cost puts it out of reach for small operators. This is the gap this study fills: a correction loop sized for an operation with neither counting staff nor sensors. It works by checking every active batch each shift through container counts and fractional estimates, then writing the measured variance back to the batch record.

The fourth objective follows from turning measured expiry risk into action. Gupta, Modgil, Bhattacharyya, and Bose (2021) and van der Waa, Nieuwburg, Cremers, and Neerincx (2021) support deterministic, transparent algorithms over opaque models where interpretability matters. Paradowski and colleagues (2021) show that the choice of weighting materially affects outcomes and therefore needs justification. Combined with the localized multi-criteria approach of Shu and colleagues (2024), the literature supports a rule-based scoring layer as the right design for a small cafe, with weighting treated as a parameter to be tested rather than assumed.

The fifth objective concerns evidence. Dewi, Dantes, and Divayana (2023) show that poor usability produces inventory errors directly. Patel and

Naini (2025) and Kosch and colleagues (2023) establish cognitive load as a design constraint for time-sensitive, multi-step tasks, which the closing audit is. Wulandari and colleagues (2026) and Kurniawan and colleagues (2026) show that ISO 25010 and the System Usability Scale produce quantifiable evaluation evidence. Because these instruments measure usability and functional conformance rather than numerical correctness, this study adds controlled accuracy testing and a before-and-after comparison of closing duration, so the system's central accuracy claim is supported by measurement rather than by what users think of it.

Conceptual Framework

The conceptual framework serves as the structural blueprint for the design and development of the BrewManager system. This study uses the Input-Process-Output (IPO) model to map how data entered by each user role is handled, how the offline SQLite database processes it through deterministic algorithms, and what results are returned to the users.

The framework is arranged to show a correction loop rather than a one-way flow. Sales produce automatic deductions, which generate a theoretical stock level. That level drifts from physical stock because unrecorded losses occur outside sales. The end-of-shift audit measures the gap, and the correction is written back into the same inventory records the deduction engine reads from. Each cycle therefore begins from a physically counted quantity rather than an estimate carried forward. Figure 2.1 presents the conceptual paradigm of the

study, showing the boundaries between barista tasks, manager configuration, the internal logic of the application, and the resulting outputs.

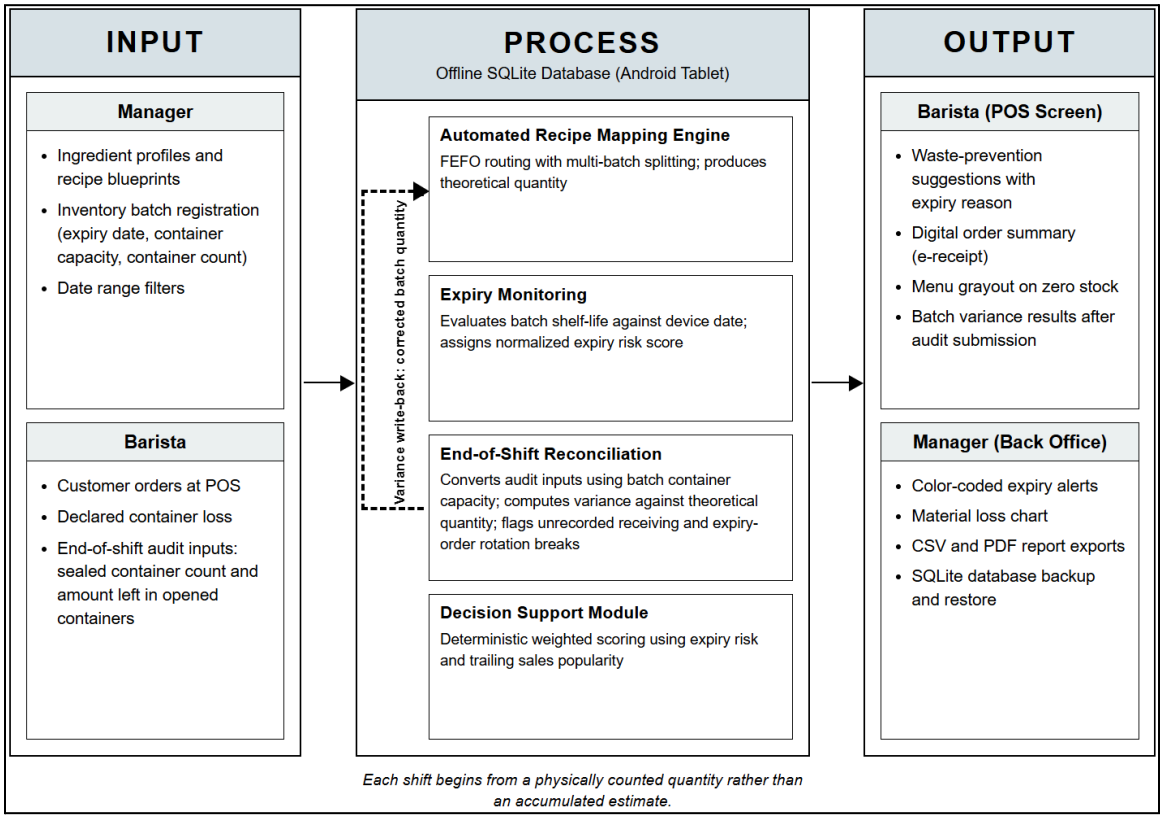


Figure 2.1 Conceptual Framework

Input

The input phase covers all data entered into the system through the Android tablet interface. The PIN-based permission matrix controls what each role can enter, so inputs are divided between the two roles.

Under the Manager role, inputs are mainly configuration and setup data. These include recipe blueprints defining the ingredients required for each beverage, and inventory batch records created at receiving. Each batch record holds one expiration date, one container capacity, and the number of containers received with that date. Because a single purchase can contain containers with

different expiry dates or different sizes, one receiving session may create several batch records. The Manager also supplies date range parameters when filtering the analytics dashboard.

Under the Barista role, inputs are operational and occur during the shift. During service, baristas enter customer orders through the POS screen. They may also record a mid-shift loss, such as a dropped container, through the declare container lost function. At the close of the shift, they submit an audit for every active batch. Each batch takes two inputs: a count of sealed, unopened containers, and the combined amount left across all opened containers of that batch, entered in quarter-container steps. No weighing, unit conversion, or arithmetic is required.

Process

The process phase covers the logic the system performs inside the device's local SQLite database. None of these operations require an internet connection.

When a barista completes a checkout, the system triggers the Automated Recipe Mapping Engine. This applies FEFO and multi-batch splitting to deduct the recipe amount of each ingredient from the earliest-expiring active batch, carrying any remainder to the next batch in expiry order. The result is the theoretical quantity for each batch.

At the same time, the system evaluates each batch against the device clock to determine its shelf-life status and assigns a normalized expiry risk score.

During the end-of-shift audit, the system multiplies the submitted container count and fractional value by the batch's registered container capacity to obtain the audited quantity. It computes the variance against the theoretical quantity and writes the corrected value back to the batch record. This write-back is what closes the loop: the inventory the deduction engine reads at the start of the next shift is the quantity someone physically counted, not an accumulated estimate. The system also checks for two conditions during this process. A submitted count higher than the batch's recorded containers is flagged as possible unrecorded receiving. Two batches of the same ingredient returning variances that cancel each other out are flagged as a likely break in expiry-order rotation.

Separately, the rule-based decision support algorithm runs a deterministic weighted scoring formula. It ranks available menu items using the highest expiry risk score among an item's ingredients and the item's trailing seven-day sales popularity.

Output

The output phase covers what the system presents back to users after processing.

On the barista's POS screen, the system displays ranked waste-prevention suggestions shown together with the expiry condition that triggered them, on-screen digital order summaries serving as e-receipts, and automatic grayout of any menu item whose required ingredients have reached zero stock. After an audit is submitted, the barista is shown the computed variance for each batch along with any rotation or unrecorded receiving flags.

On the manager's dashboard, the system displays color-coded expiry alerts for all active batches, current stock balances, and a material loss chart drawn from reconciliation logs and filterable by date range and ingredient. The Manager can export sales, inventory, and reconciliation reports as CSV or PDF files for review, and can copy the complete SQLite database file to external storage as a recoverable backup.

Chapter 3

TECHNICAL BACKGROUND

Project Description

BrewManager is an offline-first mobile Point-of-Sale and reactive inventory management system developed specifically for local independent coffee shops. The system runs entirely on a single Android tablet device using a locally embedded SQLite database, which means it does not require an internet connection to perform its core functions. The application is built using Flutter configured for Android tablet landscape layout, and all data is stored and processed on the device itself through a centralized database helper class.

The development of BrewManager follows a ground-up, dependency-based build sequence where no module is constructed before the components it relies on are already in place. This approach ensures that each phase of development is stable before the next one builds on top of it.

The process begins with the project foundation, where the Flutter project is configured, the SQLite database schema is defined, and the state management and navigation routing structures are set up. A default Manager PIN is also seeded at this stage so the application is not locked on first use. The system requires this PIN to be changed on first launch. From there, the PIN-based authentication system is built to separate Manager and Barista

access before any feature screens are developed, since every other part of the system sits behind this role gate.

Once authentication is working, the back-office CRUD modules are constructed so that the Manager can configure ingredients, menu items, and recipe blueprints. These records are the foundation that every other feature depends on. The inventory receiving module is built alongside them, allowing the Manager to register incoming stock as inventory batches. Each batch holds one expiration date, one container capacity, and the number of containers received with that date, so a single receiving session may produce several batch records where dates or container sizes differ.

After the data layer is populated, the First-Expired First-Out engine is built and tested in isolation as a pure backend logic module, covering single-batch deductions, multi-batch splits, and zero-stock edge cases with unit tests.

With the FEFO engine verified, the POS transaction screen is developed on top of it, allowing baristas to take orders, trigger automatic ingredient deductions, generate on-screen digital receipts, record mid-shift losses through the Declare Container Lost function, and reverse mistaken orders through the Manager-authorized void function. Following the POS, the Proactive Expiry Monitoring Matrix is added to evaluate batch shelf-life against the device clock and feed normalized expiry risk scores to the later phases.

The End-of-Shift Reconciliation module is then built. This module computes the theoretical quantity of every active batch, accepts a physical count from staff through a sealed container tally and a fractional estimate of what

remains open, computes the variance between the two, and writes the corrected value back to the batch record. Blind audit conditions are enforced so that theoretical values are hidden until the count is submitted.

The final functional phases add the rule-based decision support module for waste prevention, the descriptive analytics dashboard with fl_chart visualizations, the manual CSV and PDF export feature, and the database backup and restore function. The project concludes with a structured system evaluation phase using ISO 25010 quality attributes, the System Usability Scale questionnaire, controlled accuracy testing against known quantities, and a comparison of end-of-shift procedure duration before and after deployment. The raw materials and tools used throughout development are listed below:

Component	Specification
Development Framework	Flutter SDK (Android tablet, landscape layout)
Programming Language	Dart
Local Database	SQLite via database_helper.dart
State Management	Provider or Riverpod
Chart Library	fl_chart
Target Device	Android tablet
IDE	Android Studio or Visual Studio Code
Version Control	Git
Export Formats	CSV, PDF
Backup Format	SQLite database file copy
Evaluation Instruments	ISO 25010, System Usability Scale

	(SUS), controlled accuracy testing, timed procedure comparison

Table 3.1 Raw Materials and Tools Table

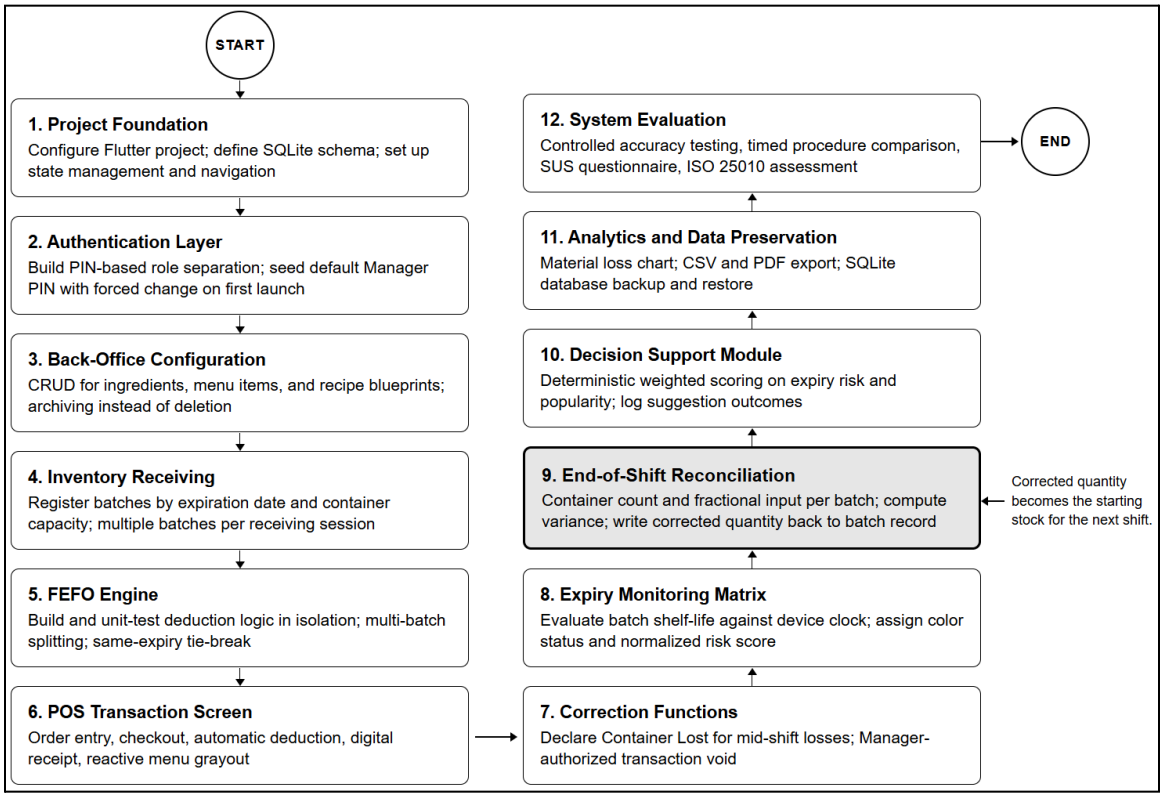


Figure 3.1 Project Development Flowchart

System Development Life Cycle (SDLC)

The development of BrewManager follows the Incremental Model as its System Development Life Cycle. This model was selected because the system is constructed in separate, functional stages where each increment builds directly on top of what was already completed and verified. This approach suits the architecture of BrewManager well, since the core inventory logic cannot function without the database and ingredient records being in place first, the POS screen cannot operate without a working FEFO engine beneath it, the reconciliation module cannot compute a variance without recorded transactions to produce a

theoretical quantity, and the analytics dashboard has no meaningful data to present without completed reconciliation logs. Because every phase depends on the one before it, delivering the system in ordered increments is both practical and structurally sound.

The development of BrewManager is organized into five increments. Each increment produces a functional component of the complete system, and each is aligned to a specific objective of this study.

Increment 1: System Foundation and Security Layer

The first increment establishes everything the rest of the system is built on. This covers the creation of the Flutter project configured for Android tablet landscape layout, the setup of the SQLite database helper class, and the definition of the complete database schema covering all tables including users, ingredients, menu items, recipe blueprints, inventory batches, shift sessions, transactions, transaction items, batch deductions, reconciliation logs, loss events, and suggestion logs. State management and navigation routing structures are also configured at this stage.

A default Manager PIN is seeded into the database so the application is not locked on first launch. The system requires this PIN to be changed before any other function becomes available, and PINs are stored as hashes rather than plain text.

With the foundation in place, the PIN-based authentication system is then developed. This includes building the numeric keypad entry screen, querying the users table upon PIN submission, detecting the role associated with the entered

PIN, and routing Manager accounts to the back-office dashboard while directing Barista accounts to the POS screen. An idle session lock is also implemented so that the application automatically returns to the PIN entry screen after a period of inactivity, with role-based routing applied again upon re-entry.

This increment produces no user-facing features on its own. It exists to guarantee that every later module operates on a stable schema behind an enforced role gate.

Increment 2: Back-Office Configuration, Inventory Receiving, and FEFO Engine

The second increment builds the data and logic layer that all transactional features depend on. It delivers the foundation of the first specific objective.

The Manager back-office screens are developed here, covering full CRUD operations for ingredient profiles, menu items, and recipe blueprints. Records that are referenced by completed transactions are archived rather than permanently deleted, so that historical sales and deduction data remain accurate. A uniqueness constraint is applied to recipe blueprint entries so that the same ingredient cannot be registered twice against a single menu item.

The inventory receiving module is developed alongside the configuration screens. The Manager registers incoming stock by recording each distinct combination of expiration date and container capacity present in a purchase, together with the number of containers matching that combination. Each combination produces a separate inventory batch record holding the expiry date, the capacity of a single container, the number of containers received, and the

total quantity remaining. No consolidation or approximation of expiration dates is applied. Because container capacity is stored at the batch level rather than on the ingredient, the same ingredient may be held in different container sizes at the same time.

After the configuration and receiving modules are in place, the First-Expired First-Out engine is developed and tested as an isolated backend module before being connected to any screen. This engine queries all active batches associated with a given ingredient, orders them by expiration date from earliest to latest, and deducts the required recipe quantity starting from the earliest-expiring batch. Where two batches share an expiration date, the batch holding the smaller remaining quantity is depleted first, so that partially used batches are closed out before full ones are opened.

When a batch does not have sufficient quantity to cover the full requirement, the engine fully depletes it and carries the remaining amount over to the next available batch. This multi-batch splitting logic continues until the full recipe quantity is satisfied. Every deduction is recorded in the batch deductions table with a reference to the originating transaction, and the quantity remaining in each affected inventory batch is updated accordingly. Recording deductions at this level of detail is what later allows a voided transaction to return each quantity to the exact batch it came from.

Unit tests are written to cover single-batch deductions, multi-batch splits, the same-expiry tie-break, and the zero-stock edge case before this engine is considered complete.

Increment 3: POS Transactions and Expiry Monitoring

The third increment connects the backend logic built in Increment 2 to the front-of-house interface. It completes the first specific objective and delivers the second.

The POS transaction screen is built first, presenting the menu grid to the Barista, graying out menu items whose dependent ingredients have reached zero stock, and updating this status reactively in real time the moment any batch is fully depleted. The Barista can build an order, view a running total, and proceed to checkout, at which point the FEFO engine is triggered for every item in the order. The entire checkout operation is wrapped in a single database transaction, so that a failure at any point rolls back every deduction rather than leaving ingredients deducted for a sale that was never recorded.

A digital on-screen receipt is generated, and the transaction is saved to the transactions and transaction items tables. Each transaction item stores the selling price at the moment of sale, so that later changes to a menu item's price do not alter the recorded value of past transactions.

Two correction functions are built into this screen. The Declare Container Lost function allows staff to record a mid-shift physical loss, such as a dropped container, by selecting the affected batch and indicating whether the loss was of sealed containers or of an opened one, together with how many. Where an opened container is reported, the system deducts the quantity it currently holds rather than a full container’s capacity. The system deducts that amount from the batch and writes a loss event record, which is kept separate from the

unexplained variance measured during reconciliation. The Transaction Void function, which requires Manager authorization, reverses a completed sale by reading its batch deduction records and returning each quantity to the specific batch it was originally taken from.

Following the POS, the Proactive Expiry Monitoring Matrix is developed. This module queries all active inventory batches, compares each batch's expiration date against the current device date, and assigns a color-coded status: green for batches with more than seven days remaining, yellow for batches expiring within seven days, and red for batches expiring within 48 hours or already expired. Each batch is also assigned a normalized expiry risk score, where red receives 1.0, yellow receives 0.5, and green receives 0.0, which is later used by the decision support module.

A device clock check is implemented at this stage. The system stores the latest timestamp it has recorded and displays a warning if the device clock is subsequently found to be earlier than that value, since a backward clock change would cause expiry evaluation and FEFO routing to operate on incorrect dates.

Increment 4: End-of-Shift Reconciliation and Decision Support

The fourth increment delivers the third and fourth specific objectives. It contains the correction loop that distinguishes this system from one that only deducts.

The End-of-Shift Reconciliation module is developed first. A shift session is opened when a Barista signs in and closed when reconciliation is submitted, so that a single calendar day may contain more than one reconciliation.

The reconciliation process is initiated by the Barista or Manager through a dedicated shift-close button. At shift close, the system computes the theoretical quantity of every active batch from the inventory batches table and presents an audit interface listing one block per batch. Each block displays the ingredient name, the batch expiration date, and the batch container capacity, so that staff can identify which physical containers belong to which batch.

Each batch accepts two inputs. The first is a count of sealed, unopened containers. The second is the combined amount remaining across all opened containers of that batch, entered through a stepper control in quarter-container increments. This control is not bounded at a single container, so staff submit one combined value regardless of how many containers are open, and the number of inputs per batch stays constant. Each control is initialized at the value recorded during the previous shift, so that routine closings require only small adjustments rather than entry from zero. Because an initialized value could otherwise be submitted without having been verified, each batch block must be marked as checked before the audit can be submitted, either by adjusting a control or by confirming the quantity as unchanged. Leaving a value unaltered therefore remains valid, but becomes a deliberate action rather than an omission.

To prevent bias, theoretical quantities are hidden from staff until the audit is submitted and locked. One validation is applied during input: where a submitted sealed count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value, since such a value is physically impossible rather than merely unexpected. If confirmed, the system

updates the batch container count, records the surplus as probable unrecorded receiving, and flags it for Manager review.

After submission, the system multiplies the submitted values by the batch container capacity to obtain the audited quantity, computes the variance against the theoretical quantity, and writes the corrected value directly to the batch record. This write-back is the operation that closes the correction loop, since the quantity the FEFO engine reads at the start of the next shift is the quantity that was physically counted rather than an estimate carried forward. Every variance is saved to the reconciliation logs table with a reference to the batch and the shift session, and is categorized so that unexplained variance, declared loss, and unrecorded receiving remain distinguishable in later analysis.

A post-submission check compares variances across batches of the same ingredient. Where one batch returns a surplus and another returns a deficit of comparable magnitude, the system flags the pattern as a probable deviation from expiry-order stock rotation, since this signature indicates that material was taken from a batch other than the one the FEFO engine deducted from.

The rule-based decision support module is built after reconciliation, since it operates on corrected batch quantities. The module calculates a popularity score for each menu item by counting how frequently it appears in transaction items over the trailing seven days and normalizing the result against the highest-selling item. It then retrieves the highest expiry risk score among all ingredients in each menu item's recipe blueprint. These two values are combined using a weighted formula in which expiry risk accounts for 60 percent

and popularity accounts for 40 percent. Weight determination in multi-criteria decision models admits several accepted approaches, including both objective methods derived from the data and subjective methods assigned from domain judgement (Paradowski et al., 2021). This study applies a subjective weighting, justified by the purpose of the module. The larger weight is assigned to expiry risk because the module exists to prevent waste, while popularity is included to ensure that the item recommended is one customers are likely to order, since a recommendation that is not acted upon prevents no waste. The specific division of 60 and 40 percent is the smallest at which an item containing stock near expiry is guaranteed to rank above an item containing only fresh stock, regardless of the second item's popularity. At an equal weighting the two can tie, which would permit a fresh but popular item to displace one at risk of spoilage. The weighting is implemented as a configurable parameter rather than a fixed constant, so that it may be adjusted without modification to the scoring logic. Where no sales history exists, as occurs during the first seven days of deployment, the module falls back to ranking by expiry risk alone and states this condition on screen.

The resulting scores rank available menu items, and the top-ranked items are displayed on the POS screen together with the expiry condition that triggered them, updating automatically after each completed transaction. Every suggestion issued is written to a suggestion log along with whether the suggested item was subsequently sold, so that the module's effect on spoilage can be measured during evaluation.

Increment 5: Analytics, Data Preservation, and System Evaluation

The fifth and final increment completes the system with the descriptive analytics dashboard, the data preservation features, and the formal evaluation phase. It delivers the fifth specific objective.

The analytics dashboard is developed for the Manager and displays material loss over time, drawn from reconciliation logs and filterable by date range and ingredient. Declared and unexplained losses are presented as separate ranked lists rather than as a combined total, because the two categories indicate different operational causes and call for different responses. Declared losses point to handling or storage practice, while unexplained losses point to over-pouring, unrecorded giveaways, or recipes that do not match what is actually prepared. Where several ingredients are shown together, quantities are expressed as container equivalents so that ingredients measured in different units can be compared, with the exact quantities in their own units shown alongside. Surpluses arising from unrecorded receiving are excluded from loss totals, since these represent record corrections rather than material waste.

Two data preservation features are then built. The manual export feature allows the Manager to generate CSV and PDF reports covering sales, inventory, and reconciliation history, saved to the tablet's internal storage for record-keeping outside the application. Separately, a database backup function copies the complete SQLite database file to external or user-selected storage, with a corresponding restore function that loads a saved database file back onto a

device. Unlike the report exports, this copy preserves the full relational state of the system and can be used to rebuild operations following device loss or failure.

Once all functional phases are complete, the system evaluation is conducted. Test scripts are prepared covering the core workflows including POS transactions, FEFO deductions with multi-batch splitting, transaction voiding, reconciliation audits with variance write-back, expiry alerts, and decision support suggestions.

The evaluation combines four instruments. Controlled accuracy testing establishes whether the system's computed quantities are correct, by loading known starting quantities, executing a scripted sequence of transactions, and comparing the system's output against independently calculated expected values. Timed procedure comparison measures the duration of the establishment's existing end-of-shift inventory routine before deployment and the duration of the BrewManager reconciliation process after deployment. The System Usability Scale questionnaire is administered to participating coffee shop staff and owners. The system is additionally evaluated against ISO 25010 quality attributes covering functional suitability, reliability, performance efficiency, and usability. Collected results across all four instruments are documented for inclusion in the final capstone report.

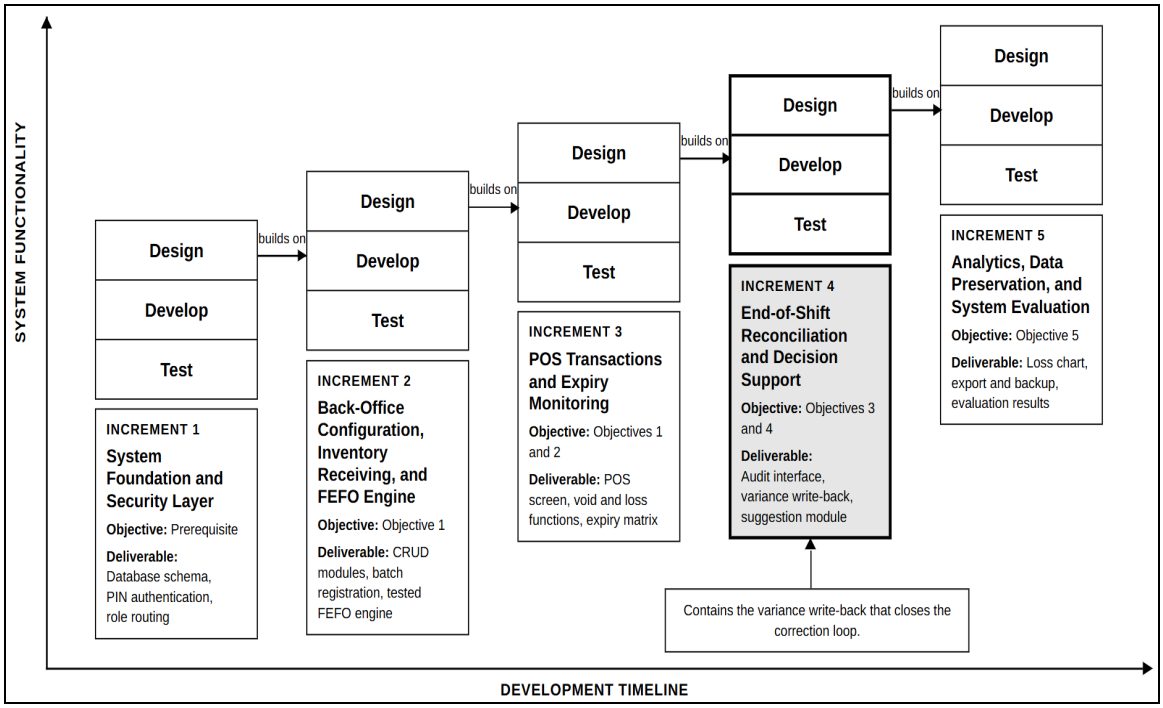


Figure 3.2 Incremental Model

Environment

Locale

The client for this study is Small House Cafe, an independent local coffee shop situated at 15 Tandang Sora Avenue, Sangandaan, Quezon City, 1116 Metro Manila. The cafe operates as a single-branch establishment and serves as the primary deployment environment for the BrewManager system. Because the system is designed with an offline-first, single-device architecture, no additional branches or remote access points are involved. All transactions, inventory records, and reconciliation data are managed on one Android tablet device located within the cafe premises.

Population of the Study

The population of this study consists of the individuals who interact with the BrewManager system during its deployment, testing, and evaluation. Because the system is designed for single-branch, single-device operation, the study follows a single-case deployment at Small House Cafe, and the population reflects the actual staffing of that establishment.

The administrator population consists of one owner who also functions as the Manager of the cafe. This user has full access to the system's back-office features, including ingredient and menu item configuration, recipe blueprint management, inventory receiving, the expiry monitoring dashboard, the analytics dashboard, transaction voiding, and the data export and backup functions. The Manager account is also the only account authorized to add or modify Barista PINs.

The front-of-house population consists of three staff members who work as cashiers and baristas and who share a single role within the system. These users operate the POS transaction screen to process customer orders, record mid-shift losses where they occur, and perform the end-of-shift reconciliation audit at the close of each shift. Because the cafe operates on rotating shifts, more than one reconciliation may be recorded within a single calendar day, and the system supports this through shift session records. The establishment therefore has four system users in total, all of whom participate in the study.

In addition to these four users, a minimum of three information technology practitioners or faculty members with system development experience participate

as evaluators of the system's functional suitability, reliability, and performance efficiency, since these attributes require technical observation rather than end-user experience.

The customers of Small House Cafe are not direct users of the BrewManager system, as the application does not include a customer-facing interface. Because the system operates entirely behind the counter, no customer data is collected, and customers are not counted within the study population. They are indirect beneficiaries, since FEFO routing, expiry monitoring, and reactive stock management help ensure that orders are fulfilled using safe and available ingredients.

Organizational Chart

The organizational structure of Small House Cafe reflects a lean, owner-operated setup typical of independent local coffee shops. At the top of the structure is the Owner, who also serves as the Manager responsible for all back-office decisions, inventory oversight, and system configuration. Reporting directly to the Owner are three staff members working as Cashiers and Baristas, who handle all front-of-house operations including order-taking, drink preparation, mid-shift loss recording, and end-of-shift inventory audits. These staff work on rotating shifts rather than simultaneously. This two-level structure corresponds directly to the two access levels enforced by the system's permission matrix, where the Owner holds Manager access and all front-of-house staff share Barista access. Figure 3.3 below illustrates this structure.

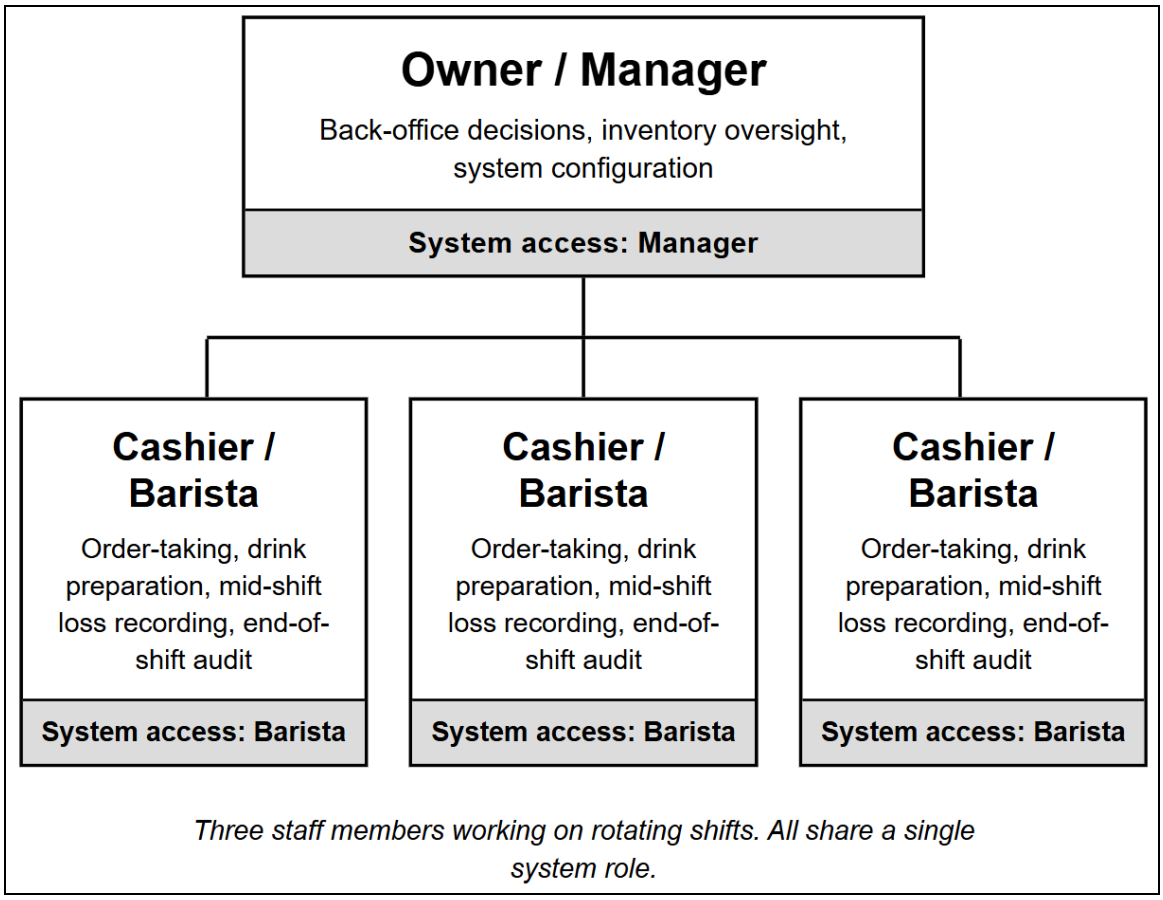


Figure 3.3 Organizational Chart

Requirements Modeling

The requirements of the BrewManager system are described through five categories: input, process, output, performance, and control. These categories define what data the system receives, what it does with that data, what it produces, how well it must perform, and what rules govern access and behavior.

Input

The inputs to the system are divided between the two user roles defined by the permission matrix.

Under the Manager role, inputs include the configuration data needed to set up and maintain the system. These are ingredient profiles containing the ingredient name and unit of measurement; menu item records containing the item name and selling price; and recipe blueprints that map a specific quantity of each ingredient to a menu item. The Manager also enters inventory receiving data. For each purchase, the Manager records every distinct combination of expiration date and container capacity present in that purchase, together with the number of containers matching each combination. Each combination produces one inventory batch record. A single receiving session may therefore create several batch records where expiry dates or container sizes differ. The Manager additionally supplies date range and ingredient parameters when filtering the analytics dashboard, and provides authorization when a transaction void is requested.

Under the Barista role, inputs consist of the customer orders entered through the POS screen during a shift, mid-shift loss declarations, and the physical count submitted at shift close. Audit input is collected per batch and consists of two values. The first is a count of sealed, unopened containers belonging to that batch. The second is the combined amount remaining across all opened containers of that batch, entered through a stepper control in quarter-container increments. This control is not limited to a single container, so one combined value is submitted regardless of how many containers are open. No weighing, unit conversion, or arithmetic is required from the Barista at any point.

Process

Once inputs are received, the system executes several automated processes entirely within the device's local SQLite database without requiring an internet connection.

When a Barista completes a checkout, the Automated Recipe Mapping Engine is triggered. This engine reads the recipe blueprint for each item in the order, identifies all active batches associated with the required ingredients, and sorts them by expiration date from earliest to latest. Where two batches share an expiration date, the batch holding the smaller remaining quantity is placed first. The engine then deducts the required quantity beginning with the earliest-expiring batch. If that batch does not hold enough to cover the full requirement, the engine fully depletes it and carries the remaining amount to the next batch in order. This multi-batch splitting continues until the full recipe quantity is satisfied. Every deduction is recorded in the batch deductions table with a reference to the originating transaction, and the entire checkout is executed as a single database transaction so that a failure at any point rolls back all deductions.

At the same time, the system evaluates each active inventory batch against the current device date to determine shelf-life status. Batches with more than seven days remaining are marked green, batches expiring within seven days are marked yellow, and batches expiring within 48 hours or already past their expiration date are marked red. Each status is converted to a normalized expiry risk score for use by the decision support module.

During end-of-shift reconciliation, the system multiplies the Barista's submitted container count and fractional value by the registered container capacity of that batch to obtain the audited quantity. It computes the variance by comparing this figure against the theoretical quantity the system recorded through automated deductions, then writes the audited quantity back to the batch record as the corrected value. Two additional checks are performed. A submitted count exceeding the total containers recorded for a batch is flagged as probable unrecorded receiving. Two batches of the same ingredient returning variances that offset each other are flagged as a probable deviation from expiry-order stock rotation.

When a transaction void is authorized, the system reads the batch deduction records associated with that transaction and returns each deducted quantity to the specific batch it was originally taken from, then removes the transaction from sales records.

The rule-based decision support module counts how often each menu item has appeared in completed transactions over the trailing seven days and normalizes this into a popularity score. It combines this with the highest expiry risk score among the item’s recipe ingredients using a weighted formula of 60 percent expiry risk and 40 percent popularity, the greater weight being assigned to expiry risk because the module’s purpose is waste prevention, then ranks all currently available menu items by the resulting score. Where no sales history exists, ranking falls back to expiry risk alone.

Output

The outputs of the system are the results and information presented to each user after the processes above are completed.

On the Barista's POS screen, the system outputs a real-time menu grid that grays out items whose dependent ingredients have reached zero stock, an on-screen digital receipt generated after each completed checkout, and a ranked list of waste-prevention suggestions displayed together with the expiry condition that triggered each one, updating automatically after every transaction. At the close of a shift, the system presents the computed variance for each batch to the Barista after the physical count has been submitted and locked, along with any unrecorded receiving or stock rotation flags raised during the process.

On the Manager's dashboard, the system outputs a color-coded expiry monitoring matrix showing the shelf-life status of all active inventory batches, current stock balances, and a material loss chart showing variance per ingredient over time from reconciliation logs, filterable by date range and ingredient. Loss is presented by category so that unexplained variance can be distinguished from declared losses such as dropped containers.

For data preservation, the system produces CSV and PDF reports covering sales, inventory, and reconciliation history for manual export to the tablet's internal storage, and a complete copy of the SQLite database file for backup to external storage.

Performance

The system must perform reliably within the operational constraints of a single Android tablet device running an embedded SQLite database without internet connectivity.

All core processes, including checkout, FEFO deduction, and menu grayout, must execute quickly enough that the Barista does not experience noticeable delay during a transaction. The reactive menu grayout must update the moment any batch reaches zero stock, not only at the point of checkout. Decision support scoring and suggestion display must refresh after each completed transaction so that suggestions are always based on current data.

The end-of-shift reconciliation interface must present all active batches without perceptible loading delay, since the audit is performed at closing when staff are working to finish. The audit is designed so that a complete reconciliation can be submitted in a small number of interactions per batch, and reducing the duration of this procedure relative to the establishment's existing manual routine is a measured outcome of this study.

The system must also remain stable during continuous use across a full operating shift without data loss or application failure. Because all data resides on a single device with no automatic synchronization, the database backup function must produce a file that can be restored onto a replacement device with no loss of relational integrity.

Control

The system enforces control through a PIN-based User Access Control permission matrix that strictly separates what each role can see and do. Baristas are restricted to the POS transaction screen, the mid-shift loss declaration, and the end-of-shift reconciliation audit interface. They cannot access ingredient configurations, recipe blueprints, inventory receiving, the analytics dashboard, transaction voiding, or the data export and backup functions. These features are reserved exclusively for the Manager account. PINs are stored as hashes rather than plain text, and the default Manager PIN seeded at installation must be changed before any other function becomes available.

Blind audit enforcement is applied as a control measure during reconciliation. The system does not display the theoretical quantity to the Barista before the physical count is submitted. The theoretical value is revealed only after submission is locked, preventing the count from being adjusted to match the system's expectation. One narrow exception applies. Where a submitted sealed container count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value. This validation identifies a physically impossible entry rather than an unexpected one, and does not disclose the theoretical quantity.

Transaction voiding is controlled through Manager authorization. Because a void reverses both a sale and its associated inventory deductions, allowing it without approval would permit front-of-house staff to alter sales and inventory

records without oversight. Voids are recorded rather than deleted, so that the reversal itself remains visible in the transaction history.

FEFO routing acts as an automated control over ingredient usage, ensuring that no transaction consumes a later-expiring batch while an earlier-expiring batch still holds stock. Because the system routes deductions in software but cannot control which physical container staff open, deviation from expiry-order withdrawal is treated as a condition to be detected during reconciliation rather than as an assumption. Where the Declare Container Lost function is used, the system deducts the specified number of containers from the identified batch and records the event as a declared loss, kept separate from the unexplained variance measured at shift close.

Finally, because the system operates entirely offline, the device's internal clock serves as the sole time reference for expiry calculations and FEFO routing. The system retains the latest timestamp it has recorded and displays a warning if the device clock is subsequently found to be earlier than that value. It cannot independently verify the correct time without network access, which represents a control boundary that must be maintained by the cafe's management.

Requirements Specification

Operational Feasibility

Operational feasibility examines whether the proposed system can realistically function within the day-to-day environment of the client. For Small House Cafe, the assessment confirms that BrewManager is operationally viable

because it is designed around the actual workflow of a two-role, single-device coffee shop.

The system does not require internet connectivity, so service continues during network outages that would halt a cloud-based point-of-sale system. It does not depend on external hardware beyond the Android tablet itself. The reconciliation module in particular was designed to avoid any dependence on weighing equipment, since requiring a digital scale would impose a purchase on the client and introduce a device that could fail or be misplaced. Instead, the audit records stock as a count of sealed containers and a fractional estimate of what remains open, which requires no equipment and no calculation from staff.

The system also fits the labor available at the establishment. The most time-sensitive procedure, the end-of-shift audit, is performed at closing when staff are working to finish. It is therefore limited to two inputs per active batch, with each control initialized at the previous shift's value so that routine closings require only small adjustments. Reducing the duration of this procedure relative to the cafe's existing manual routine is a measured outcome of this study.

The PIN-based role separation ensures that each user sees only the screens relevant to their tasks, which keeps the interface focused for both the Manager and the Barista. Because all front-of-house staff share a single Barista role, no additional configuration is required when shift assignments change.

Adoption risk is addressed by keeping the front-of-house workflow close to the existing one. Baristas continue to take orders and count stock at closing as they already do. What changes is that the counting is entered into a guided

interface rather than written on paper, and that the deduction of ingredients happens automatically rather than being recorded manually or not at all.

System Flowchart

The system flowchart presents the overall operational flow of BrewManager from application launch through role authentication to the completion of a shift. It shows the two role paths diverging after PIN entry, the transaction and deduction sequence performed during service, and the reconciliation sequence performed at shift close, including the write-back of corrected quantities to inventory records. Figure 3.4 presents this flow.

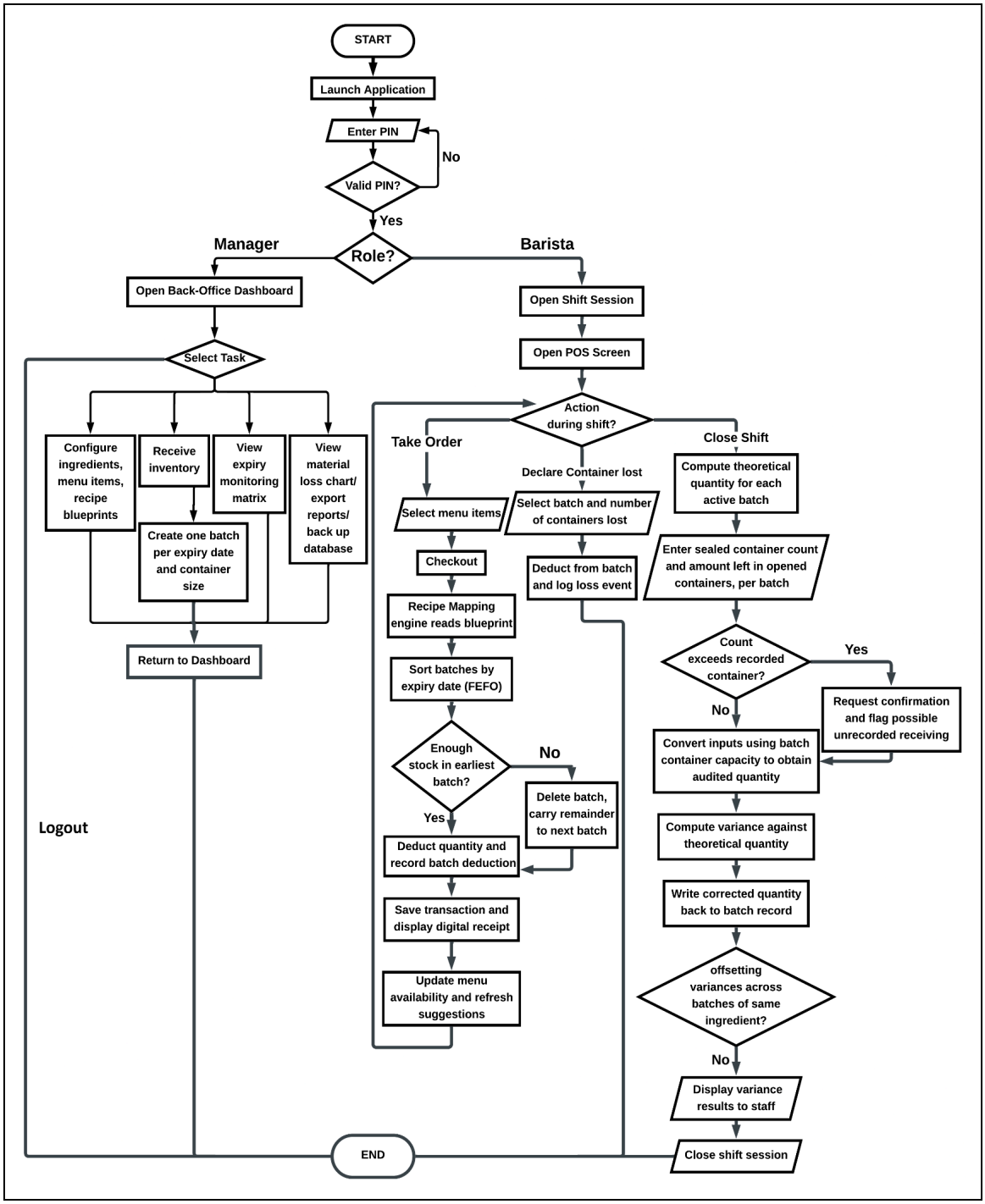


Figure 3.4 System Flowchart

Site Map

The site map presents the screen hierarchy of the application and shows which screens are reachable from each role after authentication. The Manager

path leads to the back-office dashboard and its configuration, receiving, monitoring, analytics, and data preservation screens. The Barista path leads to the POS transaction screen and the end-of-shift reconciliation interface. The separation shown in the site map corresponds directly to the permission matrix described under Control. Figure 3.5 presents this hierarchy.

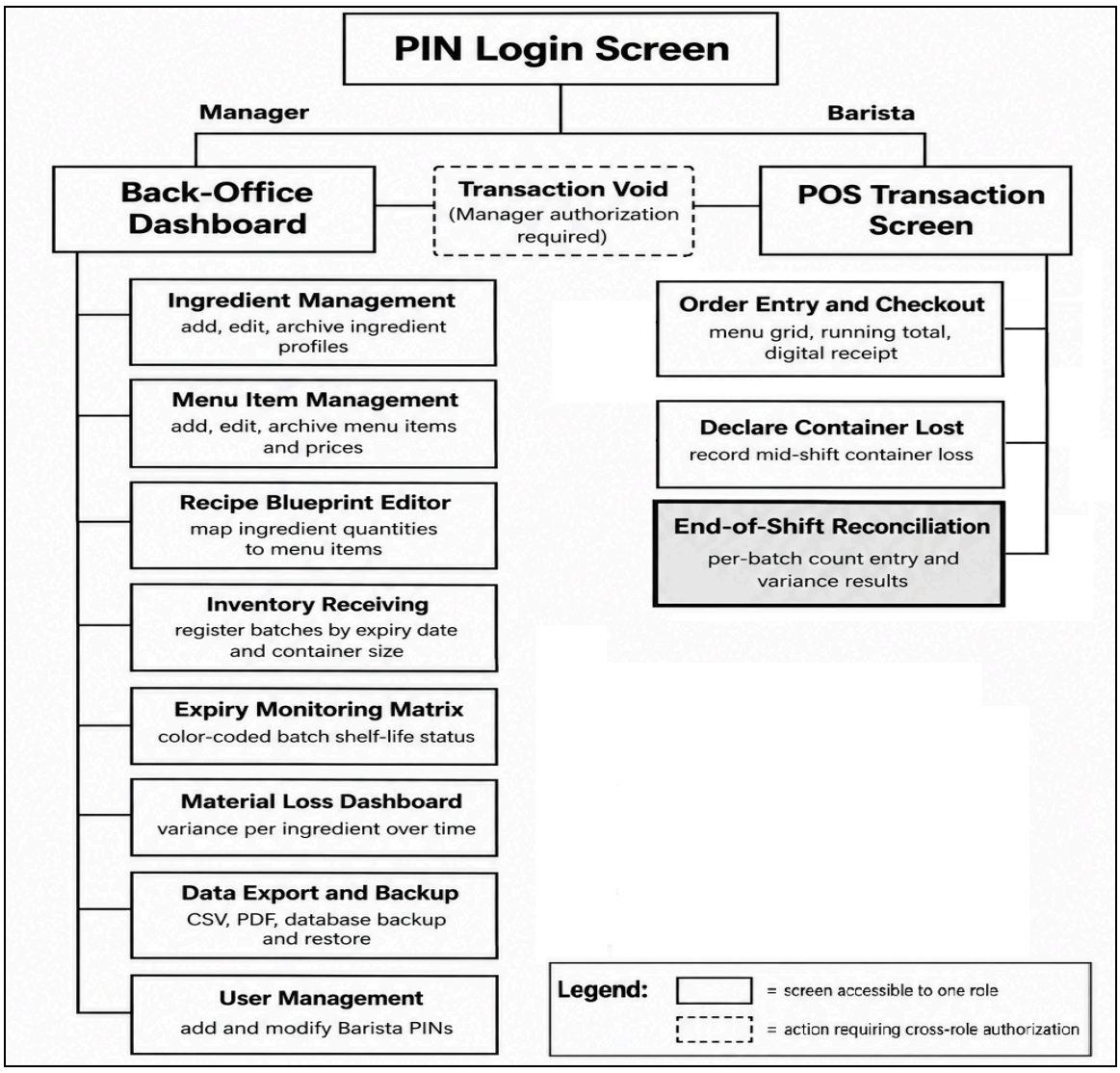


Figure 3.5 Site Map

User Journey Map

The user journey map traces the sequence of actions each role performs across a full operating day. The Manager journey covers opening configuration and inventory receiving, periodic monitoring of expiry status, and end-of-day review of reconciliation results and material loss. The Barista journey covers shift sign-in, order processing during service, recording of any mid-shift losses, and completion of the closing audit. Figure 3.6 presents both journeys.

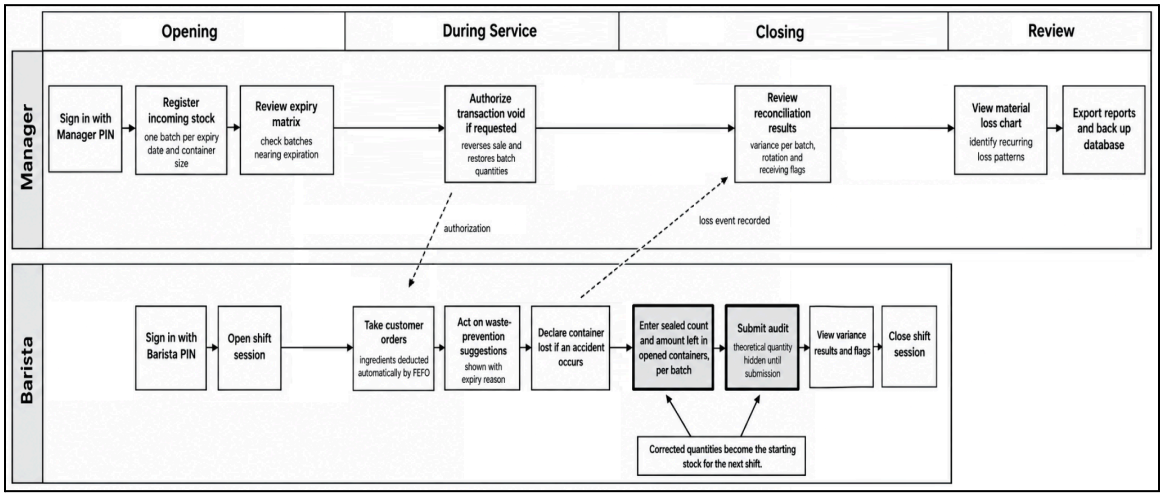


Figure 3.6 User Journey Map

Schedule Feasibility

Schedule feasibility assesses whether the BrewManager system can be completed within the time available for the capstone development cycle. The assessment covers the full period from initial project proposal through system implementation and evaluation at the client site.

The development sequence follows the five increments described earlier in this chapter. Because each increment depends on the one before it, the schedule is arranged so that no increment begins before its prerequisites are

complete and tested. The foundation and security layer must be finished before any feature screens are built. The FEFO engine must be tested on its own before the POS screen is connected to it. Transaction records must exist before reconciliation has a theoretical quantity to audit against. This chain of dependencies sets the order of the schedule and limits how much work can be done at the same time.

Two scheduling concerns needed particular attention. The first is recipe blueprint configuration, which depends on getting accurate ingredient measurements from the client. Independent coffee shops often prepare drinks by practiced estimation rather than by measured amounts, so written recipe quantities could not be assumed to exist. Time was therefore set aside for on-site observation and measurement of actual pours across a representative set of drinks. This activity runs alongside development rather than after it, since the deduction engine cannot be tested properly without accurate blueprints.

The second concern is the evaluation phase, which requires the system to run at the client site long enough to build up reconciliation records. Material loss patterns cannot be seen from a single shift, and the analytics dashboard needs several reconciliation cycles before it shows anything meaningful. The schedule therefore allows deployment time before the evaluation activities begin, rather than treating implementation and evaluation as one stage.

Table 3.2 presents the planned and actual timelines for all project activities from proposal through evaluation. Figure 3.7 presents the same information as a

Gantt chart, with planned durations shown against actual durations so that any deviations can be identified.

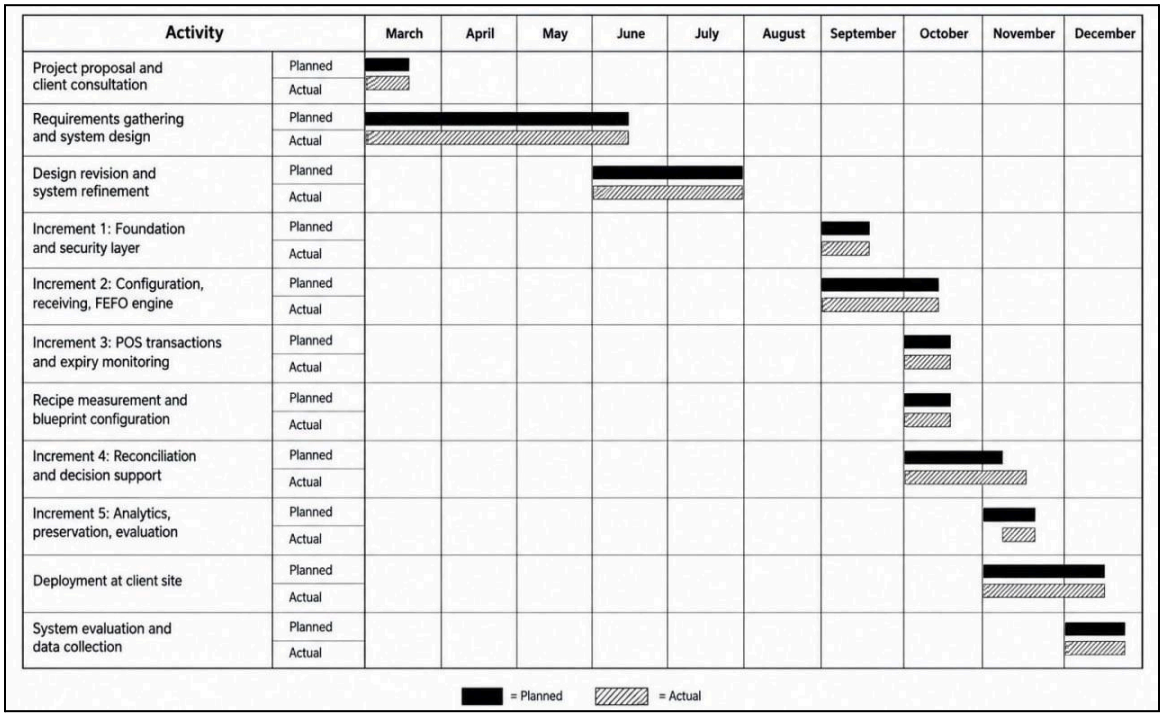


Figure 3.7 Gantt Chart

Economic Feasibility

Economic feasibility examines whether the cost of building and deploying BrewManager is reasonable compared to the value it gives back to the client. The assessment covers development costs, hardware costs, ongoing costs, and the savings the system is expected to produce.

Cost and Benefit Analysis

The development cost of BrewManager is low because the system is built entirely on free and open-source tools. Flutter, SQLite, and the supporting Dart packages have no licensing fees. The proponents carry out the development as

part of the capstone requirement, so there is no cost for hired labor. Version control and project hosting use free-tier services.

The main cost to the client is hardware. The system needs one Android tablet that can run the application in landscape orientation. Because the system works offline with a local database and does not perform heavy computation, a mid-range device is enough. A suitable tablet is available for roughly PHP 8,000 to 12,000. No server, network equipment, or subscription service is needed.

The system was designed to avoid extra hardware costs wherever possible. The reconciliation module records stock through container counts and fractional estimates rather than through weight, so the client does not need to buy a digital scale. Receipts are shown on screen rather than printed, so there is no cost for a receipt printer or for paper. These were deliberate design choices, made so that the total cost to the client stays limited to a single device.

Ongoing costs are minimal. The system does not use cloud hosting or synchronization, so there are no monthly fees. The electricity used to charge the tablet is very small next to the cafe's normal operating costs. Data is preserved through the built-in export and backup functions, which write to storage already on the device or to an inexpensive external drive.

The benefits fall into three areas. The first is less material waste. The expiry monitoring matrix, FEFO routing, and the waste-prevention suggestions work together to help ingredients get used before they spoil. Perishable ingredients such as milk and syrups make up a large share of what a coffee shop buys regularly, so even a small drop in spoilage adds up over time.

The second is less work at closing. The end-of-shift audit replaces manual counting and recording with two guided inputs per batch. Reducing the time this takes is a measured outcome of this study, and time saved at closing is time returned to other tasks or to staff going home earlier.

The third is better purchasing decisions. The material loss dashboard shows which ingredients are lost most often and in what amounts. This gives the owner information that manual tracking does not provide, which makes it possible to spot and correct over-ordering.

Cost Recovery Scheme

BrewManager is developed as an academic capstone project and given to the client at no charge, so there is no purchase price to recover. The cost recovery scheme therefore covers the client's hardware spending rather than a software fee.

The tablet is the only real expense the client takes on. This cost is recovered through the reduction in material waste the system is designed to produce. Because the system measures material loss directly through reconciliation logs, the client can see the value of ingredients lost over a given period and compare it against what the tablet costs. The system therefore supplies the data needed to check its own recovery rather than relying on an estimate.

No recurring costs are passed to the client, since the system needs no subscription, no hosting, and no licensed software. If the tablet ever needs

replacing, the database backup function allows the system to be restored onto a new device with the historical records intact, so the client does not start over.

Data and Process Modelling

This section describes how data is stored and how it moves through the BrewManager system. Because the application keeps all records in a local SQLite database on a single device, the data model must hold every relationship the system depends on without relying on an external server. Four models are presented: the entity relationship diagram, the data dictionary, the context diagram, and the data flow diagram.

Entity Relationship Diagram

The entity relationship diagram shows the tables in the BrewManager database and how they are linked. The design is built around three ideas that shape the rest of the system.

The first is that inventory is tracked at the batch level rather than at the ingredient level. An ingredient record holds only the name and unit of measurement. The actual stock lives in inventory batch records, each of which holds one expiration date, the capacity of a single container, the number of containers received, and the quantity remaining. One ingredient can therefore have many batches at the same time, each with its own expiry date and container size. This is what allows FEFO routing to choose between batches and what allows reconciliation to correct each batch separately.

The second is that every deduction is recorded individually. When a sale takes ingredients from one or more batches, each deduction is written to its own record linked to both the transaction and the batch it came from. Storing deductions this way is what makes a transaction void possible, since the system can read these records and return each amount to the exact batch it was taken from.

The third is that reconciliation results are stored per batch rather than per ingredient. Each reconciliation log records the theoretical quantity, the audited quantity, the variance between them, and the type of variance found. Because the log is tied to a batch and to a shift session, the system can show which batch drifted and during which shift, rather than only reporting a total for the ingredient.

The main entities and their relationships are as follows:

Users holds the accounts for the Manager and Barista roles, with PINs stored as hashes. One user can open many shift sessions.

Shift Sessions records the opening and closing time of each shift and the user who opened it. One shift session contains many transactions and many reconciliation logs. This allows more than one reconciliation to be recorded in a single calendar day.

Ingredients holds the profile of each raw material, including its name, unit of measurement, and archive status. One ingredient has many inventory batches and appears in many recipe blueprints.

Inventory Batches holds the stock itself. Each record belongs to one ingredient and stores an expiration date, container capacity, the number of containers

received, and the quantity remaining. One batch can be the source of many batch deductions and can have many reconciliation logs across different shifts.

Menu Items holds the drinks offered for sale, including the name, selling price, and archive status. One menu item has many recipe blueprint entries and appears in many transaction items.

Recipe Blueprints links a menu item to the ingredients it needs and the quantity of each. A uniqueness rule prevents the same ingredient from being added twice to one menu item.

Transactions holds each completed sale, with a reference to the shift session it belongs to, the time of sale, the total amount, and whether the sale has been voided. One transaction contains many transaction items.

Transaction Items holds each line of a sale, linking a transaction to a menu item and storing the quantity ordered and the selling price at the moment of sale. Storing the price here keeps past sales accurate even if the menu price is changed later.

Batch Deductions records each individual deduction, linking a transaction to the specific batch the quantity was taken from. One transaction can produce several deductions when multi-batch splitting occurs.

Reconciliation Logs holds the result of each audited batch, linking a shift session to an inventory batch and storing the theoretical quantity, audited quantity, variance, and variance type.

Loss Events records mid-shift losses declared through the Declare Container Lost function, linking a shift session to a batch and storing the amount lost.

These are kept separate from reconciliation logs so that known losses and unexplained variance can be told apart in the analytics.

Suggestion Logs records each waste-prevention suggestion the decision support module produces, along with the menu item suggested, the expiry condition that triggered it, and whether the item was sold afterwards. This is what allows the module to be evaluated.

Figure 3.8 presents the entity relationship diagram.

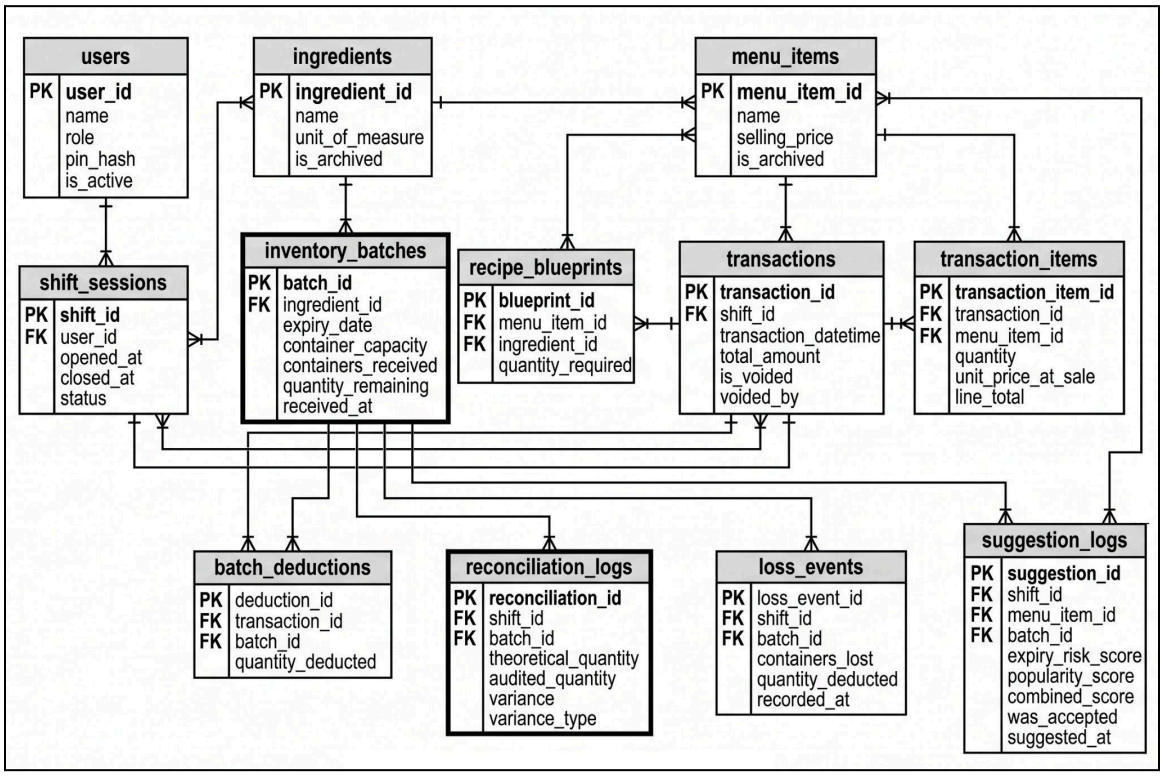


Figure 3.8 Entity Relationship Diagram

Data Dictionary

The data dictionary lists every field in the BrewManager database, along with its data type, constraints, and purpose. Because the system runs on SQLite, the types used are the ones SQLite supports directly: INTEGER for whole numbers and identifiers, REAL for quantities that may include fractions,

TEXT for names and timestamps, and INTEGER used as a boolean where a field records a yes or no state.

Three conventions apply throughout. Quantities are stored in the base unit of measurement recorded on the ingredient, so that all arithmetic within a single ingredient is consistent. Timestamps are stored as TEXT in ISO 8601 format, which SQLite can sort and compare directly. Records that must be preserved for historical accuracy are marked with an archive flag rather than being deleted, so that past transactions and deductions continue to reference valid rows.

Field Name	Data Type	Constraint	Description
user_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each user account
name	TEXT	Not Null	Display name of the staff member
role	TEXT	Not Null	Access level of the account, either Manager or Barista
pin_hash	TEXT	Not Null	Hashed value of the user's PIN so the PIN is never held in plain text
is_active	INTEGER	Not Null, Default 1	Indicates whether the account is currently in use

Table 3.2 Data Dictionary Table (users)

Field Name	Data Type	Constraint	Description
shift_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each shift session
user_id	INTEGER	Foreign Key, Not Null	References the user who opened the shift
opened_at	TEXT	Not Null	Timestamp when the shift began
closed_at	TEXT	Nullable	Timestamp when the shift was closed through reconciliation
status	TEXT	Not Null	Current state of the shift, either open or closed

Table 3.3 Data Dictionary Table (shift_sessions)

Field Name	Data Type	Constraint	Description
ingredient_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each raw ingredient
name	TEXT	Not Null, Unique	Name of the ingredient
unit_of_measure	TEXT	Not Null	Base unit used for all quantities of this ingredient, such as milliliters or grams
is_archived	INTEGER	Not Null, Default 0	Marks the ingredient as retired while preserving its historical records

Table 3.4 Data Dictionary Table (ingredients)

Field Name	Data Type	Constraint	Description
batch_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each inventory batch
ingredient_id	INTEGER	Foreign Key, Not Null	References the ingredient this batch holds
expiry_date	TEXT	Not Null	Expiration date shared by all containers in this batch
container_capacity	REAL	Not Null	Full quantity held by one unopened container in this batch
containers_received	INTEGER	Not Null	Number of containers recorded when the batch was received
quantity_remaining	REAL	Not Null	Current quantity remaining in the batch, updated by deductions and by reconciliation
received_at	TEXT	Not Null	Timestamp when the batch was registered
corrected_by	INTEGER	Foreign Key, Nullable	User who last corrected figures entered wrongly at receiving; null if the batch has never been corrected
corrected_at	TEXT	Nullable	Timestamp of that correction
previous_quantity	REAL	Nullable	The quantity remaining as it stood immediately before

Field Name	Data Type	Constraint	Description
			that correction

Table 3.5 Data Dictionary Table (inventory_batches)

Field Name	Data Type	Constraint	Description
menu_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each menu item
name	TEXT	Not Null, Unique	Name of the beverage or item offered for sale
selling_price	REAL	Not Null	Current selling price of the item
is_archived	INTEGER	Not Null, Default 0	Marks the item as removed from the menu while preserving its sales history
is_ingredient_tracked	INTEGER	Not Null, Default 1	Indicates whether the item consumes ingredients from inventory. Prepared items bought ready to sell, such as pastries, are sold without a recipe and are not deducted from stock

Table 3.6 Data Dictionary Table (menu_items)

Field Name	Data Type	Constraint	Description
blueprint_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each blueprint entry
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item this entry belongs to

Field Name	Data Type	Constraint	Description
ingredient_id	INTEGER	Foreign Key, Not Null	References the required ingredient
quantity_required	REAL	Not Null	Amount of the ingredient needed to prepare one unit of the menu item

Table 3.7 Data Dictionary Table (recipe_blueprints)

Field Name	Data Type	Constraint	Description
transaction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each completed sale
shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which the sale occurred
transaction_datetime	TEXT	Not Null	Timestamp when the sale was completed
total_amount	REAL	Not Null	Total value of the sale
is_voided	INTEGER	Not Null, Default 0	Indicates whether the sale has been reversed
voided_by	INTEGER	Nullable	References the Manager account that authorized the reversal

Table 3.8 Data Dictionary Table (transactions)

Field Name	Data Type	Constraint	Description
transaction_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each line of a sale
transaction_id	INTEGER	Foreign Key, Not Null	References the sale

Field Name	Data Type	Constraint	Description
			this line belongs to
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item sold
quantity	INTEGER	Not Null	Number of units of the item sold in this line
unit_price_at_sale	REAL	Not Null	Selling price of the item at the moment of sale, stored so later price changes do not alter past records
line_total	REAL	Not Null	Total value of this line, calculated from quantity and unit price at sale

Table 3.9 Data Dictionary Table (transaction_items)

Field Name	Data Type	Constraint	Description
deduction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each individual deduction
transaction_id	INTEGER	Foreign Key, Not Null	References the sale that triggered the deduction
batch_id	INTEGER	Foreign Key, Not Null	References the specific batch the quantity was taken from
quantity_deducted	REAL	Not Null	Amount deducted from this batch for this sale

Table 3.10 Data Dictionary Table (batch_deductions)

Field Name	Data Type	Constraint	Description
reconciliation_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each audited batch result
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the audit was performed
batch_id	INTEGER	Foreign Key, Not Null	References the batch that was audited
theoretical_quantity	REAL	Not Null	Quantity the system calculated should remain, based on recorded deductions
audited_quantity	REAL	Not Null	Quantity derived from the physical count submitted by staff
variance	REAL	Not Null	Difference between theoretical and audited quantity, negative for loss and positive for surplus
variance_type	TEXT	Not Null	Category of the variance, recorded as unexplained, declared loss, or unrecorded receiving

Table 3.11 Data Dictionary Table (reconciliation_logs)

Field Name	Data Type	Constraint	Description
loss_event_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each declared mid-shift loss
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the

Field Name	Data Type	Constraint	Description
			loss occurred
batch_id	INTEGER	Foreign Key, Not Null	References the batch the loss was taken from
containers_lost	REAL	Not Null	Number of containers lost, recorded as sealed containers or as a single opened container
quantity_deducted	REAL	Not Null	Amount deducted from the batch, calculated from containers lost and container capacity
recorded_at	TEXT	Not Null	Timestamp when the loss was declared

Table 3.12 Data Dictionary Table (loss_events)

Field Name	Data Type	Constraint	Description
suggestion_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each suggestion issued
shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which the suggestion appeared
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item that was suggested
batch_id	INTEGER	Foreign Key, Not Null	References the batch whose expiry risk triggered the suggestion

expiry_risk_score	REAL	Not Null	Normalized expiry risk value used in the scoring calculation
popularity_score	REAL	Not Null	Normalized trailing seven-day sales popularity used in the scoring calculation
combined_score	REAL	Not Null	Weighted result of the two scores, ranking the item against other available items
was_accepted	INTEGER	Not Null, Default 0	Indicates whether the suggested item was sold after the suggestion appeared
suggested_at	TEXT	Not Null	Timestamp when the suggestion was displayed

Table 3.13 Data Dictionary Table (suggestion_logs)

Context Diagram

The context diagram presents BrewManager as a single process and shows what enters and leaves the system across its boundary. Because the system runs entirely on one offline device, there are no external systems, servers, or third-party services to represent. The only entities that interact with the system are the two user roles defined by the permission matrix.

The Manager supplies the configuration and stock data the system depends on. This includes ingredient profiles, menu items, recipe blueprints, and inventory batch records created during receiving. The Manager also supplies filter parameters for the analytics dashboard and authorization when a

transaction void is requested. In return, the system provides the Manager with expiry status information, material loss reports, exported files, and database backups.

The Barista supplies the operational data generated during a shift. This includes customer orders, declared container losses, and the physical count submitted at the close of each shift. In return, the system provides the Barista with digital receipts, current menu availability, waste-prevention suggestions, and the variance results produced after an audit is submitted.

The device clock is shown as an external source rather than as a user, since expiry evaluation and FEFO routing depend on the current date but the system cannot influence or verify it. This is the only input the system receives that does not come from a person. Figure 3.9 presents the context diagram.

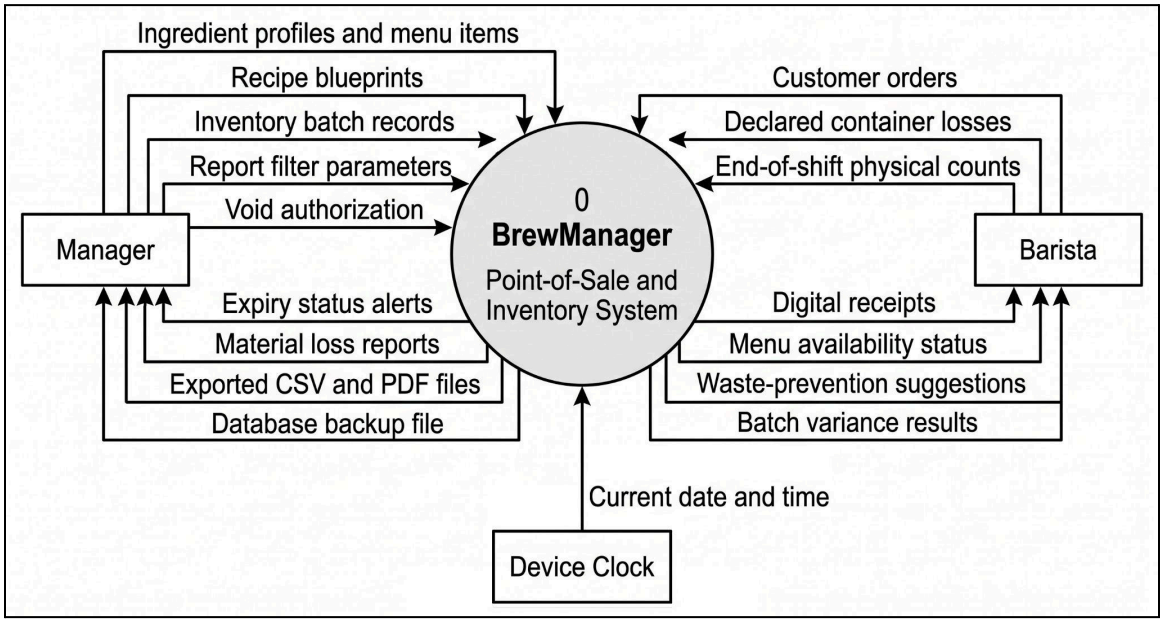


Figure 3.9 Context Diagram

Data Flow Diagram

The data flow diagram breaks the single process shown in the context diagram into the processes that make up the system, and shows the data stores each one reads from and writes to. Four processes are identified, corresponding to the main functions described under Requirements Modeling. The data stores shown group the database tables presented in the data dictionary according to how they are used.

Process 1, Setup and Configuration, receives configuration and stock data from the Manager. This covers PIN authentication records, ingredient profiles, menu items, recipe blueprints, and inventory batch registration. Configuration records are written to the configuration store, while batch registration writes one inventory batch record for each distinct combination of expiration date and container capacity in a purchase.

Process 2, Sales and Deduction, receives customer orders from the Barista. It reads recipe blueprints and prices from the configuration store, reads active batches from the inventory batches store, and applies FEFO routing with multi-batch splitting to determine which batches to deduct from. It updates the quantity remaining in the affected batches and writes the completed sale, its line items, and each individual deduction to the activity records store. This process also evaluates batch shelf-life against the device clock and produces the ranked waste-prevention suggestions returned to the Barista, along with digital receipts and current menu availability.

Process 3, Reconciliation and Correction, reads the inventory batches store to compute the theoretical quantity of every active batch, then receives the physical count submitted by the Barista. It converts the submitted container count and fractional value using each batch's container capacity, computes the variance against the theoretical quantity, and writes the corrected quantity back to the inventory batches store. Each result is written to the activity records store together with its variance type. Mid-shift losses declared through the Declare Container Lost function are handled by this process and deducted from the affected batch.

Process 4, Reporting and Backup, reads the activity records store to produce the material loss chart and the exported CSV and PDF reports for the Manager. It also produces a copy of the complete database file for backup.

The write-back performed by Process 3 is the flow that distinguishes this system from one that only records deductions. Because the corrected quantity is written to the same inventory batches store that Process 2 reads from at the start of the next shift, each shift begins from a physically counted quantity rather than an estimate carried forward since the last correction. Figure 3.10 presents the data flow diagram.

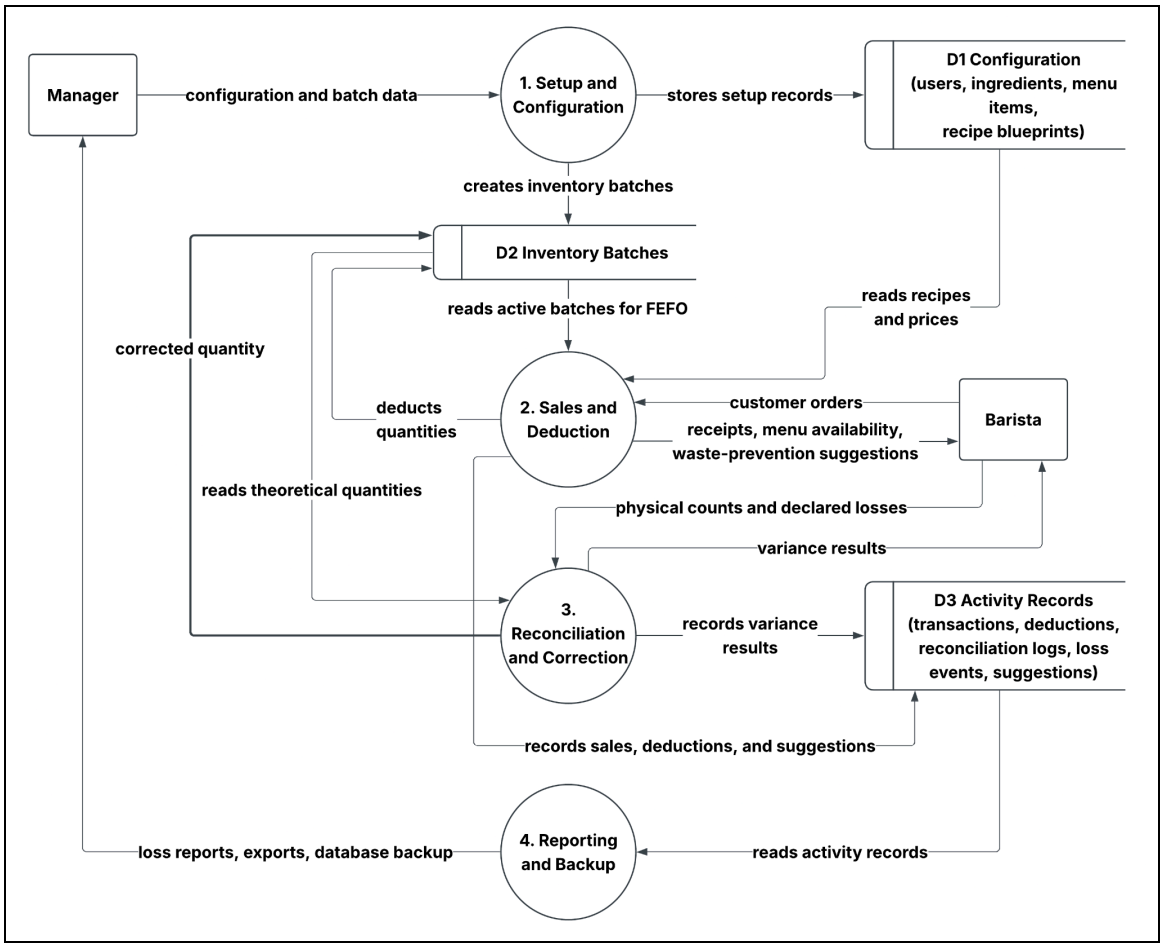


Figure 3.10 Data Flow Diagram

Object Modelling

This section describes the system in terms of its actors, its objects, and the order in which its operations occur. Three models are presented: the use case diagram, the class diagram, and the sequence diagram. A risk assessment follows.

Use Case Diagram

The use case diagram shows what each user role can do with the system. The two actors match the two access levels enforced by the permission matrix,

so the diagram also serves as a visual record of what each role is and is not allowed to reach.

The Manager has access to all configuration and review functions. These include managing ingredient profiles, managing menu items, editing recipe blueprints, and registering incoming inventory batches. The Manager also views the expiry monitoring matrix, views the material loss dashboard, exports reports, backs up and restores the database, and manages Barista PINs.

The Barista has access to the operations performed during a shift. These include opening and closing a shift session, processing customer orders, declaring a container lost, and submitting the end-of-shift audit.

One use case is shared between the two actors. Voiding a transaction is initiated by the Barista at the POS screen but requires Manager authorization before it takes effect. It is therefore connected to both actors rather than placed under either one.

Several use cases include or extend others. Processing a customer order includes the automatic deduction of ingredients, since no order can be completed without it. Submitting the end-of-shift audit includes the calculation of variance and the write-back of corrected quantities to the affected batches. Voiding a transaction includes the restoration of deducted quantities to the batches they came from. These included use cases are drawn separately because they are system operations rather than user choices, and because they are the operations that carry the system's inventory accuracy. Figure 3.11 presents the use case diagram.

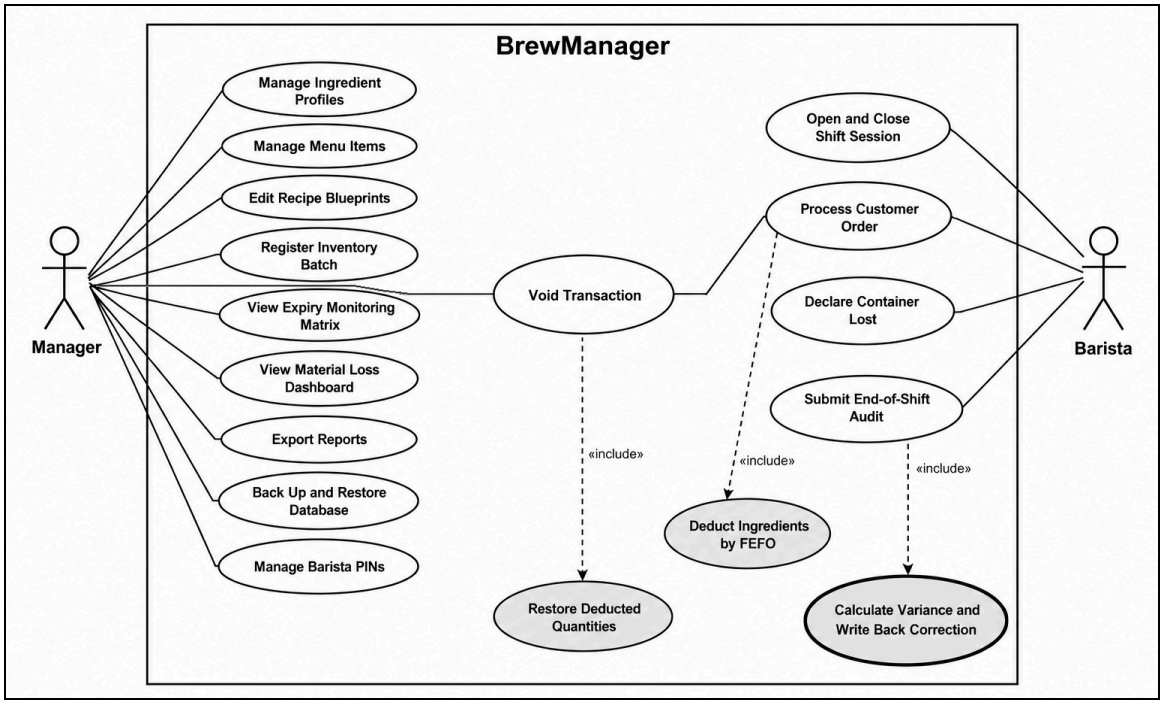


Figure 3.11 Use Case Diagram

Class Diagram

The class diagram shows the main classes that make up the application, the data each one holds, and the operations each one performs. The classes fall into three groups.

The first group is the model classes. These represent the records stored in the database and mirror the tables described in the data dictionary. Each model class holds the fields of its table and provides methods to convert between an object and a database row. Two model classes carry logic of their own. The inventory batch class evaluates its own expiry status against the current date and returns a normalized expiry risk score, since these values depend only on the batch's own data.

The second group is the service classes. These hold the logic that operates on the models. The FEFO engine sorts batches by expiry date and

performs deductions with multi-batch splitting. The transaction service creates sales records, records each deduction against the batch it came from, and reverses both when a void is authorized. The reconciliation service computes theoretical quantities, converts audit input using container capacity, calculates variance, and writes corrected quantities back to the affected batches. The decision support service calculates expiry risk and popularity scores and ranks menu items. Keeping this logic in service classes rather than inside the models allows each service to be tested on its own before it is connected to any screen.

The third group is the database helper. A single helper class manages the connection to the local SQLite database and provides the query methods the services depend on. It also provides the method that runs several writes as one database transaction, which is what allows a failed checkout to roll back every deduction rather than leaving ingredients deducted for a sale that was never recorded. Because the system runs on one device with no network layer, all data access passes through this one class.

Two design points are worth noting. The reconciliation service depends on the container capacity stored in the inventory batch, since the audit cannot convert a container count into a quantity without knowing how much one container holds. And the transaction service records each deduction against the specific batch it came from, which is what allows a void to return each quantity to the correct batch.

Relationships between the model classes carry multiplicity labels showing how many instances of one class relate to another. The service classes and the

database helper are shown without relationship lines, since they operate on objects passed to them rather than holding those objects as their own data. The order in which the services call one another is shown in the sequence diagram that follows. Figure 3.12 presents the class diagram.

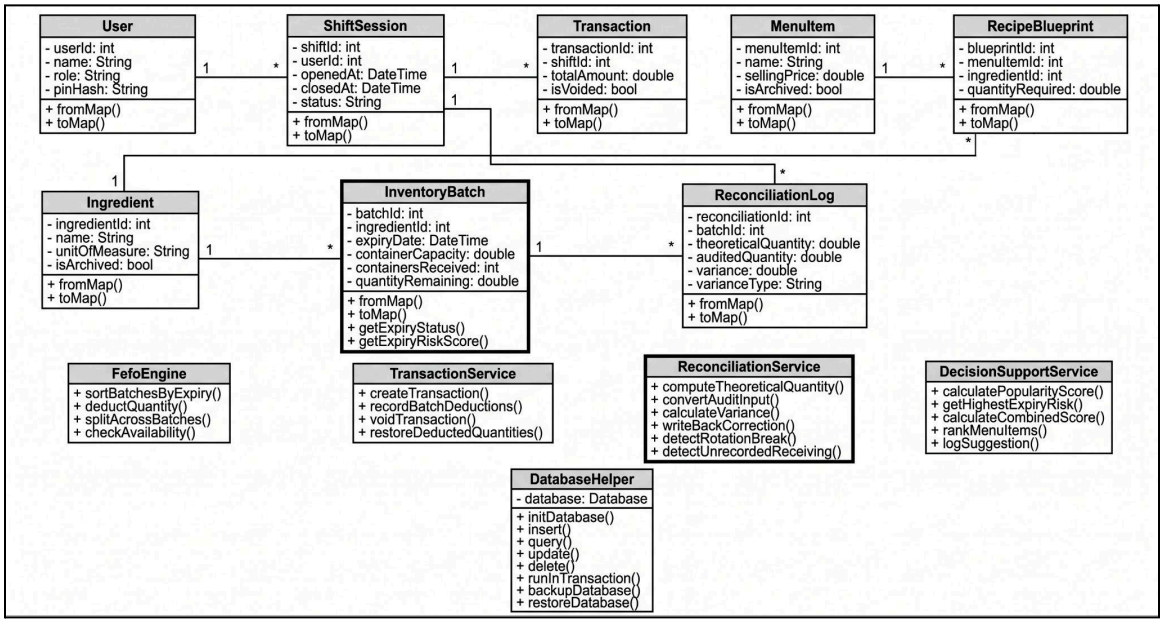


Figure 3.12 Class Diagram

Sequence Diagram

The sequence diagram shows the order in which objects call one another to complete an operation. Two sequences are presented, because the system has two operations that carry its main claims.

The first is the checkout sequence. When the Barista completes an order, the POS screen passes it to the transaction service, which opens a single database transaction so that every write either completes together or is rolled back together. The service reads the recipe blueprint for each ordered item, then calls the FEFO engine to determine which batches to deduct from. The engine sorts the available batches by expiry date, deducts from the earliest-expiring

batch first, and where that batch cannot cover the full amount, depletes it and carries the remainder to the next batch in order. Each deduction is written as its own record before the database transaction is committed. Once committed, the screen refreshes menu availability and requests updated suggestions from the decision support service.

The second is the reconciliation sequence. When the Barista closes a shift, the reconciliation service reads every active batch and computes its theoretical quantity, but does not display these values. The audit screen collects the sealed container count and the amount left in opened containers for each batch. Once the Barista submits, the service converts the submitted values using the container capacity stored on each batch, calculates the variance against the theoretical quantity, and writes the corrected quantity back to the batch record. It then writes the reconciliation log, checks for offsetting variances across batches of the same ingredient, and finally returns the results to the screen for display.

The order of steps in the second sequence is what enforces the blind audit. The theoretical quantity is computed near the beginning of the sequence but is not returned to the screen until after the count has been submitted and locked, so the Barista cannot adjust the count to match the system's expectation.

Figure 3.13 presents the checkout sequence and Figure 3.14 presents the reconciliation sequence.

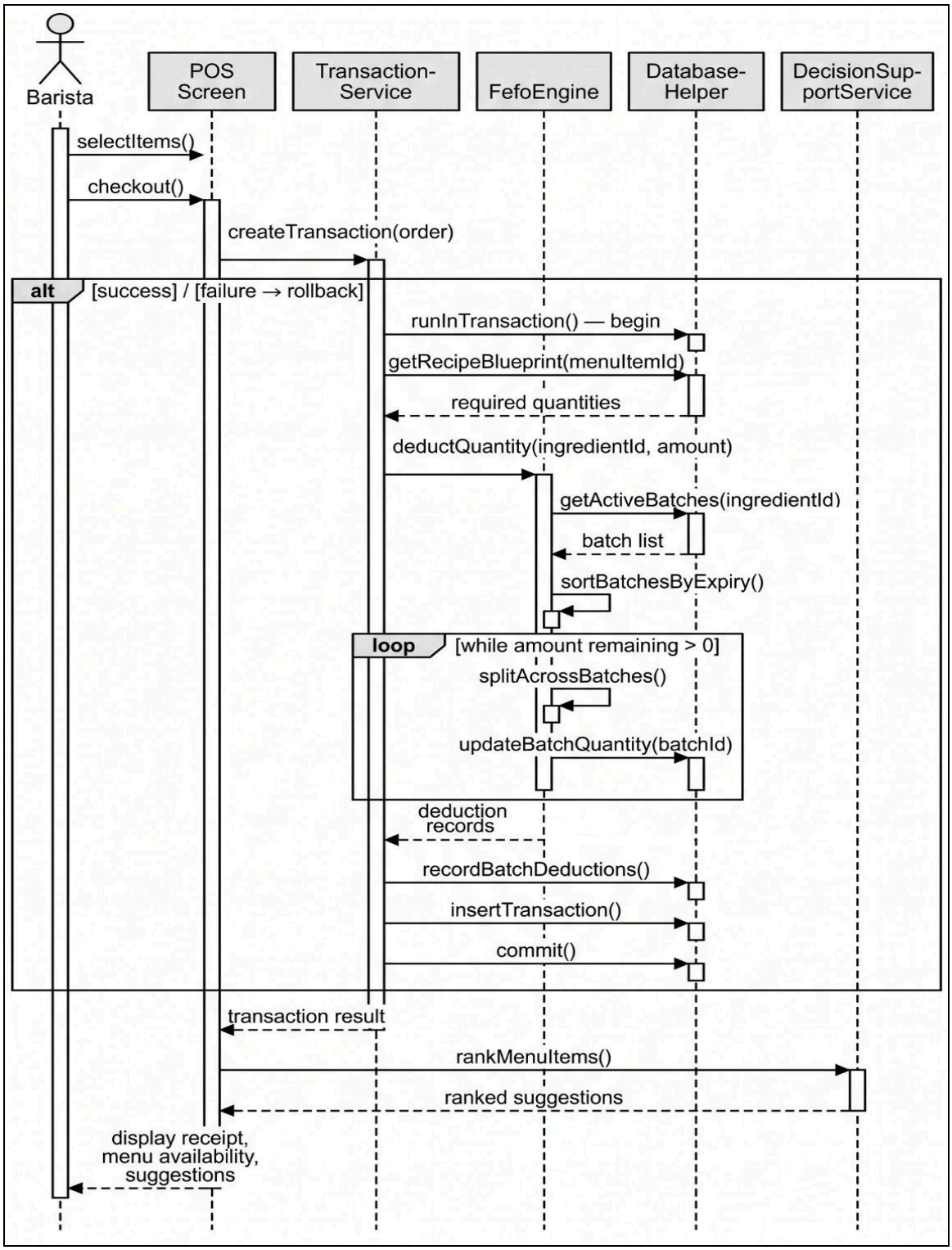


Figure 3.13 Sequence Diagram (Checkout)

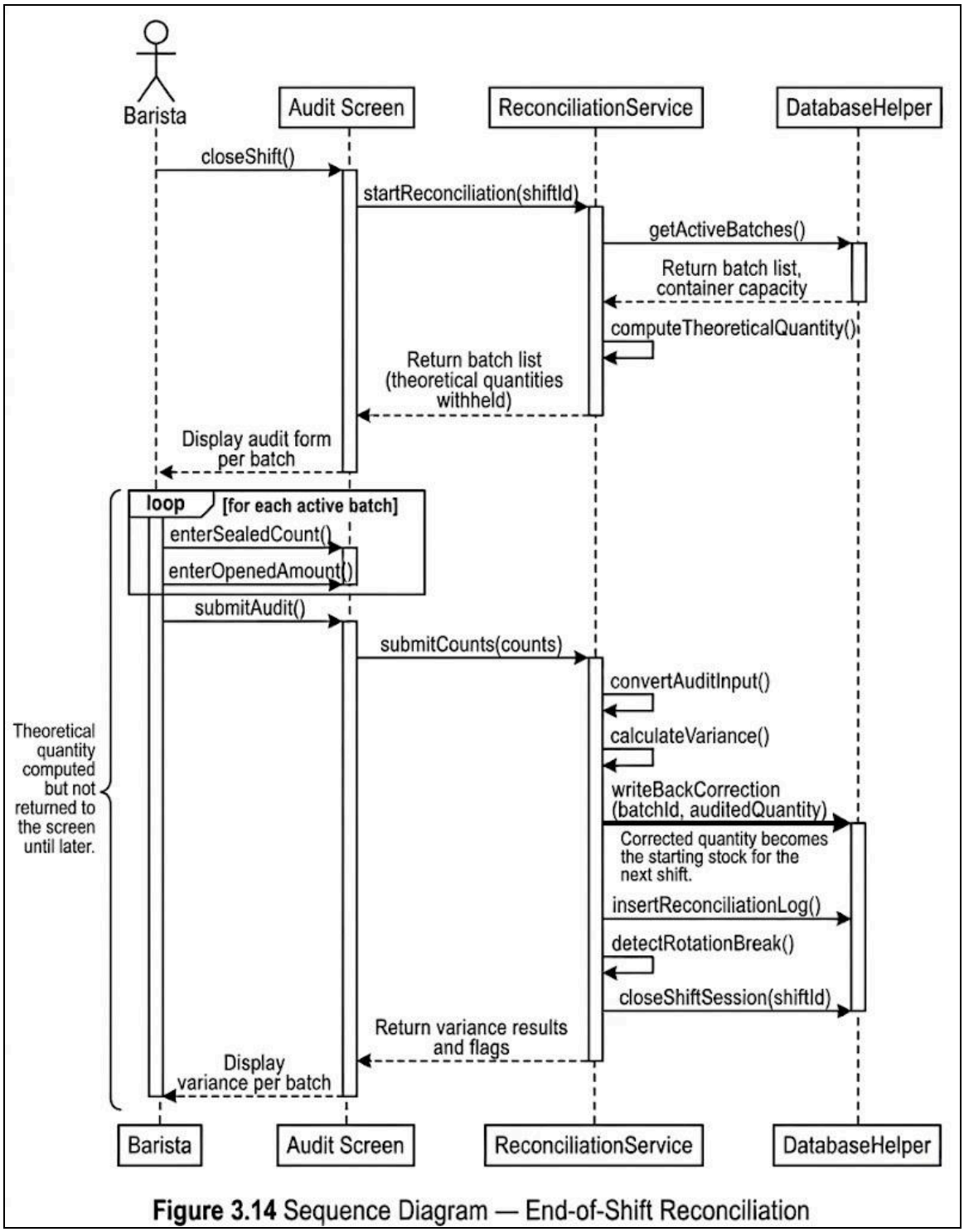


Figure 3.14 Sequence Diagram (Reconciliation)

Risk Assessment / Analysis

This section identifies the risks that could affect the accuracy or availability of the BrewManager system, and describes how each one is addressed. The risks are grouped by what causes them: the offline single-device architecture, the dependence on staff input, and the boundaries of what the system can detect.

Risk 1: Total Data Loss from Device Failure

Because the system stores all records in a local SQLite database on one tablet, physical damage, theft, or a factory reset would destroy every transaction, inventory batch, and reconciliation record on the device. There is no cloud copy to fall back on.

This risk is addressed through the database backup function, which copies the complete SQLite file to external or user-selected storage. Unlike the CSV and PDF exports, which are reports and cannot be reloaded, this copy preserves the full state of the system and can be restored onto a replacement device. The residual risk is that backup is started by the user rather than run automatically, so any activity recorded after the last backup would be lost. This is reduced by prompting the Manager to back up at the end of each operating week.

Risk 2: Incorrect Device Clock

The system has no network access, so it depends entirely on the tablet's internal clock to evaluate expiry dates and to route deductions under FEFO. If the clock is set incorrectly, batches may be marked with the wrong shelf-life status and deductions may be routed to the wrong batch.

This risk is addressed by storing the latest timestamp the system has recorded and comparing it against the current device time at launch. If the clock appears to have moved backwards, the system displays a warning to the user. This detects the most common form of clock error, but the system cannot independently verify the correct time, so a clock set forward or set to a plausible but wrong date will not be caught. Keeping the device clock correct remains the responsibility of the cafe's management.

Risk 3: Human Error During Audit Input

The accuracy of the reconciliation module depends on staff entering the physical count correctly. A miscounted sealed container or a misjudged fractional estimate produces a variance that does not reflect what actually happened, and the incorrect value is then written back to the batch record.

This risk is reduced in four ways. The audit asks for only two inputs per batch and requires no weighing, unit conversion, or arithmetic, which limits the opportunities for error. Each control starts at the value recorded during the previous shift, so a routine closing requires only a small adjustment rather than entry from zero. Because a carried-over value could otherwise be submitted without verification, each batch must be marked as checked before submission, which converts leaving a value unaltered from an omission into a deliberate confirmation. And where a submitted sealed count exceeds the total containers recorded for that batch, the system requests confirmation before accepting the value, since such a count is physically impossible rather than merely unexpected.

The residual risk is that a plausible but incorrect count will be accepted, since the system has no way to verify a physical count independently.

Risk 4: Incorrect Recipe Blueprints

All automated deduction depends on the recipe blueprints configured by the Manager. If a blueprint records the wrong quantity for an ingredient, every sale of that item deducts the wrong amount, and the resulting error accumulates until it is discovered.

This risk is treated as a configuration and measurement task rather than a software problem. Because independent coffee shops often prepare drinks by practiced estimation rather than measured amounts, time is set aside during development for on-site observation and measurement of actual pours. The reconciliation module also acts as a detection mechanism, since a consistently wrong blueprint produces a variance in the same direction on the same ingredient shift after shift, which appears as a pattern on the material loss dashboard.

Risk 5: Incorrect Receiving Entry

Expiry monitoring, FEFO routing, and reconciliation all draw on the batch records created by the Manager at receiving. If an expiration date is misread, a container capacity is entered incorrectly, or a purchase is not recorded at all, that batch will be tracked inaccurately for as long as it exists.

The system flags a counted quantity that exceeds recorded holdings as probable unrecorded receiving, which catches purchases that were never entered. It cannot, however, detect a mistyped expiry date or an incorrect

container capacity, since both produce internally consistent records. This risk is reduced by keeping the receiving screen simple and by requiring the Manager to enter each distinct expiry date and container size separately rather than in a single combined entry.

Risk 6: Deviation from Expiry-Order Stock Rotation

The system routes deductions to the earliest-expiring batch, but it cannot control which physical container staff take from the shelf. If staff open a later-expiring container while an earlier one remains, the system's records will no longer match which batch was actually consumed.

This risk is addressed through detection rather than prevention. Because reconciliation is performed per batch, using stock out of order produces a distinctive pattern: the batch the system assumed was consumed shows a surplus, while the batch actually used shows a deficit, and the two offset. The system flags this pattern for the Manager. Physical support is also provided by labeling containers with their batch at receiving and arranging stock so that earlier-expiring batches are positioned for first access. The residual risk is that a rotation break occurring alongside material loss of similar size may not be distinguishable from ordinary waste.

Risk 7: Unauthorized Access to Configuration

Because the system runs on a shared device behind the counter, an unauthorized user could reach configuration screens, alter recipe blueprints, or reverse transactions if access is not controlled.

This risk is addressed through the PIN-based permission matrix. Baristas can reach only the POS screen, the loss declaration, and the audit interface. Configuration, receiving, voiding, analytics, and data preservation are reserved for the Manager account. PINs are stored as hashes rather than plain text, the default Manager PIN seeded at installation must be changed before any other function becomes available, and an idle session lock returns the application to the PIN entry screen after a period of inactivity.

System Design

This section describes the interface layout of the BrewManager system and the reports it produces. The design follows the two access levels enforced by the permission matrix, so the screens available to each role are described separately.

User-Interface

The application is built for an Android tablet in landscape orientation. This layout was chosen because the POS screen needs to display a menu grid and a running order at the same time, and because the audit screen needs to show several batch entries at once without scrolling through a narrow column.

The interface is organized around the two roles. After a user enters a PIN, the system routes a Manager account to the back-office dashboard and a Barista account to the POS transaction screen. Neither role can reach the other's screens.

The Manager screens cover configuration and review. These are the ingredient management screen, the menu item management screen, the recipe blueprint editor, the inventory receiving screen, the expiry monitoring matrix, the material loss dashboard, the export and backup screen, and the user management screen. The receiving screen allows several batch entries to be recorded in one session, since a single purchase may contain containers with different expiry dates or container sizes.

The Barista screens cover shift operations. These are the POS transaction screen, the declare container lost dialog, and the end-of-shift reconciliation screen. The POS screen shows the menu as a grid of items with the running order beside it. Items whose ingredients have reached zero stock are grayed out and cannot be selected. Waste-prevention suggestions appear on the same screen together with the expiry condition that triggered them, so the barista sees why an item is being suggested rather than only which one.

The reconciliation screen presents one block per active batch. Each block shows the ingredient name, the batch expiry date, and the container capacity, so staff can identify which physical containers belong to which batch. Each block accepts two inputs: a count of sealed containers and a stepper for the amount left across all opened containers, in quarter-container steps. The opened-stock control displays the entered quantity as proportionally filled container graphics and offers single-tap selection of commonly occurring values, so that the figure can be compared against the shelf by eye rather than converted mentally. Each block is marked as checked once either control is adjusted or the quantity is

confirmed as unchanged, and the screen reports how many blocks have been checked. Theoretical quantities are not shown on this screen until the audit has been submitted.

Three design decisions apply across all screens. Controls are sized for touch input rather than for a mouse, since staff may be working quickly or with wet hands. Actions that change inventory records, such as voiding a transaction or submitting an audit, require a confirmation step. And color is used only to reinforce information that is also carried by text or position, so that expiry status remains readable without relying on color alone. The wireframes for each screen are presented in Figures 3.15 through 3.2X.

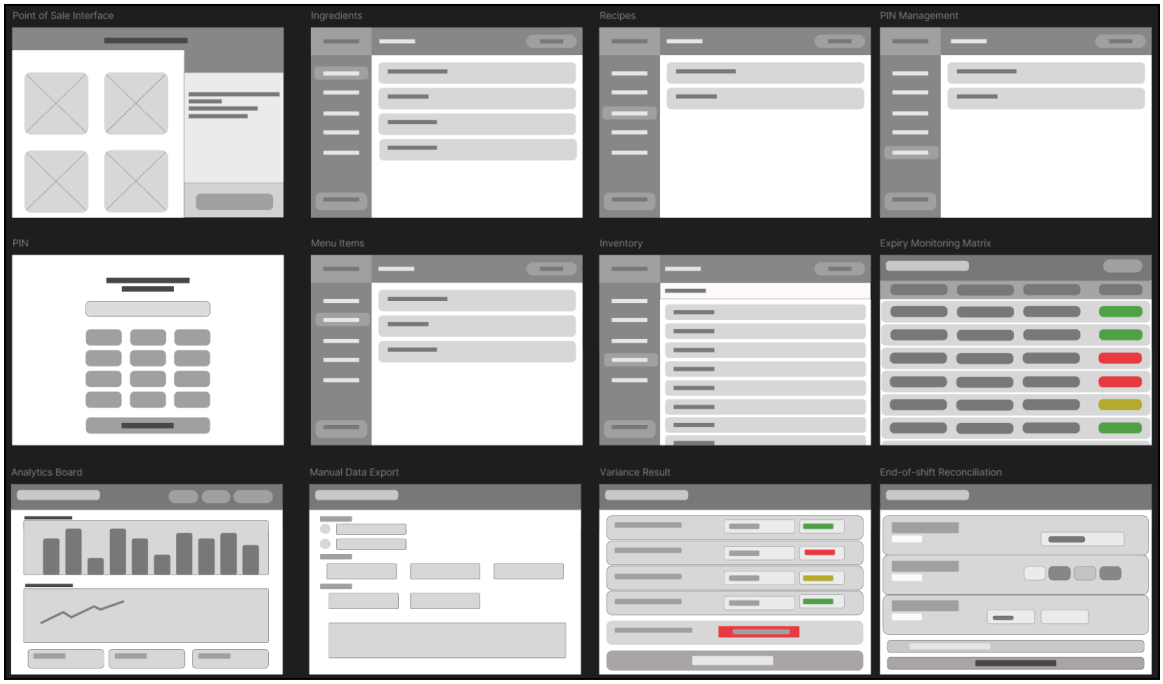


Figure 3.14 BrewManager Wireframes

Reports

The system produces two kinds of output. On-screen views allow the Manager to inspect current data within the application, and exported files allow records to be kept or printed outside it.

User Inquiry / View

The expiry monitoring matrix lists every active inventory batch with its ingredient name, expiry date, container capacity, and quantity remaining. Each row carries a color-coded status showing whether the batch expires in more than seven days, within seven days, or within forty-eight hours or sooner. The list is sorted by expiry date so the most urgent batches appear first.

The material loss dashboard presents variance per ingredient over time, drawn from the reconciliation logs. The Manager can filter by date range and by ingredient. Declared and unexplained losses are presented as separate ranked lists rather than as a combined total, because the two categories indicate different operational causes and call for different responses. Declared losses point to handling or storage practice, while unexplained losses point to over-pouring, unrecorded giveaways, or recipes that do not match what is actually prepared. Where several ingredients are shown together, quantities are expressed as container equivalents so that ingredients measured in different units can be compared, with the exact quantities in their own units shown alongside. Surpluses arising from unrecorded receiving are excluded from loss totals, since these are record corrections rather than material waste.

After each reconciliation is submitted, the system displays the result per batch, showing the theoretical quantity, the counted quantity, and the variance between them, together with any rotation or unrecorded receiving flags raised during the audit.

Print-out Report

The Manager can export five data files in CSV format and one summary report in PDF format to the tablet's internal storage. Each file is named for the dataset it contains and the period selected, and is timestamped so that repeated exports do not overwrite one another.

The material loss report lists every recorded loss over a selected date range, giving the ingredient, the quantity lost in its own unit of measurement, the equivalent in containers, and whether the loss was declared at the time or found as an unexplained shortfall during reconciliation.

The suggestion report lists every waste-prevention suggestion issued over a selected date range, with the expiry risk and popularity scores that produced it, the resulting combined score, and whether the suggested item was subsequently sold. This is the file used to calculate the module's acceptance rate during evaluation.

The sales report lists completed transactions over a selected date range, showing the date and time of each sale, the items sold, the quantity of each, the price at the time of sale, and the transaction total. Voided transactions are included and marked as such rather than removed, so that the record remains complete.

The inventory report lists all active batches with their ingredient, expiry date, container capacity, containers received, and quantity remaining, allowing current stock to be reviewed away from the device.

The reconciliation report lists audit results over a selected date range, showing the shift, the batch audited, the theoretical and counted quantities, the variance, and the variance type. This is the report a Manager would use to review material loss over a longer period than the dashboard displays.

In addition to these data files, the system produces a summary report in PDF format covering material loss and reconciliation results for the selected period. Unlike the CSV files, which are intended for analysis, this report is formatted for printing or for attachment to a document.

The system does not print physical receipts. Order summaries are shown on screen at the end of each transaction, which removes the cost of a receipt printer and of consumable paper. Sample layouts for each report are presented in Figures 3.2X through 3.2X.

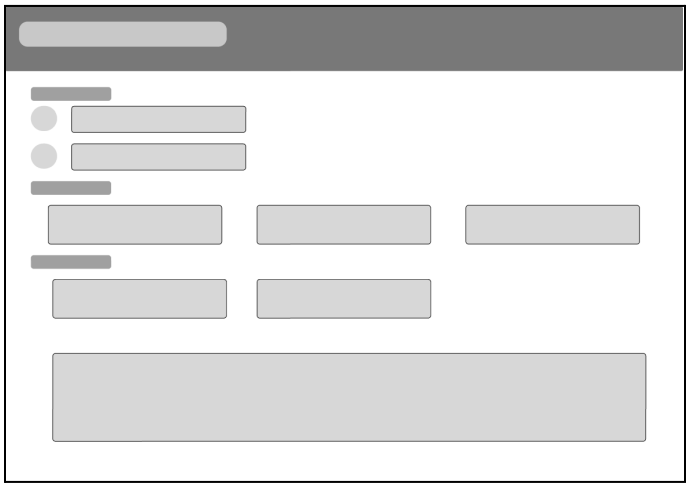


Figure 3.18 Manual Data Export Interface

System Development

Technical Feasibility

Technical feasibility examines whether the BrewManager system can be built and run with the tools, skills, and hardware available to the proponents and the client. The assessment covers the minimum hardware and software needed to run the system, whether those components work together, and why each technology was chosen.

The system is technically feasible because it uses established, freely available tools that the proponents are already familiar with, and because it runs on a single mid-range Android tablet rather than on a server or a network. No component of the system requires specialized hardware, paid licensing, or an internet connection.

Minimum Requirements

The following table presents the minimum hardware and software requirements for running BrewManager on the target deployment device.

Category	Requirement	Minimum Specification
Client Hardware	Device type	Android tablet
	Android version	Android 8.0 (Oreo) or higher
	Processor	Quad-core 1.4 GHz or equivalent
	Memory (RAM)	2 GB or higher
	Internal storage	8 GB available
	Display Size	8 inches or larger
	Display Resolution	1280 × 800 pixels

	Network	Not required
Development Hardware	Device Type	Laptop or desktop computer
	Operating System	Windows 10 (64-bit) or later, macOS 11 or later
	Processor	Intel Core i5 or equivalent
	Memory (RAM)	8 GB
	Internal Storage	20 GB available
	Testing Device	Physical Android tablet meeting client hardware specification
Development Software	Framework	Flutter SDK 3.0 or later
	Programming Language	Dart 3.0 or later
	Integrated Development Environment	Android Studio or Visual Studio Code
	Android Build Tools	Android SDK, API Level 26 or higher
	Database Package	sqflite
	Charting Package	fl_chart
	File Handling Packages	path_provider, csv, pdf
	Version Control	Git

Table 3.14 Minimum Requirements Table

Compatibility Checking

Compatibility was checked in three areas: between the software components, between the software and the target device, and between the system and the client's existing way of working.

The software components are compatible because they belong to the same ecosystem. Flutter compiles to native Android code, Dart is the language Flutter uses, and the SQLite packages selected are maintained for Flutter and are widely used in production applications. No component depends on a service

that must be reached over a network, so nothing in the stack fails when the device is offline.

The choice of Android 8.0 as the minimum version was made after checking the version requirements of the packages used. Setting the minimum at this level covers the large majority of Android tablets currently in use while remaining within the supported range of every package in the project. Testing is carried out on a physical tablet matching the minimum specification rather than only on higher-end hardware, so that performance on the weakest supported device is known rather than assumed.

Compatibility with the client's existing operations was also considered. The system does not require the cafe to change suppliers, container sizes, or preparation methods. It does not require a digital scale, a receipt printer, or a network connection. The only new practice it introduces is labeling containers with their batch at receiving, which supports a stock-rotation habit that food service already follows.

Relevance of the Technologies

Each technology was chosen for a specific reason rather than for familiarity alone.

Flutter was chosen because it produces a native Android application from a single codebase and provides a widget set suited to touch interfaces. Its layout system handles the landscape tablet screens the system depends on, and its state management allows the menu grid to update the moment a batch reaches zero stock, which is what makes the reactive grayout possible.

Dart is the language Flutter uses. Its asynchronous handling is relevant to this system because checkout involves several database writes that must complete in order, and its support for structured error handling allows a failed checkout to be caught and rolled back rather than leaving ingredients deducted for a sale that was never recorded.

SQLite was chosen because it is a relational database that runs entirely on the device with no server. This matters for two reasons. The system's data model depends on relationships between batches, deductions, transactions, and reconciliation logs, which a simple key-value store could not represent. And SQLite supports grouping several writes into one transaction, which is what allows a multi-item checkout with multi-batch splitting to either complete fully or roll back fully.

fl_chart was chosen for the material loss dashboard because it renders charts locally without contacting an external service, which keeps the system consistent with its offline design.

Version control through Git was used throughout development. Because the system is built in five dependent increments, being able to return to a known working state after a failed change was necessary rather than optional.

Taken together, these choices support the system's central requirement: that a small independent coffee shop can run a complete point-of-sale and inventory system on one affordable device, without a server, a subscription, or a network connection.

References

Best, J., Glock, C. H., Grosse, E. H., Rekik, Y., & Syntetos, A. (2022). On the causes of positive inventory discrepancies in retail stores. *International Journal of Physical Distribution & Logistics Management, 52*(5/6), 414–430. https://doi.org/10.1108/IJPDLM-07-2021-0291

Çalı, S., & Yilmaz Balaman, Ş. (2019). Improved decisions for marketing, supply and purchasing: Mining big data through an integration of sentiment analysis and intuitionistic fuzzy multi criteria assessment. *Computers & Industrial Engineering, 129*, 315–332. https://doi.org/10.1016/j.cie.2019.01.033

Carbajal-Vásquez, K. A., Piscoya-Alvites, R. A., Quiroz-Flores, J. C., García-López, Y., & Nallusamy, S. (2023). Minimization of smashed products in sustenance industries by lean and machine learning tools. *SSRG International Journal of Mechanical Engineering, 10*(10), 26–36. https://doi.org/10.14445/23488360/IJME-V10I10P102

DeHoratius, N., Holzapfel, A., Kuhn, H., Mersereau, A. J., & Sternbeck, M. (2023). Evaluating count prioritization procedures for improving inventory accuracy in retail stores. *Manufacturing & Service Operations Management, 25*(1), 288–306. https://doi.org/10.1287/msom.2022.1119

Destro, I. R., Staudt, F. H., Somensi, K., & Taboada, C. (2023). The impacts of inventory record inaccuracy and cycle counting on distribution center

performance. *Production, 33*, e20220077. https://doi.org/10.1590/0103-6513.20220077

Dewi, G. A. P. A., Dantes, G. R., & Divayana, D. G. H. (2023). Usability testing on inventory application using performance measurement, retrospective think aloud, and system usability scale technique. *Journal of Computer Networks, Architecture and High Performance Computing, 5*(1), 336–346. https://doi.org/10.47709/cnahpc.v5i1.2336

Divina, J., Olan, A., Perez, N. C., Sarmiento, M., Acepcion, R., & Cabututan, G. (2025). Data-driven point-of-sale and inventory system for Pastil Sa Tabi: Integrating sales forecasting algorithms and predictive analytics. *International Journal of Research and Innovation in Applied Science, 10*(10). https://doi.org/10.51584/IJRIAS.2025.1010000066

Doodala, A. N. K. (2023). Offline-first Android architecture for waste management in low connectivity zones. *International Journal of Emerging Trends in Computer Science and Information Technology, 4*(1), 201–209. https://doi.org/10.63282/3050-9246.IJETCSIT-V4I1P121

Geminarqi, E. R., & Purnomo, H. (2023). Improving operational management efficiency in the food and beverage industry: A systematic literature review. *Open Access Indonesia Journal of Social Sciences, 6*(5), 1143–1149. https://doi.org/10.37275/oaijss.v6i5.184

Gong, H., & Hui, H. (2024). Research on intelligent inventory management system based on automation and real-time data processing. *Mathematical Modeling and Algorithm Application, 3*(2), 39–43. https://doi.org/10.54097/4yafsk85

Gupta, S., Modgil, S., Bhattacharyya, S., & Bose, I. (2021). Artificial intelligence for decision support systems in the field of operations research: Review and future scope of research. *Annals of Operations Research, 308*(1–2), 215–274. https://doi.org/10.1007/s10479-020-03856-6

Herron, C. B., Garner, L. J., Siddique, A., Huang, T.-S., Campbell, J. C., Rao, S., & Morey, A. (2022). Building "first expire, first out" models to predict food losses at retail due to cold chain disruption in the last mile. *Frontiers in Sustainable Food Systems, 6*, Article 1018807. https://doi.org/10.3389/fsufs.2022.1018807

Hietasari, D. N., Subianto, D. I., & Adi, T. W. (2024). Conceptual framework of warehouse management system with auto suggesting features for FIFO/FEFO implementation towards lean warehousing. *AIP Conference Proceedings, 2951*(1), Article 030011. https://doi.org/10.1063/5.0181463

Kosch, T., Karolus, J., Zagermann, J., Reiterer, H., Schmidt, A., & Woźniak, P. W. (2023). A survey on measuring cognitive workload in human-computer interaction. *ACM Computing Surveys, 55*(13s), Article 283. https://doi.org/10.1145/3582272

Kurniawan, R. A., Arrozi, M., Endika, Y. V., & Afdhal, M. (2026). Development and usability evaluation of a web-based POS for hardware stores using SUS. *Jurnal Riset Informatika, 8*(2), 321–327. https://doi.org/10.34288/jri.v8i2.501

Maheshwari, P., Kamble, S., Pundir, A., Belhadi, A., Ndubisi, N. O., & Tiwari, S. (2021). Internet of things for perishable inventory management systems: An application and managerial insights for micro, small and medium enterprises. *Annals of Operations Research, 350*, 395–423. https://doi.org/10.1007/s10479-021-04277-9

Natividad, E. A. E., Musngi, A. M. M., Molinyawe, R. B., Rivera, E. A., Torres, R. C., & Moreno, D. E. L. (2024). System quality and micro food businesses' acceptance of the cloud-based point of sale system for inventory management. In *Proceedings of the 2024 15th International Conference on E-business, Management and Economics (ICEME '24)*. ACM. https://doi.org/10.1145/3691422.3691480

Neve, B. V., & Schmidt, C. P. (2021). Point-of-use hospital inventory management with inaccurate usage capture. *Health Care Management Science, 25*(1), 126–145. https://doi.org/10.1007/s10729-021-09573-1

Paradowski, B., Shekhovtsov, A., Bączkiewicz, A., Kizielewicz, B., & Sałabun, W. (2021). Similarity analysis of methods for objective determination of weights in multi-criteria decision support systems. *Symmetry, 13*(10), Article 1874. https://doi.org/10.3390/sym13101874

Patel, S., & Naini, R. (2025). Human-centered design of integrated food service management systems: Reducing cognitive load in resource-constrained kitchen operations. In *AHFE 2025 Hawaii International Conference on Human Factors in Design, Engineering, and Computing*. AHFE International. https://doi.org/10.54941/ahfe1006820

Pothineni, S. H. (2024). Offline-first mobile architecture: Enhancing usability and resilience in mobile systems. *Journal of Artificial Intelligence General Science (JAIGS), 7*(1), 320–326. https://doi.org/10.60087/jaigs.v7i01.387

Ramidi, M. (2022). Developing resilient offline-first architectures for mobile health and clinical research applications. *International Journal of Computer Technology and Electronics Communication (IJCTEC), 5*(1), 4518–4529. https://doi.org/10.15680/IJCTECE.2022.0501005

Rekik, Y., Oliva, R., Glock, C. H., & Syntetos, A. A. (2026). Inventory record inaccuracy in grocery retailing: Impact of promotions and product perishability, and targeted effect of audits. *Journal of Business Logistics, 47*(1). https://doi.org/10.1111/jbl.70079

Salih, H. S., Ghazi, M., & Aljanabi, M. (2023). Implementing an automated inventory management system for small and medium-sized enterprises. *Iraqi Journal for Computer Science and Mathematics, 4*(2), 238–244. https://doi.org/10.52866/ijcsm.2023.02.02.021

Sari, N. N. K., & Purba, F. F. (2021). Aplikasi kasir mobile berbasis Android untuk usaha mikro kecil dan menengah. *Journal of Information Technology and Computer Science (JOINTECOMS), 1*(3), 256. https://doi.org/10.47111/jointecoms.v1i3.8820

Shu, Z., Carrasco, R. A., Sánchez-Montañés, M., & Portela García-Miguel, J. (2024). A multi-criteria decision support model for restaurant selection based on users' demand level: The case of Dianping.com. *Information Processing & Management, 61*(3), Article 103650. https://doi.org/10.1016/j.ipm.2024.103650

Suárez, C. A., Guaño, W. A., Pérez, C. C., & Roa-López, H. (2024). Multi-objective optimization for perishable product dispatch in a FEFO system for a food bank single warehouse. *Operations Research Perspectives, 12*, Article 100304. https://doi.org/10.1016/j.orp.2024.100304

Taufiqqurrachman, T., Nurrohmah, A., Warno, W., & Kusumawati, S. W. (2025). Android-based inventory information system for food expiration date alerts at Pepper Lunch restaurants. *Jurnal Ilmiah METADATA, 7*(2), 50–64. https://doi.org/10.47652/metadata.v7i2.612

Ugbebor, F., Adeteye, M., & Ugbebor, J. (2024). Automated inventory management systems with IoT integration to optimize stock levels and reduce carrying costs for SMEs: A comprehensive review. *Journal of*

Artificial Intelligence General Science (JAIGS), 6(1), 306–340. https://doi.org/10.60087/jaigs.v6i1.257

van der Waa, J., Nieuwburg, E., Cremers, A., & Neerincx, M. (2021). Evaluating XAI: A comparison of rule-based and example-based explanations. *Artificial Intelligence, 291*, Article 103404. https://doi.org/10.1016/j.artint.2020.103404

Wulandari, B., Rahman, D. F., Fitriyanto, D., Dewanto, S. A., & Munir, M. (2026). Development and ISO 25010 evaluation of a web-based laboratory and workshop inventory system. *Jurnal Ilmiah Ilmu Terapan Universitas Jambi, 10*(1). https://doi.org/10.22437/jiituj.v10i1.47430

Appendices

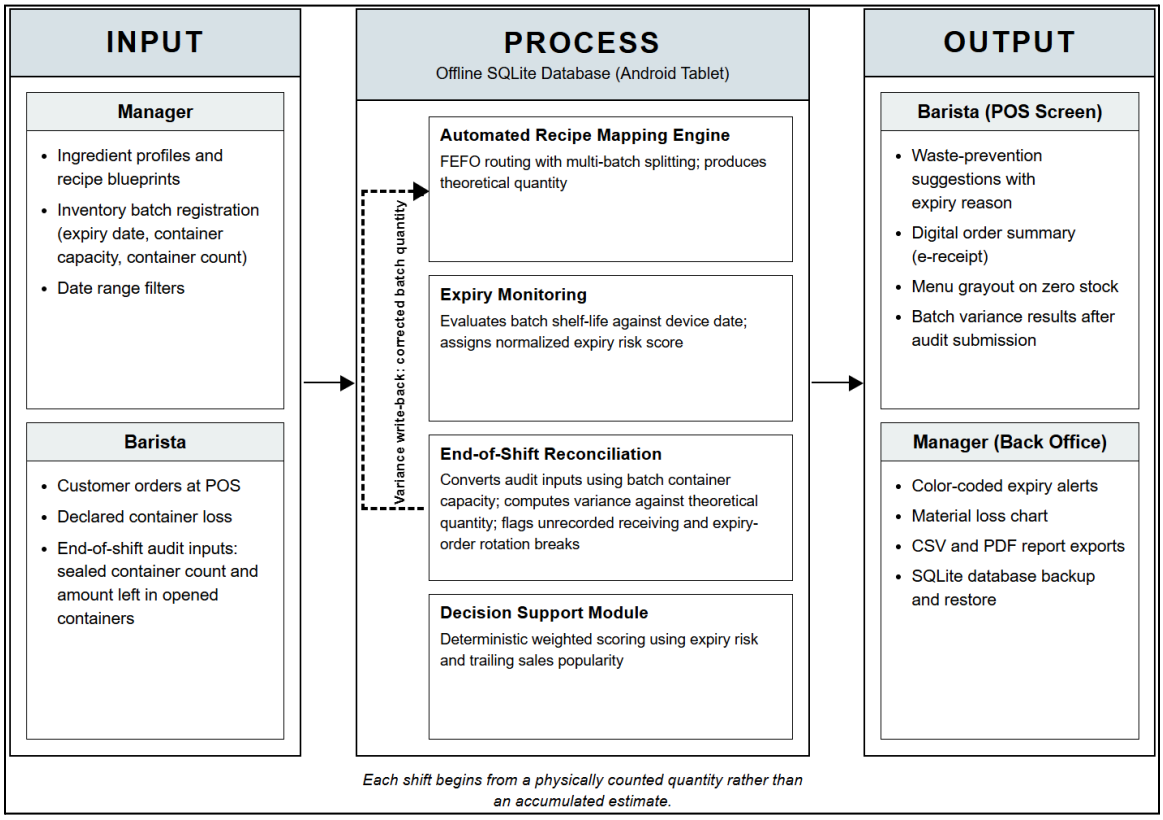


Figure 2.1 Conceptual Framework

Component	Specification
Development Framework	Flutter SDK (Android tablet, landscape layout)
Programming Language	Dart
Local Database	SQLite via database_helper.dart
State Management	Provider or Riverpod
Chart Library	fl_chart
Target Device	Android tablet
IDE	Android Studio or Visual Studio Code
Version Control	Git

Export Formats	CSV, PDF
Backup Format	SQLite database file copy
Evaluation Instruments	ISO 25010, System Usability Scale (SUS), controlled accuracy testing, timed procedure comparison

Table 3.1 Raw Materials and Tools Table

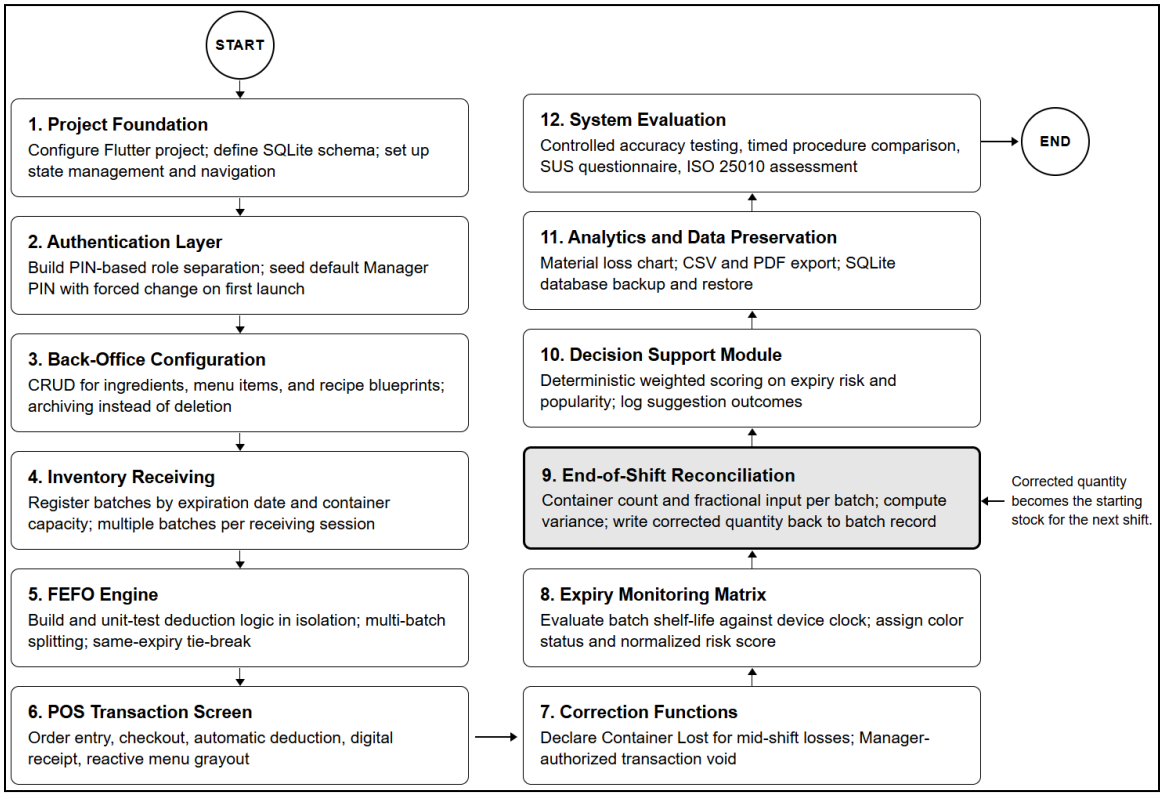


Figure 3.1 Project Development Flowchart

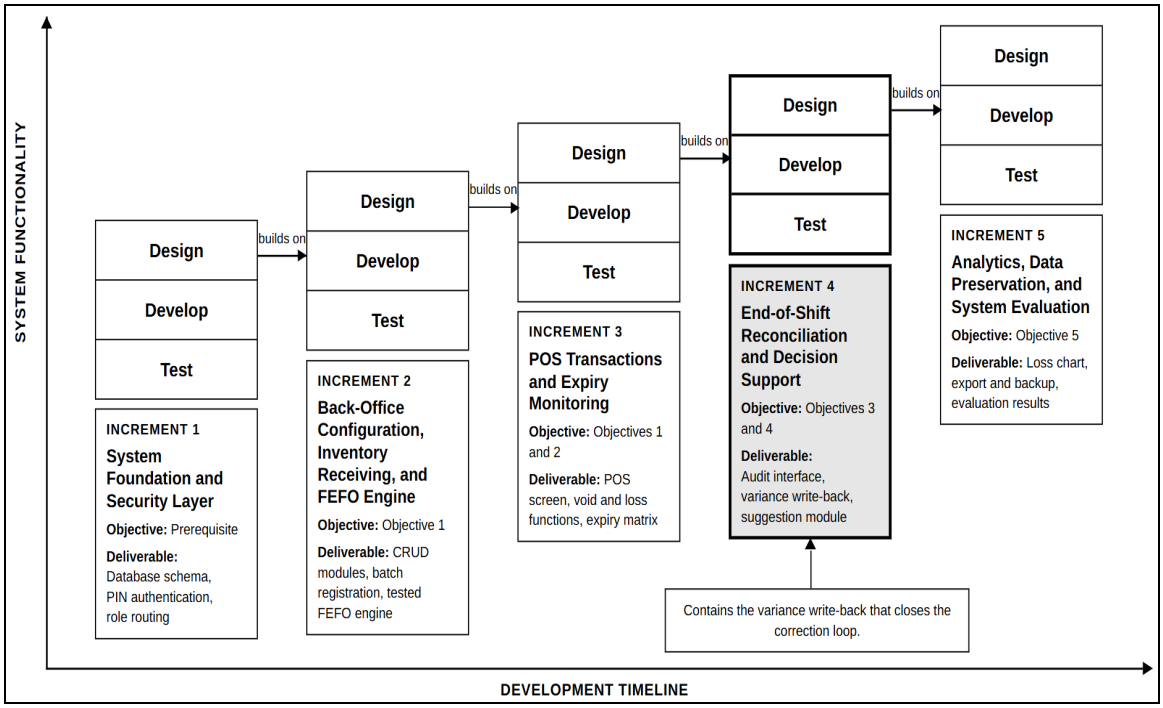


Figure 3.2 Incremental Model

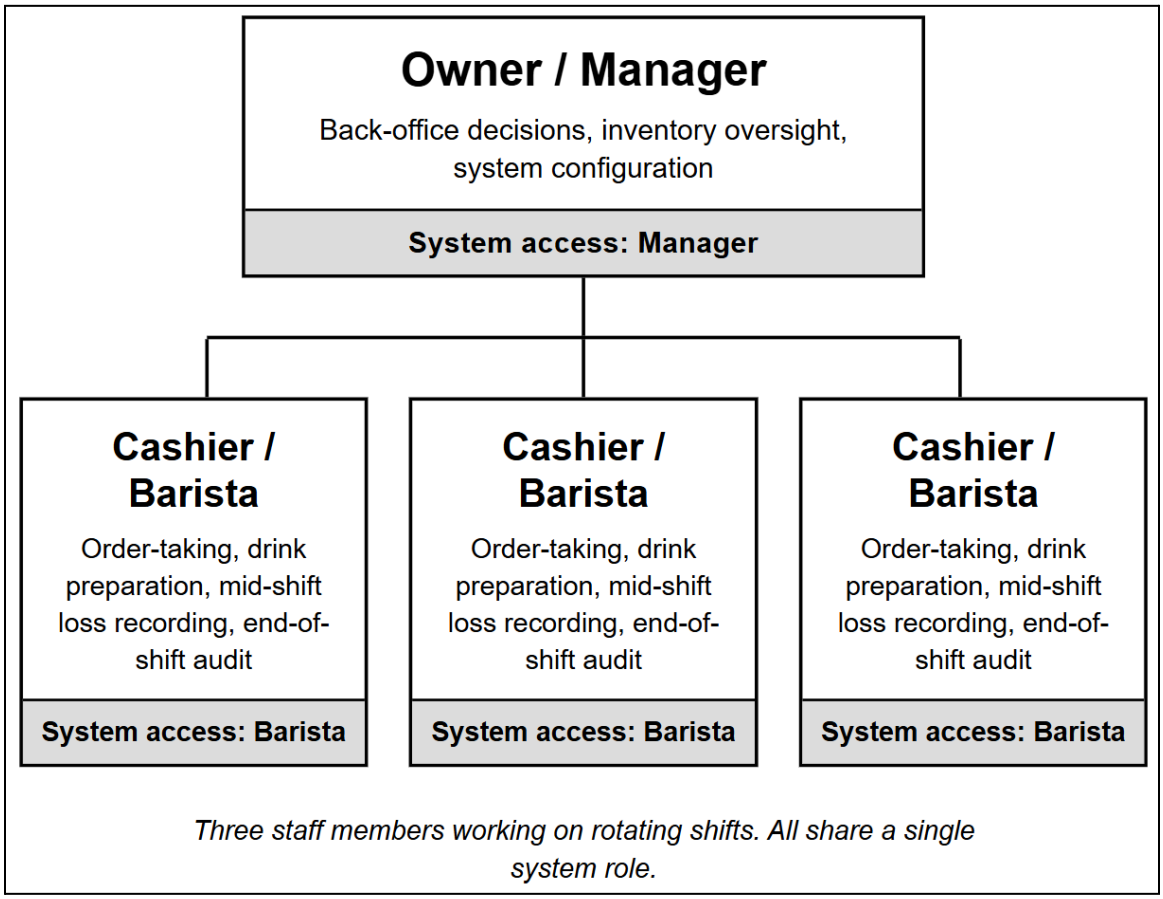


Figure 3.3 Organizational Chart

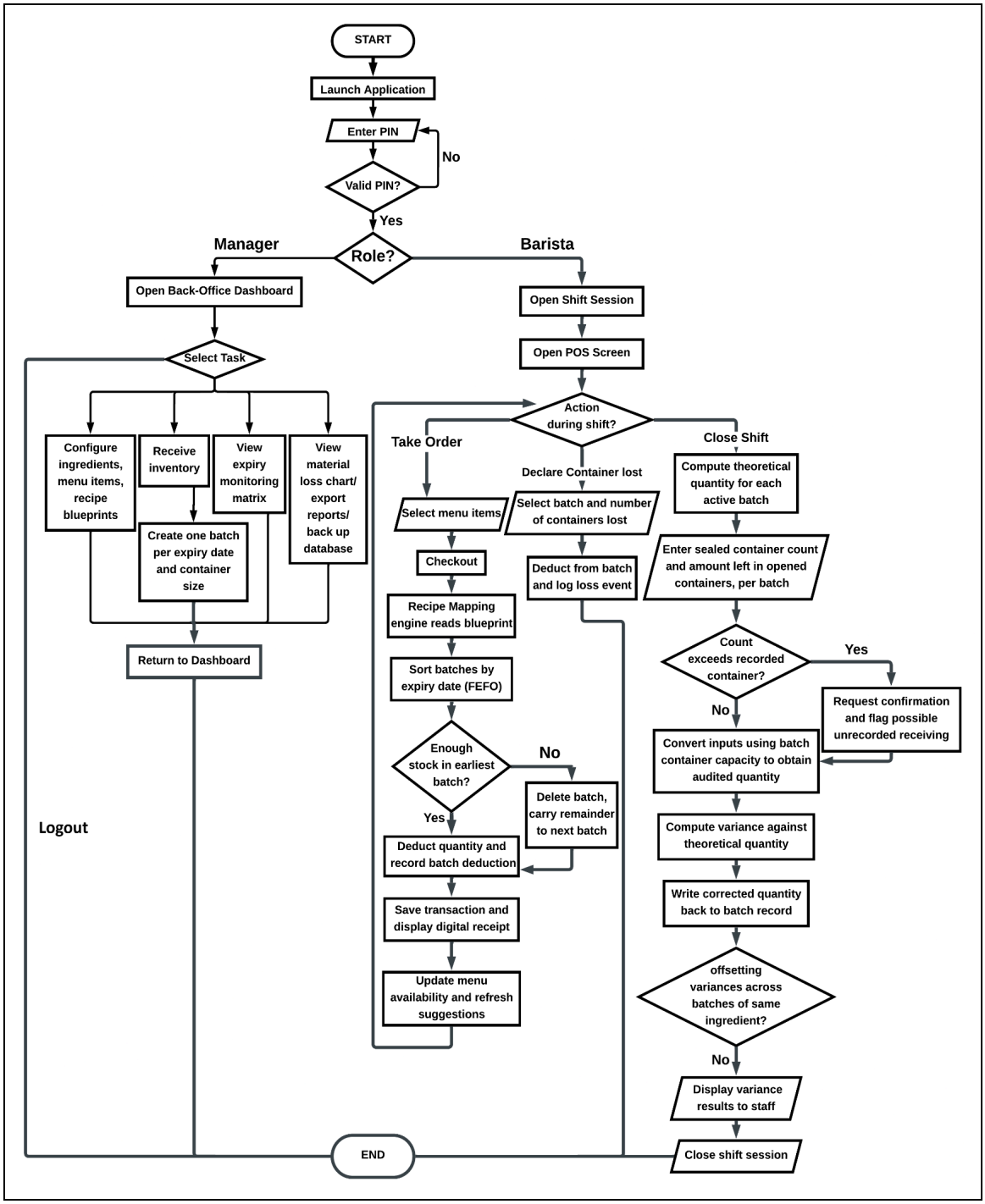


Figure 3.4 System Flowchart

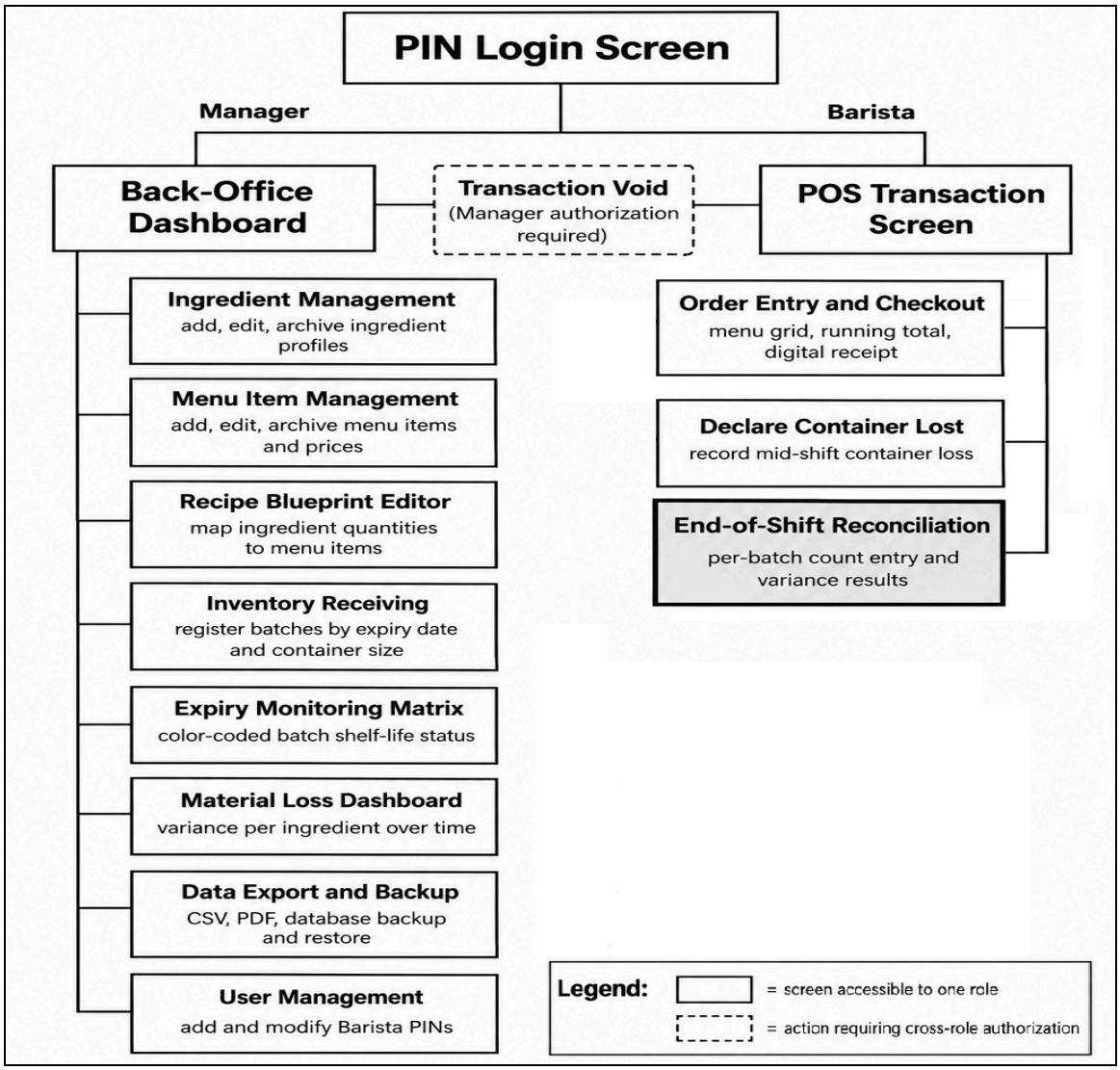


Figure 3.5 Site Map

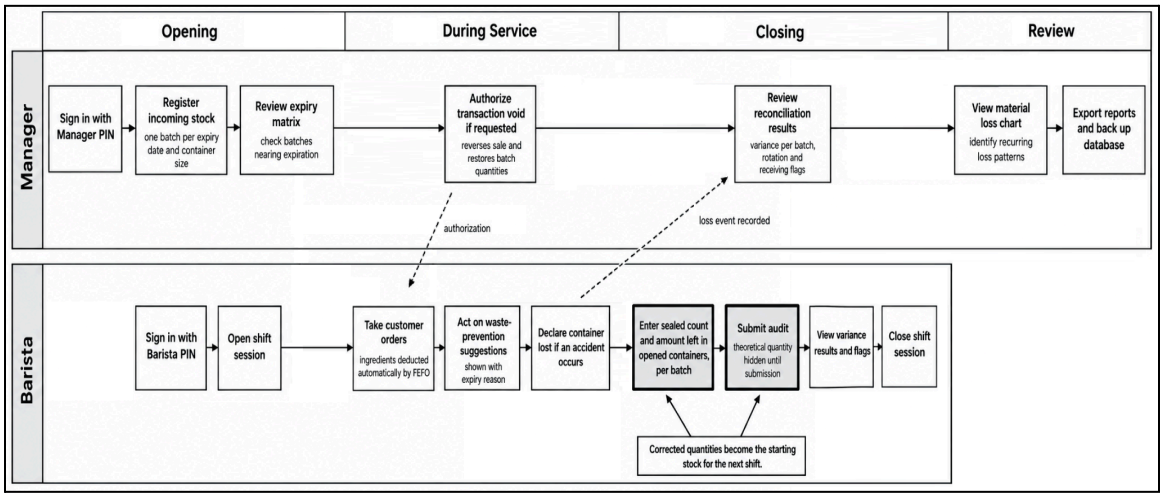


Figure 3.6 User Journey Map

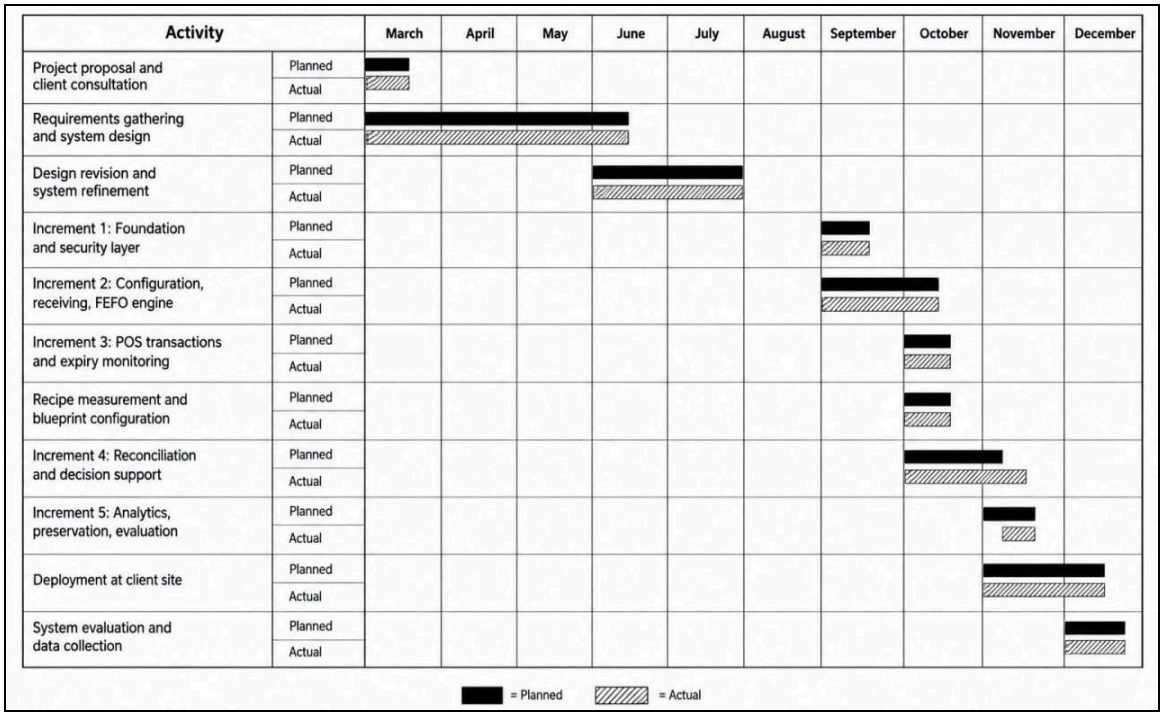


Figure 3.7 Gantt Chart

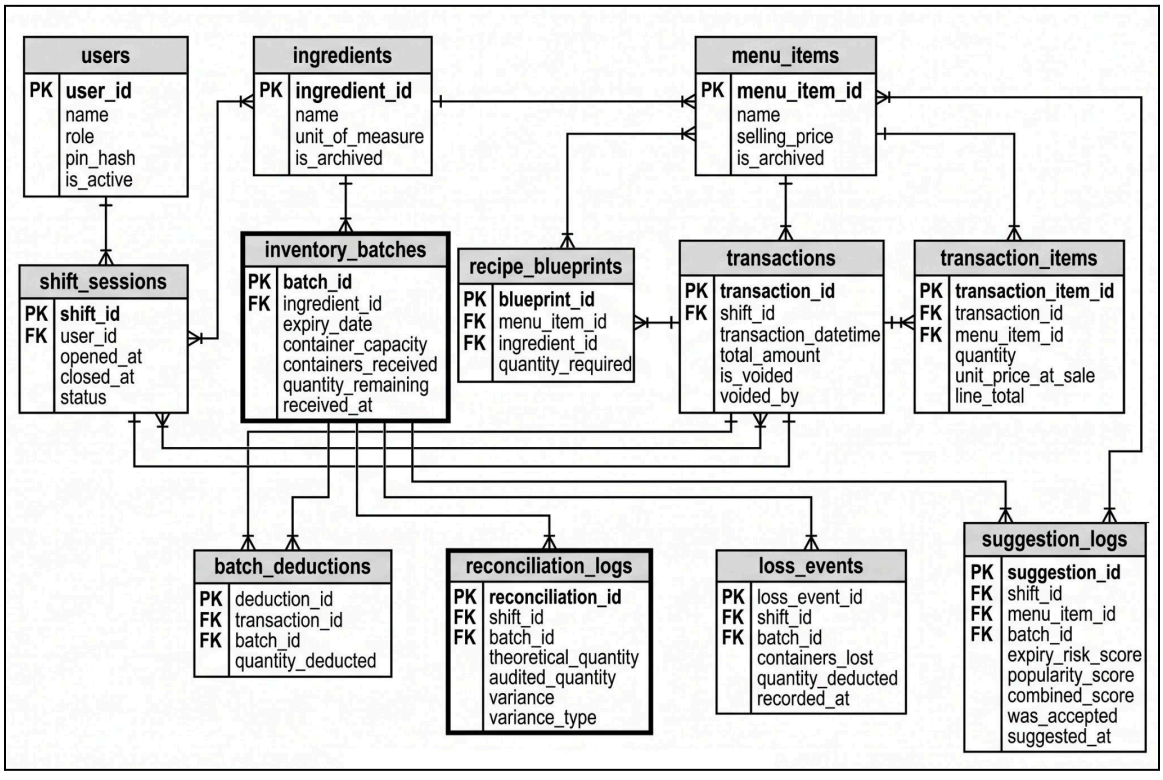


Figure 3.8 Entity Relationship Diagram

Field Name	Data Type	Constraint	Description
user_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each user account
name	TEXT	Not Null	Display name of the staff member
role	TEXT	Not Null	Access level of the account, either Manager or Barista
pin_hash	TEXT	Not Null	Hashed value of the user's PIN so the PIN is never held in plain text
is_active	INTEGER	Not Null, Default 1	Indicates whether the account is currently in use

Table 3.2 Data Dictionary Table (users)

Field Name	Data Type	Constraint	Description
shift_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each shift session
user_id	INTEGER	Foreign Key, Not Null	References the user who opened the shift
opened_at	TEXT	Not Null	Timestamp when the shift began
closed_at	TEXT	Nullable	Timestamp when the shift was closed through reconciliation
status	TEXT	Not Null	Current state of the shift, either open or closed

Table 3.3 Data Dictionary Table (shift_sessions)

Field Name	Data Type	Constraint	Description
ingredient_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each raw ingredient
name	TEXT	Not Null, Unique	Name of the ingredient
unit_of_measure	TEXT	Not Null	Base unit used for all quantities of this ingredient, such as milliliters or grams
is_archived	INTEGER	Not Null, Default 0	Marks the ingredient as retired while preserving its historical records

Table 3.4 Data Dictionary Table (ingredients)

Field Name	Data Type	Constraint	Description
batch_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each inventory batch
ingredient_id	INTEGER	Foreign Key, Not Null	References the ingredient this batch holds
expiry_date	TEXT	Not Null	Expiration date shared by all containers in this batch
container_capacity	REAL	Not Null	Full quantity held by one unopened container in this batch
containers_received	INTEGER	Not Null	Number of containers recorded when the batch was received
quantity_remaining	REAL	Not Null	Current quantity

			remaining in the batch, updated by deductions and by reconciliation
received_at	TEXT	Not Null	Timestamp when the batch was registered
corrected_by	INTEGER	Foreign Key, Nullable	User who last corrected figures entered wrongly at receiving; null if the batch has never been corrected
corrected_at	TEXT	Nullable	Timestamp of that correction
previous_quantity	REAL	Nullable	The quantity remaining as it stood immediately before that correction

Table 3.5 Data Dictionary Table (inventory_batches)

Field Name	**Data Type**	**Constraint**	**Description**
menu_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each menu item
name	TEXT	Not Null, Unique	Name of the beverage or item offered for sale
selling_price	REAL	Not Null	Current selling price of the item
is_archived	INTEGER	Not Null, Default 0	Marks the item as removed from the menu while preserving its sales history
is_ingredient_tracked	INTEGER	Not Null, Default 1	Indicates whether the

			item consumes ingredients from inventory. Prepared items bought ready to sell, such as pastries, are sold without a recipe and are not deducted from stock

Table 3.6 Data Dictionary Table (menu_items)

Field Name	Data Type	Constraint	Description
blueprint_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each blueprint entry
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item this entry belongs to
ingredient_id	INTEGER	Foreign Key, Not Null	References the required ingredient
quantity_required	REAL	Not Null	Amount of the ingredient needed to prepare one unit of the menu item

Table 3.7 Data Dictionary Table (recipe_blueprints)

Field Name	Data Type	Constraint	Description
transaction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each completed sale
shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which the sale occurred
transaction_datetime	TEXT	Not Null	Timestamp when the sale was completed
total_amount	REAL	Not Null	Total value of the sale

is_voided	INTEGER	Not Null, Default 0	Indicates whether the sale has been reversed
voided_by	INTEGER	Nullable	References the Manager account that authorized the reversal

Table 3.8 Data Dictionary Table (transactions)

Field Name	Data Type	Constraint	Description
transaction_item_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each line of a sale
transaction_id	INTEGER	Foreign Key, Not Null	References the sale this line belongs to
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item sold
quantity	INTEGER	Not Null	Number of units of the item sold in this line
unit_price_at_sale	REAL	Not Null	Selling price of the item at the moment of sale, stored so later price changes do not alter past records
line_total	REAL	Not Null	Total value of this line, calculated from quantity and unit price at sale

Table 3.9 Data Dictionary Table (transaction_items)

Field Name	Data Type	Constraint	Description
deduction_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each individual deduction

transaction_id	INTEGER	Foreign Key, Not Null	References the sale that triggered the deduction
batch_id	INTEGER	Foreign Key, Not Null	References the specific batch the quantity was taken from
quantity_deducted	REAL	Not Null	Amount deducted from this batch for this sale

Table 3.10 Data Dictionary Table (batch_deductions)

Field Name	**Data Type**	**Constraint**	**Description**
reconciliation_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each audited batch result
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the audit was performed
batch_id	INTEGER	Foreign Key, Not Null	References the batch that was audited
theoretical_quantity	REAL	Not Null	Quantity the system calculated should remain, based on recorded deductions
audited_quantity	REAL	Not Null	Quantity derived from the physical count submitted by staff
variance	REAL	Not Null	Difference between theoretical and audited quantity, negative for loss and positive for surplus

variance_type	TEXT	Not Null	Category of the variance, recorded as unexplained, declared loss, or unrecorded receiving

Table 3.11 Data Dictionary Table (reconciliation_logs)

Field Name	Data Type	Constraint	Description
loss_event_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each declared mid-shift loss
shift_id	INTEGER	Foreign Key, Not Null	References the shift session in which the loss occurred
batch_id	INTEGER	Foreign Key, Not Null	References the batch the loss was taken from
containers_lost	REAL	Not Null	Number of containers lost, recorded as sealed containers or as a single opened container
quantity_deducted	REAL	Not Null	Amount deducted from the batch, calculated from containers lost and container capacity
recorded_at	TEXT	Not Null	Timestamp when the loss was declared

Table 3.12 Data Dictionary Table (loss_events)

Field Name	Data Type	Constraint	Description
suggestion_id	INTEGER	Primary Key, Auto Increment	Unique identifier for each suggestion issued

shift_id	INTEGER	Foreign Key, Not Null	References the shift session during which the suggestion appeared
menu_item_id	INTEGER	Foreign Key, Not Null	References the menu item that was suggested
batch_id	INTEGER	Foreign Key, Not Null	References the batch whose expiry risk triggered the suggestion
expiry_risk_score	REAL	Not Null	Normalized expiry risk value used in the scoring calculation
popularity_score	REAL	Not Null	Normalized trailing seven-day sales popularity used in the scoring calculation
combined_score	REAL	Not Null	Weighted result of the two scores, ranking the item against other available items
was_accepted	INTEGER	Not Null, Default 0	Indicates whether the suggested item was sold after the suggestion appeared
suggested_at	TEXT	Not Null	Timestamp when the suggestion was displayed

Table 3.13 Data Dictionary Table (suggestion_logs)

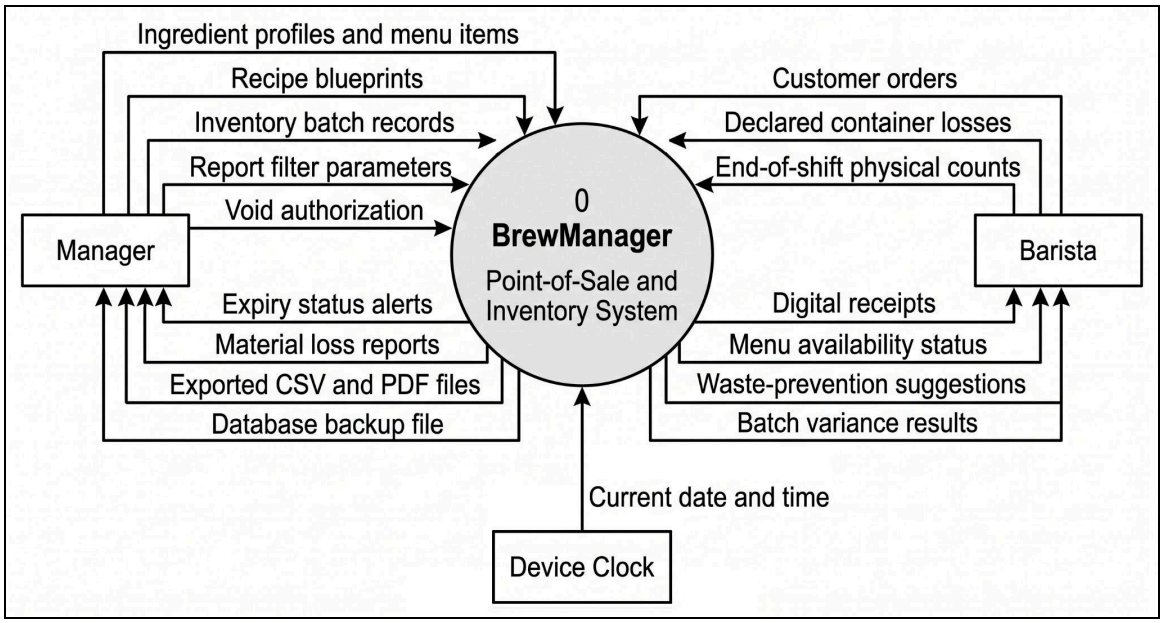


Figure 3.9 Context Diagram

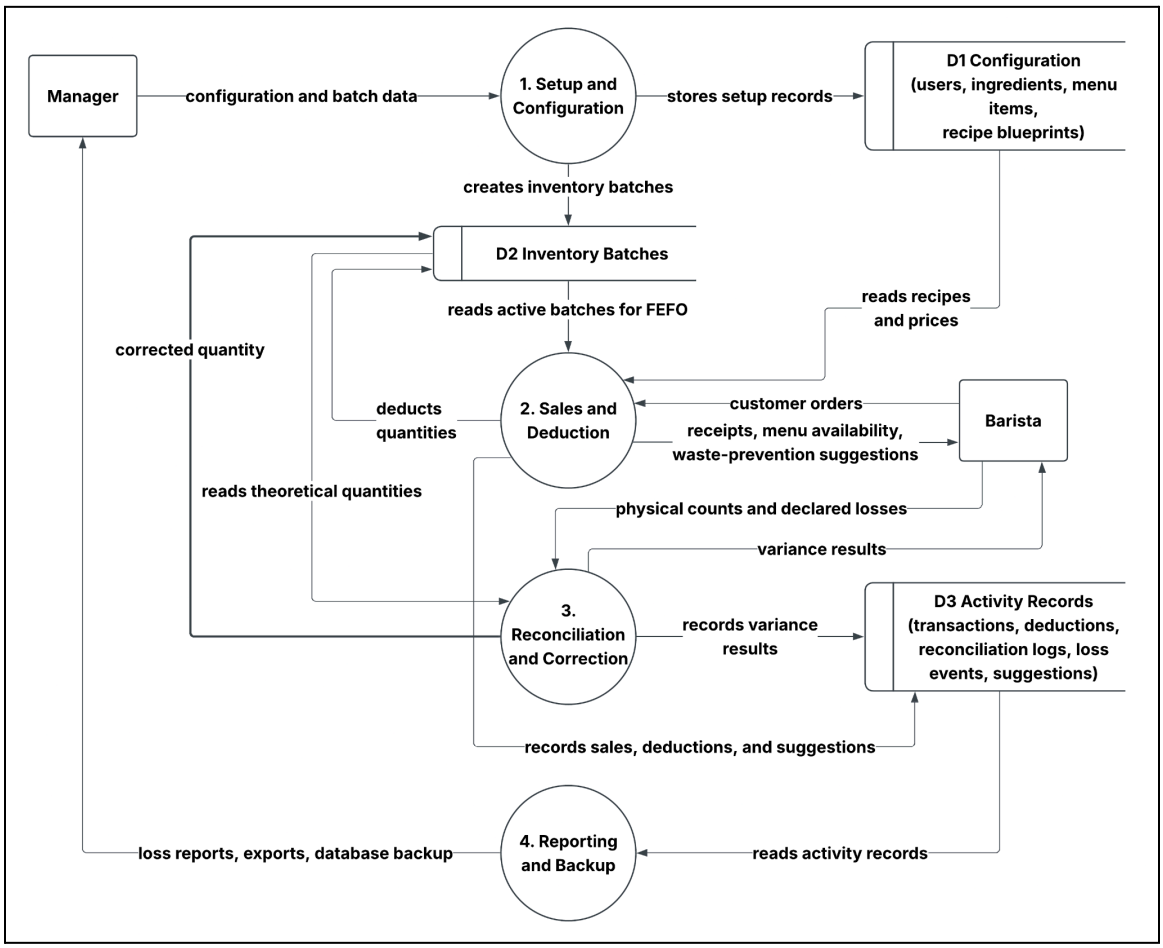


Figure 3.10 Data Flow Diagram

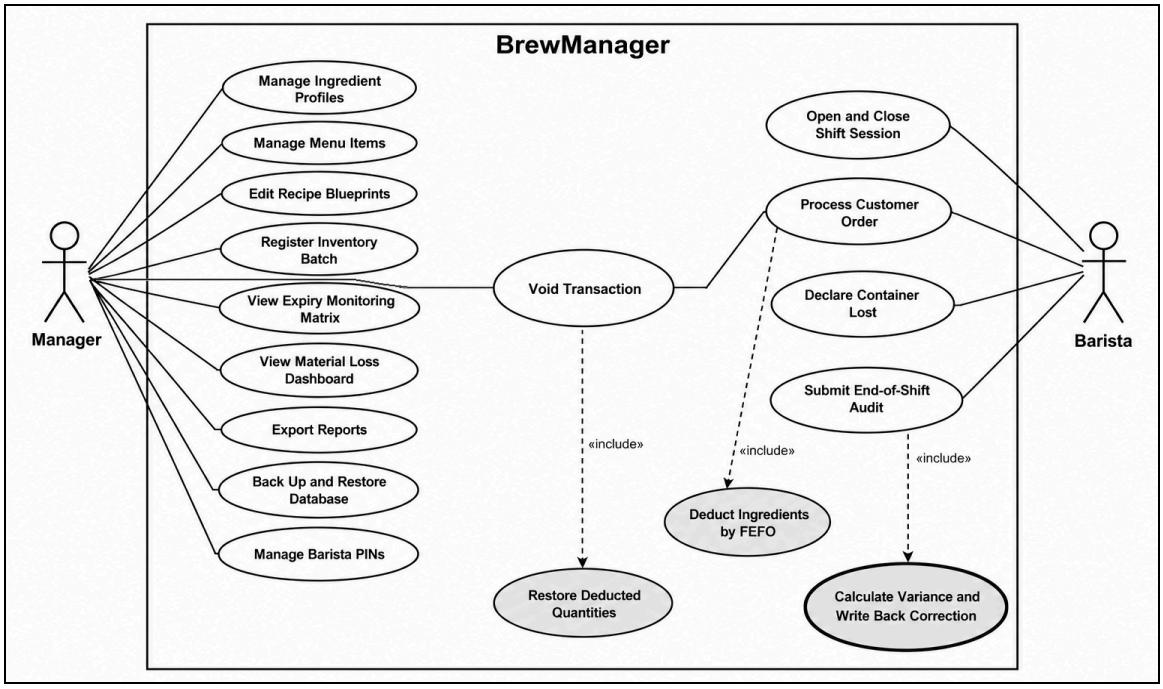


Figure 3.11 Use Case Diagram

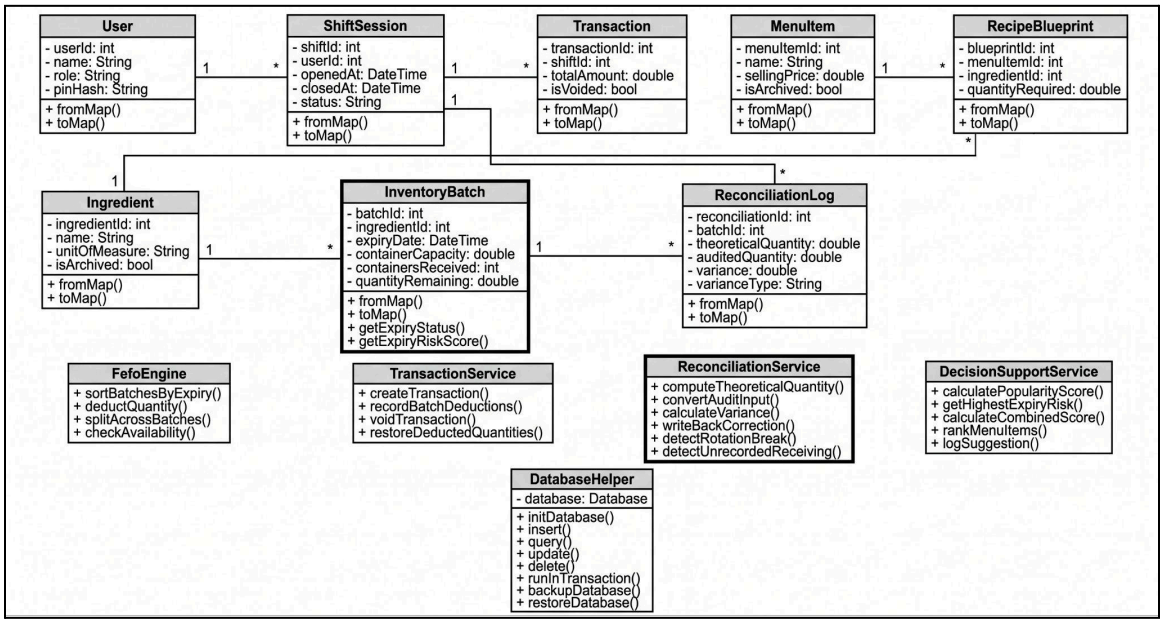


Figure 3.12 Class Diagram

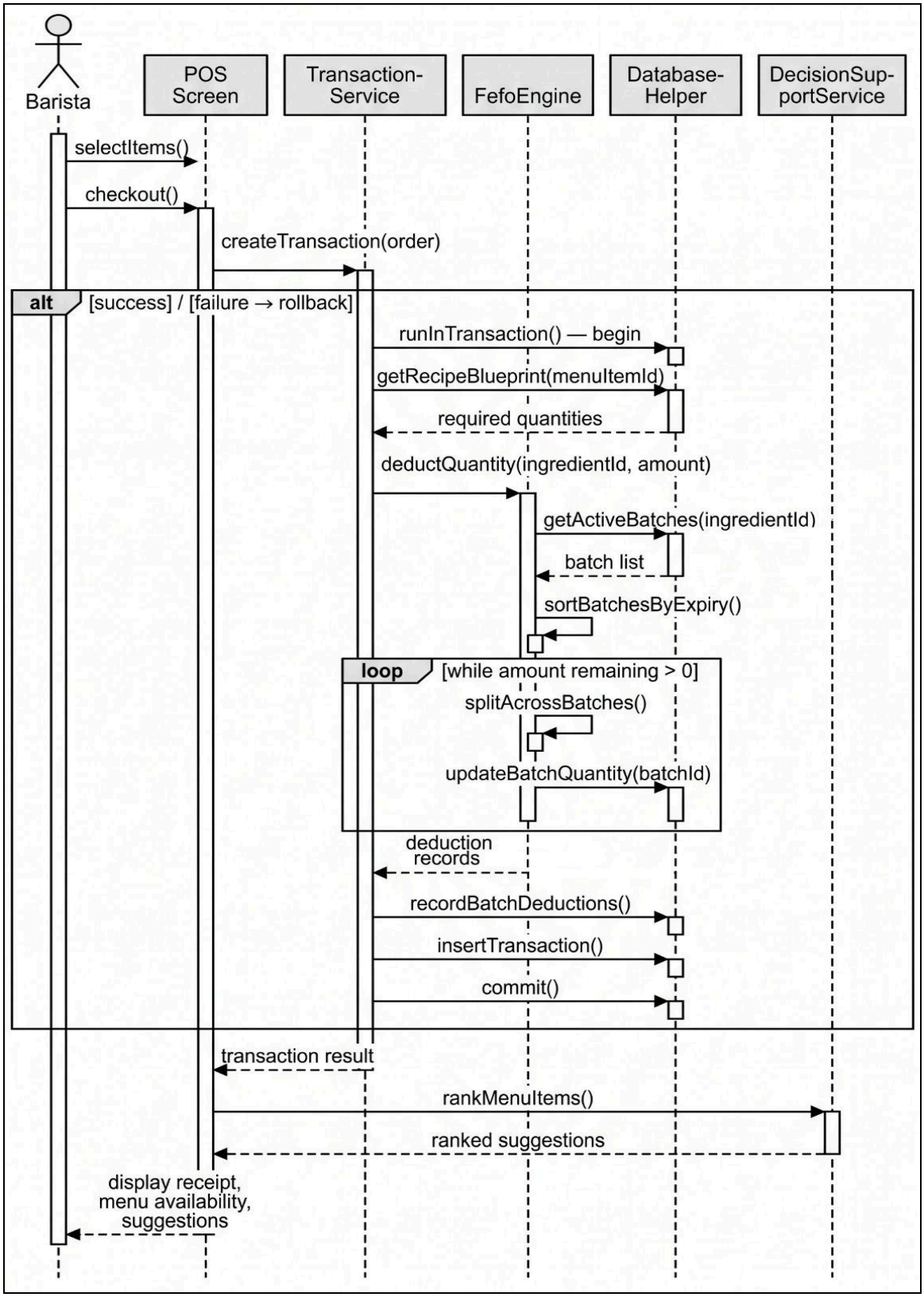


Figure 3.13 Sequence Diagram (Checkout)

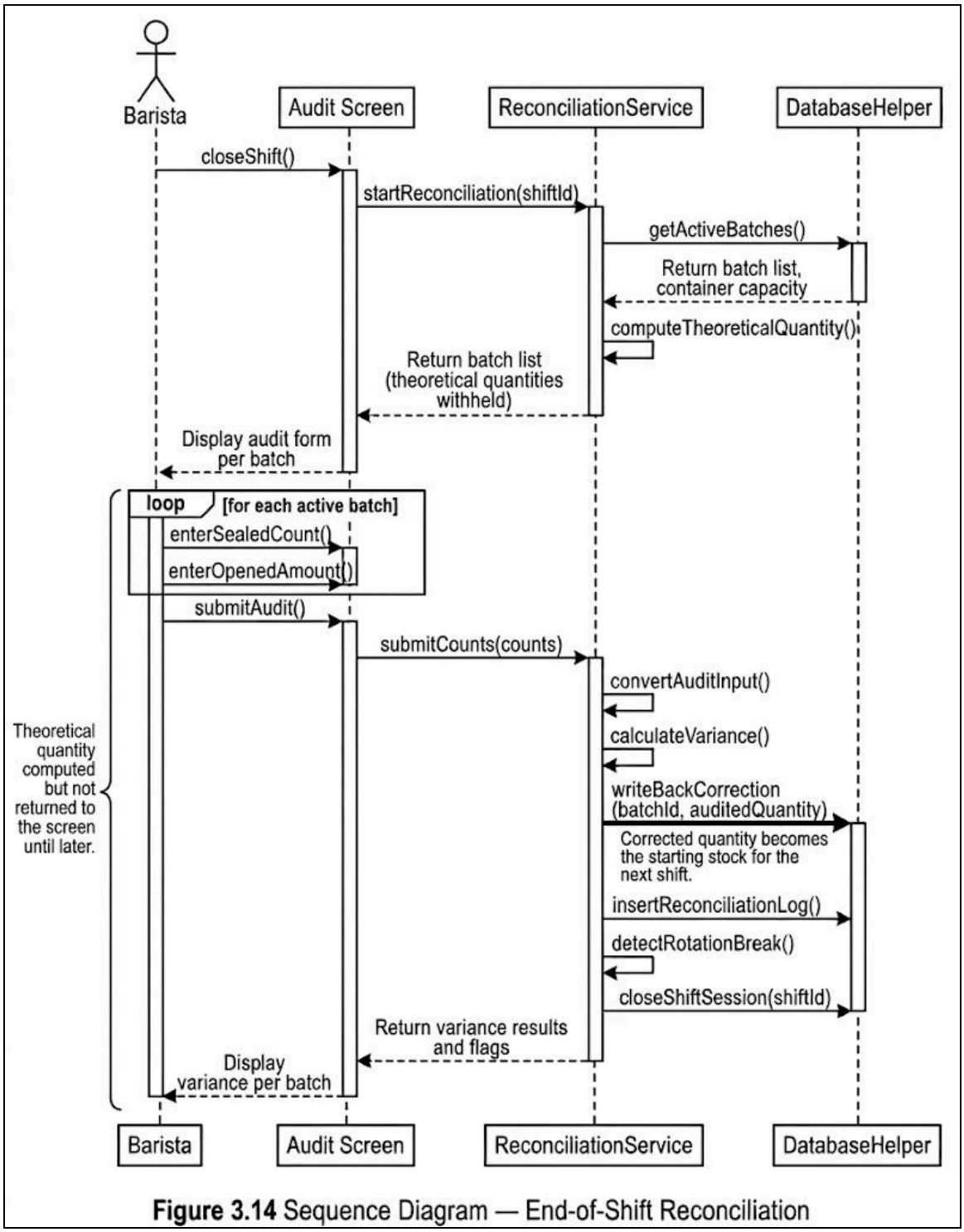


Figure 3.14 Sequence Diagram — End-of-Shift Reconciliation

Figure 3.14 Sequence Diagram (Reconciliation)

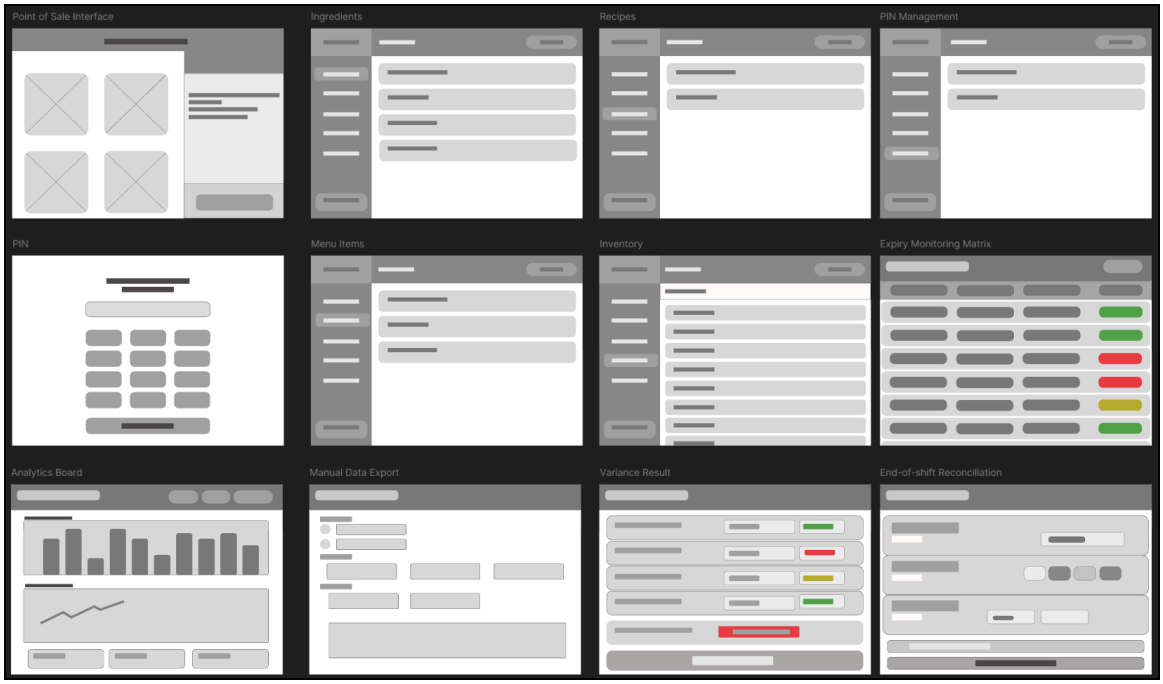


Figure 3.14 BrewManager Wireframes

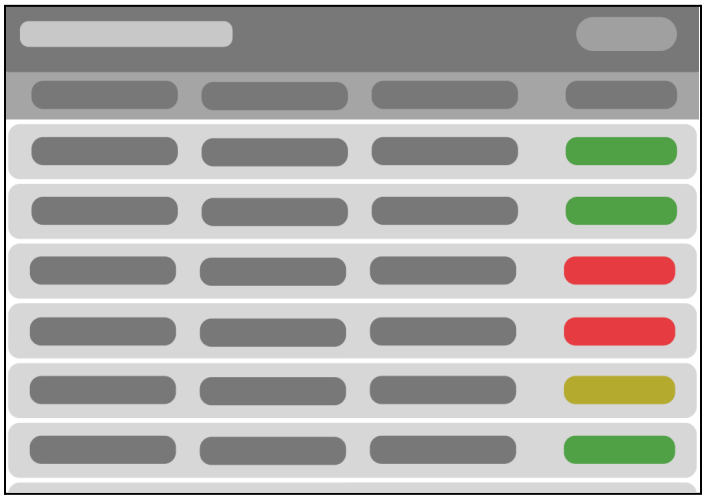


Figure 3.15 Expiry Monitoring Matrix Interface

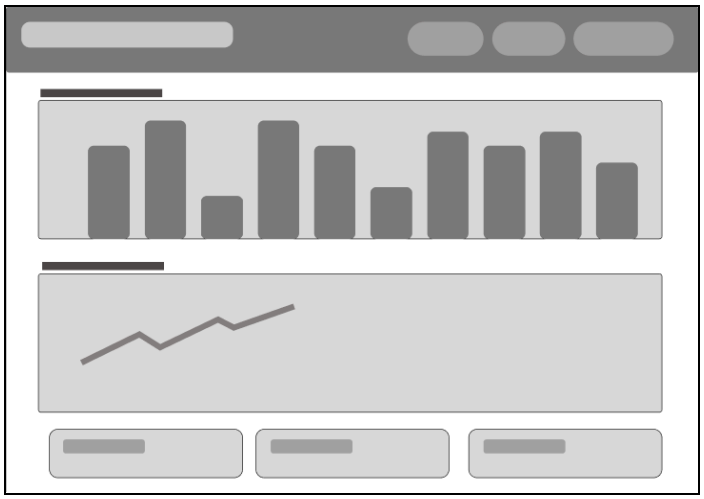


Figure 3.16 Analytics Dashboard Interface

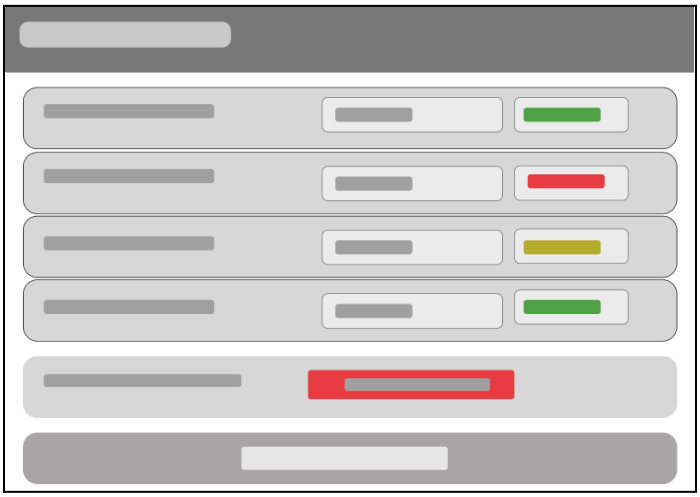


Figure 3.17 Variance Result Interface

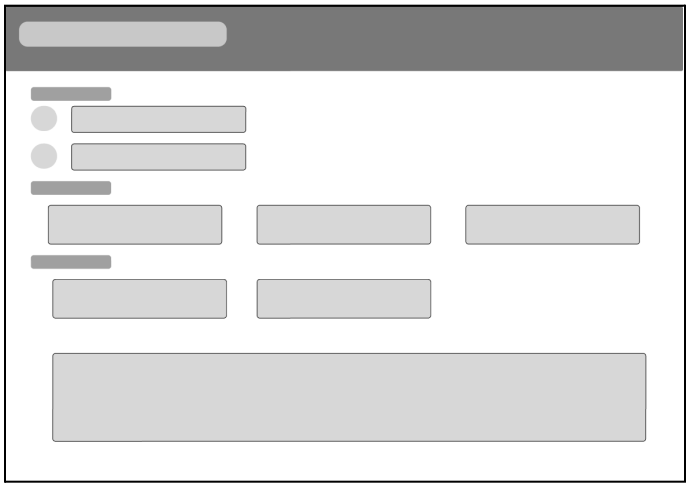


Figure 3.18 Manual Data Export Interface

Requirement	Minimum Specification
Device type	Android tablet
Android version	Android 8.0 (Oreo) or higher
Processor	Quad-core 1.5 GHz or higher
RAM	3 GB or higher
Internal storage	16 GB with at least 2 GB free
Screen size	9 inches or larger (landscape orientation)
Internet connection	Not required for daily operations
External storage	Optional, for manual CSV/PDF data exports

Table 3.11 Minimum Hardware Requirements Table

Tool	Version / Detail
Flutter SDK	3.x or higher
Dart	3.x or higher
SQLite (via sqflite package)	Latest stable
Android Studio	Latest stable
Target SDK	Android API Level 26 or higher
Minimum SDK	Android API Level 21 (Android 5.0)

Table 3.12 Minimum Software Requirements Table